



UNIVERSIDAD POLITÉCNICA DE  
MADRID

**HIGHER TECHNICAL SCHOOL OF  
COMPUTER ENGINEERS.**



EUROPEAN MASTER IN SOFTWARE ENGINEERING

Master thesis

**Segwork: A framework for Semantic  
Segmentation.**

ÁLVARO RUBIO SEGOVIA

July, 2022

Master Thesis

European Master in Software Engineering

Title: Segwork: A framework for Semantic Segmentation.  
July, 2022

Author: Álvaro Rubio Segovia

| Supervisor                                                                                                                                                                        | Co-supervisor                                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Angélica de Antonio Jiménez<br>Associate professor<br>Universidad politécnica de Madrid<br>Higher Technical School of Computer<br>Engineers.<br>Universidad politécnica de Madrid | Julián Cabrera Quesada<br>Associate professor<br>Universidad politécnica de Madrid<br>Escuela Técnica Superior de Ingenieros<br>de Telecomunicación<br>Universidad politécnica de Madrid |

## Abstract

Artificial intelligence is a growing field that has attracted the attention of many researchers and engineers because of its capabilities. These capabilities have been enhanced by the increase in computational power (ability to run in optimal devices) and data availability.

Computer vision is a powerful and compelling type of AI applied to a wide range of situations today. Self-driving cars or cancer detection are among the most influential and useful applications of computer vision. There is a set of more or less well-defined computer vision tasks that include methods to process and understand digital images and create meaningful information. Among them is semantic segmentation.

Semantic segmentation is the task of classifying each pixel in an image among a limited set of object classes. During the last year, there has been great improvement in the systems developed to perform this task. This system uses deep learning, another type of Artificial Intelligence discipline focused on deep neural networks.

This work explores the state-of-the-art methods (and related software ecosystem) of semantic segmentation for supervised training and provides a framework for building, training, and evaluating models. This framework is made up of existing mature tools and complementary software artifacts.

Pytorch is selected as the base deep learning framework. It is based on four major trends in the deep learning community: array-based programming, automatic differentiation to automate the calculation of derivatives, open source ecosystem, and hardware accelerators.

Pytorch Ignite is proposed as the base tool for training and evaluating the models because of its flexibility and simplicity, in addition to its ever-growing set of functionalities for handling events and logging results. YACS is proposed as a configuration system; deep learning experimentation involves a great number of variables in their experiment; a system to control such variables is essential for reproducibility and clarity.

More specific to the semantic segmentation task is the tool proposed for rapid experimentation with out-of-the-box models. Segmentation Pytorch is a mature Github repository with several awards that will speed up model building, providing more than one hundred backbones and nine configurable model architectures.

This set of tools are complemented by three main components developed under the name of Segwork. A configurable class registry, used as a factory method. Ablation studies in which components are regularly removed and modified need a method to avoid hardcoding of every component. This functionality is essential to allow flexible experimentation from the configuration system.

A segmentation data set structure is proposed using an abstract class. This class is equipped with several class weighting algorithms. Semantic segmentation datasets are usually imbalanced due to the huge amount of pixels that belong to background or similar classes. Class balancing is a recurrent practice, supported by flexible classes

in Segwork.

Dealing with a custom dataset usually requires a lot of boilerplate to preprocess the segmentation masks or parse data files from JSON/XML format. Segwork proposed two transformations to convert color segmentation masks to digestible Numpy index masks using a color map.

This master thesis, Segwork and the tools proposed, aimed to serve as an introduction for experimentation with deep learning based semantic segmentation systems.

# Contents

|          |                                                                                                 |           |
|----------|-------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                                             | <b>1</b>  |
| 1.1      | Scope and limitations . . . . .                                                                 | 2         |
| 1.2      | Motivation . . . . .                                                                            | 2         |
| 1.3      | Objectives and contributions . . . . .                                                          | 2         |
| 1.4      | Thesis structure . . . . .                                                                      | 3         |
| <b>2</b> | <b>Literature review</b>                                                                        | <b>4</b>  |
| 2.1      | Fundamentals of computer vision . . . . .                                                       | 4         |
| 2.1.1    | Computer vision & machine learning . . . . .                                                    | 5         |
| 2.1.2    | Computer vision tasks . . . . .                                                                 | 7         |
| 2.2      | Semantic segmentation . . . . .                                                                 | 8         |
| 2.2.1    | Deep learning for Semantic segmentation . . . . .                                               | 9         |
| 2.2.2    | Semantic segmentation metrics . . . . .                                                         | 32        |
| 2.2.3    | Semantic segmentation datasets and benchmarks . . . . .                                         | 34        |
| 2.2.4    | Train and test a semantic segmentation model . . . . .                                          | 35        |
| <b>3</b> | <b>A deep learning ecosystem for semantic segmentation</b>                                      | <b>37</b> |
| 3.1      | Pytorch - An open source machine learning framework . . . . .                                   | 37        |
| 3.1.1    | Overview of Pytorch . . . . .                                                                   | 38        |
| 3.1.2    | Pytorch packages . . . . .                                                                      | 38        |
| 3.1.3    | Vanilla Pytorch limitations . . . . .                                                           | 40        |
| 3.2      | A framework for segmentation using Pytorch - <code>segmentation_models.pytorch</code> . . . . . | 40        |
| 3.3      | Speeding-up the training and evaluation - Ignite . . . . .                                      | 41        |
| 3.4      | A configuration system for experiments - YACS . . . . .                                         | 42        |
| 3.5      | Conceptual framework for semantic segmentation . . . . .                                        | 42        |
| 3.5.1    | Basic layers . . . . .                                                                          | 43        |
| 3.5.2    | Segmentation models . . . . .                                                                   | 44        |
| 3.5.3    | Data pipeline . . . . .                                                                         | 46        |
| 3.5.4    | Training process . . . . .                                                                      | 46        |
| 3.5.5    | Evaluation process . . . . .                                                                    | 47        |
| 3.5.6    | Configuration system . . . . .                                                                  | 47        |
| <b>4</b> | <b>Improving the semantic segmentation ecosystem</b>                                            | <b>50</b> |
| 4.1      | A repository of modular components - <code>ConfigurableRegistry</code> . . . . .                | 50        |
| 4.2      | A dataset for semantic segmentation - <code>SegmentationDataset</code> . . . . .                | 52        |
| 4.2.1    | Class balancing with weights . . . . .                                                          | 53        |
| 4.3      | Parsing color mask to index masks . . . . .                                                     | 53        |

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>5 Usage of the framework</b>                         | <b>55</b> |
| 5.1 Development environment . . . . .                   | 55        |
| 5.2 How to use it . . . . .                             | 55        |
| 5.2.1 Configurable registry . . . . .                   | 56        |
| 5.2.2 SegmentationDataset and WeightAlgorithm . . . . . | 59        |
| 5.2.3 Numpy mask generator and mask transform . . . . . | 60        |
| <b>6 Conclusions</b>                                    | <b>63</b> |
| 6.1 Contributions . . . . .                             | 63        |
| 6.2 Future perspectives . . . . .                       | 64        |
| <b>Bibliography</b>                                     | <b>65</b> |
| <b>Appendix</b>                                         | <b>72</b> |
| Notebooks . . . . .                                     | 72        |
| Pycharm Diagrams . . . . .                              | 100       |

# List of Figures

|                                                                                                          |    |
|----------------------------------------------------------------------------------------------------------|----|
| Figure 2.1: Overview of Deep learning for computer vision . . . . .                                      | 6  |
| Figure 2.2: Traditional computer vision workflow vs Deep learning workflow                               | 7  |
| Figure 2.3: Overview of image segmentation task . . . . .                                                | 8  |
| Figure 2.4: Overview of the Semantic Segmentation process . . . . .                                      | 9  |
| Figure 2.5: An illustration of the architecture of AlexNet . . . . .                                     | 12 |
| Figure 2.6: An illustration of the architecture of VGG-16. . . . .                                       | 13 |
| Figure 2.7: Inception V1 module. . . . .                                                                 | 14 |
| Figure 2.8: Spatial variations of convolution . . . . .                                                  | 15 |
| Figure 2.9: Inception V2 modules . . . . .                                                               | 16 |
| Figure 2.10: Spatially separable convolution . . . . .                                                   | 17 |
| Figure 2.11: Inception module that reduces the grid-size and avoids representational bottleneck. . . . . | 18 |
| Figure 2.12: Residual learning building block. . . . .                                                   | 19 |
| Figure 2.13: Residual learning building block for CNNs. . . . .                                          | 20 |
| Figure 2.14: ResNext and Xception modules comparison. . . . .                                            | 21 |
| Figure 2.15: Depthwise separable convolutions . . . . .                                                  | 22 |
| Figure 2.16: MobileNetv1 and MobileNetv2 blocks. . . . .                                                 | 23 |
| Figure 2.17: Grouped convolutions. . . . .                                                               | 24 |
| Figure 2.18: Abstract illustration of NAS methods. . . . .                                               | 24 |
| Figure 2.19: Three dimensions used for model scaling in EfficientNet method.                             | 25 |
| Figure 2.20: Three dimensions used for model scaling in EfficientNet method.                             | 26 |
| Figure 2.21: FCN architecture overview with skip connections. . . . .                                    | 26 |
| Figure 2.22: An illustration of the Encoder-Decoder architecture . . . . .                               | 27 |
| Figure 2.23: An illustration of the PSPNet . . . . .                                                     | 28 |
| Figure 2.24: Strategies to fuse depth and RGB information. . . . .                                       | 30 |
| Figure 2.25: An illustration of the FuseNet . . . . .                                                    | 31 |
| Figure 3.1: Convolutional layers types and Pytorch class mapping . . . . .                               | 43 |
| Figure 3.2: Pooling layers types and Pytorch class mapping . . . . .                                     | 44 |

|                                                                              |     |
|------------------------------------------------------------------------------|-----|
| Figure 3.3: Non-linear activation layers types and Pytorch class mapping . . | 44  |
| Figure 3.4: Normalization layers types and Pytorch class mapping . . . . .   | 45  |
| Figure 3.5: Backbone / segmentation model and class mapping . . . . .        | 45  |
| Figure 3.6: Data pipeline for semantic segmentation tasks . . . . .          | 46  |
| Figure 3.7: Training data flow using the proposed framework . . . . .        | 47  |
| Figure 3.8: Evaluation process proposed in the framework . . . . .           | 48  |
| Figure 3.9: Metrics and logger pipeline . . . . .                            | 48  |
| Figure 3.10: Configuration system proposed in the framework . . . . .        | 49  |
| Figure 5.1: Unique key error in ConfigurableRegistry . . . . .               | 56  |
| Figure 5.2: models_reg registry status print . . . . .                       | 58  |
| Figure 5.3: Wrong class_key error . . . . .                                  | 58  |
| Figure 5.4: Format of colormap for transformations . . . . .                 | 61  |
| Figure 6.1: Diagram of registry.py from Pycharm . . . . .                    | 100 |
| Figure 6.2: Diagram of dataset.py from Pycharm . . . . .                     | 101 |
| Figure 6.3: Diagram of balance.py from Pycharm . . . . .                     | 102 |
| Figure 6.4: Diagram of transform.py from Pycharm . . . . .                   | 103 |

# Chapter 1

## Introduction

### Contents

---

|                                            |   |
|--------------------------------------------|---|
| 1.1 Scope and limitations . . . . .        | 2 |
| 1.2 Motivation . . . . .                   | 2 |
| 1.3 Objectives and contributions . . . . . | 2 |
| 1.4 Thesis structure . . . . .             | 3 |

---

Deep learning has become a well-known term for everyone in the field of computer science, with many achievements in recent years. Integration of GPU's into the computational processes of deep learning and thus higher computational power is one of the reasons why this has been possible. A MIT study in 2020 claimed that computing power ended the "AI winter"<sup>1</sup>, but the breakthrough will soon reach another limit, especially in an era with slow improvements in hardware performance [1].

These limitations are pushing the deep learning field towards more computationally-efficient algorithms. Advances in the field of deep learning encourage the creation of a wide environment of tools for research. Pytorch<sup>2</sup> from MetaAI<sup>3</sup> or OpenMMLab<sup>4</sup> are widely used open-source frameworks to reduce research and production deployment time.

Getting to know the rich ecosystem of deep learning tools is a key requirement for effective and rapid research, as complex mathematical concepts are already implemented and optimized. For example, Pytorch can set up and run CUDA operations and implements the basic operations of artificial neural networks.

---

<sup>1</sup>Historic term used for periods of time with reduced investment and interest on Artificial Intelligence (AI)

<sup>2</sup>Open source deep learning framework built to be modular and flexible for research

<sup>3</sup>Formerly known as Facebook

<sup>4</sup>Most influential open-source computer vision framework for deep learning

## 1.1 Scope and limitations

This Master's thesis provides a basic glossary of deep learning terminology and techniques; although some definitions are provided, in-depth explanations must be encountered in the citations. Deep learning has been a major advance in terms of performance, and many researchers have tried to apply deep learning to computer vision. As a result many research directions have been explored, such as the application of deep learning to different tasks, or the improvement of deep learning algorithms based on supervised training, semi-supervised training, and unsupervised training.

Due to time limitations, this Master's thesis focuses on supervised end-to-end training of fully deep convolutional networks for the semantic segmentation task. Out of scope lay unsupervised and semi-supervised training routines and models, two-stage models that cannot be trained end-to-end, or the application of Visual transformer (ViT) to semantic segmentation. These topics will be explored in future work.

## 1.2 Motivation

As part of my internship, I have collaborated on the development of an assistive solution for visually impaired or blind people. We performed requirements engineering processes, such as requirement elicitation, and found that a computer vision module to improve the understanding of the user scene and scan the environment was a key requirement. This Master's thesis is a continuation of the work done in the internship and aims to be used in the assistive system as the main framework to build the scene understanding system.

In addition, deep learning and computer vision is a growing research field, which makes it appealing to any software engineer. Two main factors are responsible for this growing attention: the accessibility of data and the increase in computational processing power, especially with GPUs. When "Semantic segmentation" is used as a search keyword in Scopus, it shows 18,273 document results<sup>5</sup>. The number of published researches grows year by year (see figure). This growing interest has helped to improve the number and quality of applications over the years.

Finally, I want to test the skills acquired in the European Master of Software Engineering in a real situation. where tools already exist and the process to build a software solution involves the analysis of current tools, the elicitation of requirements, and the design of flexible and robust software using third-party software.

## 1.3 Objectives and contributions

This work aims to design, describe and implement a framework that facilitates the research of semantic segmentation and serves as a simple guide for building, training,

---

<sup>5</sup>Date: 16/06/2022

and evaluating semantic segmentation models. Thus, an analysis of the current state of semantic segmentation models and research directions has to be performed. The framework does not intend to reinvent the wheel; instead, it will propose a technology stack along with additional artifacts specialized in semantic segmentation to reduce the time required for research and development<sup>6</sup>.

The secondary objectives that derive from the principal goal are as follows:

- Serve as an introduction to semantic segmentation.
- Provide a detailed overview of SOTA semantic segmentation models and practices for supervised training approaches and CNN based models.
- Provide tools to improve efficiency and reduce complexity.
  - Reduce the amount of effort spent building and training models.
  - Simplify tasks and avoid common errors (following good practices in deep learning).

## 1.4 Thesis structure

The Master's thesis is structured in five parts, listed below:

1. Introduction.
2. Literature review. An overview of semantic segmentation research for supervised training and CNN-based models.
3. A deep learning ecosystem for semantic segmentation. Proposed technology stack for experimentation.
4. Improvements in the semantic segmentation ecosystem. The design of the software artifacts aimed to improve the existing tools.
5. Usage of the framework. In this section, this Master's thesis describes how to use the additional software tools of Segwork.
6. Conclusions.

---

<sup>6</sup>The artifacts are packaged under the name "Segwork" and are stored in a github repository: <https://github.com/ARubiose/segwork>

# Chapter 2

## Literature review

### Contents

---

|       |                                                         |    |
|-------|---------------------------------------------------------|----|
| 2.1   | Fundamentals of computer vision . . . . .               | 4  |
| 2.1.1 | Computer vision & machine learning . . . . .            | 5  |
| 2.1.2 | Computer vision tasks . . . . .                         | 7  |
| 2.2   | Semantic segmentation . . . . .                         | 8  |
| 2.2.1 | Deep learning for Semantic segmentation . . . . .       | 9  |
| 2.2.2 | Semantic segmentation metrics . . . . .                 | 32 |
| 2.2.3 | Semantic segmentation datasets and benchmarks . . . . . | 34 |
| 2.2.4 | Train and test a semantic segmentation model . . . . .  | 35 |

---

The literature review explores what has already been written on semantic segmentation based on deep learning and supervised training and provides an overview of this broad topic. First, this thesis reviews core concepts such as computer vision, machine learning, and semantic segmentation, making relationships among them. It introduces deep convolutional neural networks and how they have evolved over the past few years to improve performance and efficiency.

It follows with a description of the evolution of semantic segmentation models for supervised training and their core components: feature extractors, metrics used to evaluate those models, dataset structure, and the core ideas behind the training and the evaluation processes.

\* \* \*

### 2.1 Fundamentals of computer vision

The term AI has probably been read or heard by all the readers of this master's thesis. It has been the subject of many investigations over the past 70 years and is present in many situations in our daily lives. Alan Mathison Turing, a prominent figure in computer science, introduced the first notion of AI in 1950 in his publication

Computing Machinery and Intelligence [2]. The paper starts with the sentence "I PROPOSE to consider the question 'Can machines think?'"

An imitation game, known as the Turing test<sup>1</sup>, is proposed to answer this question. To pass the test successfully, the machine must pretend to be a human to a knowledgeable observer. That is, the system must be able to communicate in a natural language (natural language processing), have and retain knowledge (knowledge representation), perform knowledge-based reasoning (automated reasoning) and learn from its environment (machine learning) [3]. These characteristics of a system are what people associate with intelligence in human behavior, that is, being "intelligent".

Although the previous test could be used to provide a definition, the concrete definition of AI has generated discussions and different definitions over the past decades. John McCarthy, called the father of AI<sup>2</sup> [4], argues that the Turing test is a one-sided test. A machine that passes the test should undoubtedly be considered intelligent, but a machine that does not know enough about people to impersonate a human could still be called intelligent [5].

John McCarthy defines AI as the science and engineering of making intelligent machines, especially intelligent computer programs. In this definition, intelligence for machines or programs is the varying computational portion of the ability to achieve goals, such as accurate predictions or classifications. Notice that AI does not limit itself to biologically observable methods, it may involve much more computation than a human-like method.

This wide range of possibilities has supported the application of AI in different domains, such as personalized shopping and fraud prevention in e-Commerce or autonomous vehicles, and facial recognition in lifestyle. The last two applications fall under the umbrella of Computer Vision (CV), fast pace and growing research topic strongly related to AI in the last decade. Computer vision (CV) aims to provide machines with human-like vision capabilities.

### **2.1.1 Computer vision & machine learning**

Computer vision is not a new field; research was born in the late 1950s, when neurologists began investigating the correlation of brain responses to visual input in cats [6]. For humans, recognizing an animal through subtle patterns in shape and light is an easy task learned in the first years of life. However, modeling the visual world and its complexity is a tough task for computers. Researchers have spent decades trying to understand how the visual system works and recovering the three-dimensional form and appearance of objects in images.

CV captures the world we see in more than one image and reconstructs its properties, such as shape, light, and color distribution. According to the International Business Machines Corporation (IBM)<sup>3</sup>, computer vision is an interdisciplinary scientific subfield

---

<sup>1</sup>A method of determining whether or not a computer is capable of thinking like a human being.

<sup>2</sup>John McCarthy coined the term artificial intelligence and established the orientation of the field

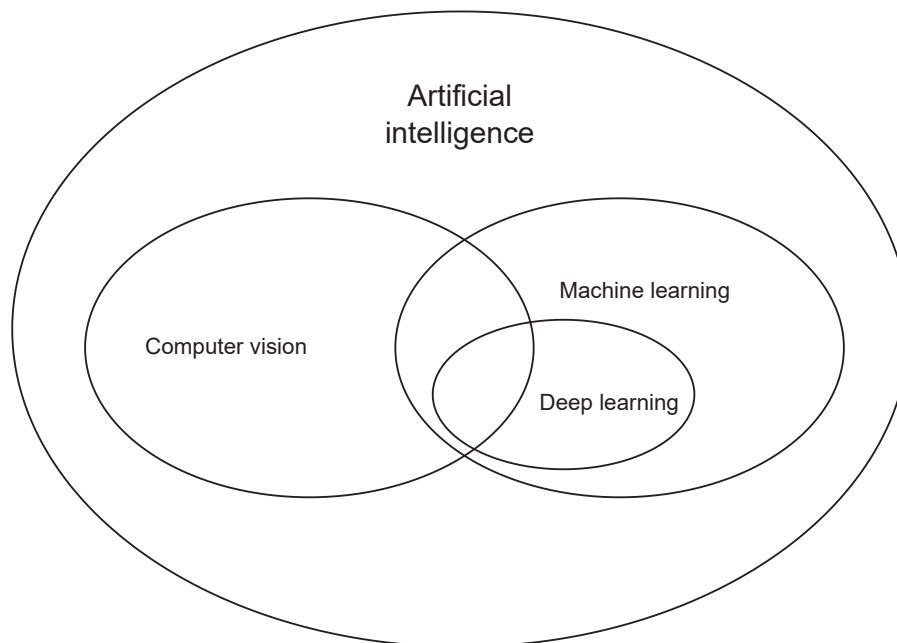
<sup>3</sup>International Business Machines Corporation (IBM) is an American multinational technology corporation

of ai that allows computers and systems to derive meaningful information and high-level understanding from videos, images, and other visual input [7].

Looking at the previous definition, CV differs from digital image processing in the desire to use the reconstructed properties of images as a stepping stone towards complete scene understanding [8] and more complex problems such as facial recognition. However, building a visual model of a scene is neither easy nor practical for many computer vision applications. To facilitate the building of the model, machine learning is applied. Machine learning is a data analysis method that automates the construction of analytical models.

Machine learning is another subfield of AI that has been closely related to computer vision. Machine learning complements computer vision in two different ways. Improving the perception of the environment, that is, improving the transformation of visual signals into internal representations and improving the association between internal representation and the representation of knowledge required by the system to perform its task. Thus, machine learning has improved computer vision, and computer vision has broadened the scope of machine learning [9].

In the machine learning literature, the internal representation consists of features. Christopher Bishop<sup>4</sup> defines feature as an individual measurable property or characteristic of a phenomenon being observed. The way machine learning algorithms use these features has generated two generations of innovation in CV.



**Figure 2.1:** Overview of Deep learning for computer vision. Figure inspired from University of Michigan course [10].

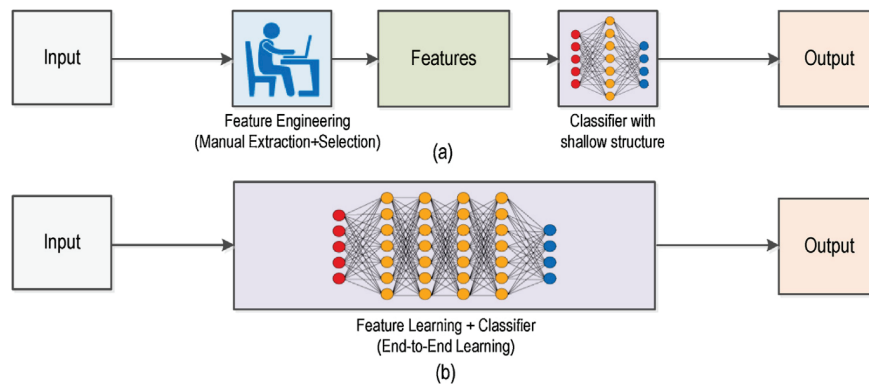
In the first stages of computer vision with machine learning, the dominant approach followed was based on hand-crafted features. The extraction of features was a manual process that required domain knowledge of the data we learned from. Thus, a type of

---

<sup>4</sup>Laboratory Director at Microsoft Research Cambridge, Honorary Professor of Computer Science at the University of Edinburgh and a Fellow of Darwin College, Cambridge

machine learning in which learning is based on predefined features. These features were obtained through well-established CV techniques such as scale invariant feature descriptor (SIFT) [11] or Speeded-Up Robust Features (SURF), algorithms which takes an image and outputs a feature vector. These algorithms are called feature descriptors.

The complexity of the traditional approach comes with the selection of the features that best describe different classes of objects in the images. As the number of classes increases, the set of best features becomes difficult to obtain through a long trial-and-error process [12]. To address this problem, Deep Learning (DL) introduces the concept of end-to-end learning (see figure 2.2), where the most descriptive and important features or underlying patterns in each class are automatically discovered (representation learning). DL attempts to learn multiple levels of abstraction from data using hierarchical architectures based on artificial neural networks [13].



**Figure 2.2:** (a) Traditional computer vision workflow vs (b) Deep learning workflow. Figure from [14].

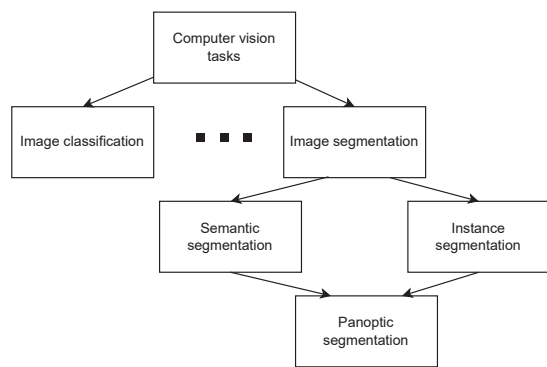
The limits of what was possible in computer vision were pushed by the application of deep learning in computer vision. In recent years, DL has dominated the domain due to substantially better performance compared to traditional methods in many computer vision tasks. This dominance was facilitated by an increase in computational power and data availability to train deep neural networks. The main advantage of deep learning is that it can learn to represent data at multiple levels of abstraction using computational models called deep neural networks.

### 2.1.2 Computer vision tasks

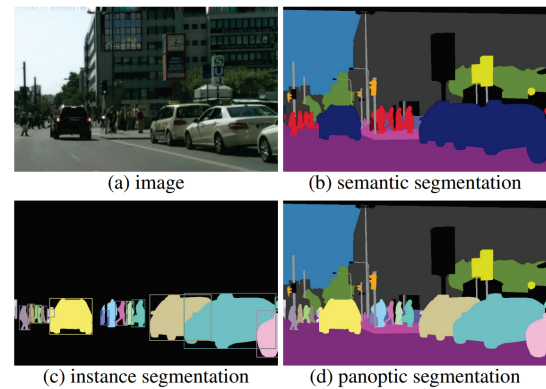
Today, deep learning-based computer vision is applied in many areas, such as automated inspections in manufacturing applications [15], navigation assistance for autonomous vehicles and visually impaired people [16], or cancer diagnosis in medicine and healthcare [17]. Each of these application areas uses a range of computer vision tasks to achieve successful results. Although task is a very general word, there is a set of more or less well-defined computer vision tasks. These tasks include methods for processing and understanding digital images to produce meaningful information that can elicit appropriate actions. Motion analysis, scene reconstruction, or object recognition are examples of these tasks.

Recognition of visual objects is an essential vision task in every animal's life. When we look through the window, we quickly identify the road, cars, and transients, among others. Thus, one of the most studied sets of computer vision tasks is object recognition, the identification of an object/s in an image. There are different schemes with respect to digital object recognition. For example, image classification outputs a label prediction of an object present in the image, and object detection locates and identifies objects present in an image [18].

Image segmentation has been a fundamental task in computer vision since the early stages. It consists of clustering similar parts of an image under meaningful labels. Depending on how it is formulated, it can be divided into semantic segmentation, classifying pixels with semantic labels, instance segmentation, partitioning of individual objects, and most recently panoptic segmentation, which includes both problems [19] (See figure 2.3).



(a) Diagram of semantic segmentation within Computer vision tasks.



(b) Ground truth image and output of related computer vision tasks. Figure from [20].

**Figure 2.3:** Overview of image segmentation task.

These labels are represented in an image segmentation task, object recognition at the pixel level, that is, produce semantic labels for each pixel. The family of computer vision tasks that learn to map at the pixel level, such as semantic segmentation or depth estimation, is called dense predictions [21], and it is a natural step in the progression from coarse to fine inference.

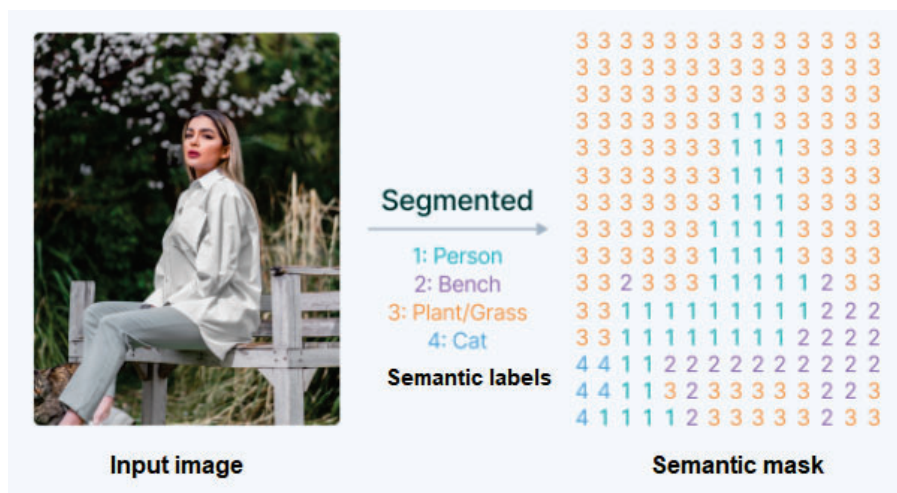
## 2.2 Semantic segmentation

Semantic segmentation is a challenging task in computer vision, used in a great variety of applications. In self-driving cars it is critical to comprehend the scenario [22] and useful to achieve efficient farming in agricultural regions through aerial inspection [23].

Semantic segmentation refers to the problem or computer vision tasks of classifying each pixel in an image among a limited set of object classes. It does not distinguish between instances of the same class. For example, if there are two cars in the image, the output label of the model for the pixels in both cars would be the same. Traditional segmentation algorithms, such as conditional and Markov random fields [24] have

become obsolete with the arrival of deep learning, which has produced a paradigm shift that achieves noticeable performance improvements.

The goal of semantic segmentation is to produce a segmentation mask, that is, to segment the input image according to semantic information and classify each pixel within a finite set of semantic categories or class labels. Images are usually represented with a 2D matrix (grey-scale) or a 3D matrix (RGB) containing values from 0 to 255 (8-bit) for light intensity. The segmentation mask is a 2D matrix with the same height and width as the input image containing values from 0 to  $N - 1$ , where  $N$  is the number of class labels.



**Figure 2.4:** An overview of the Semantic Segmentation process. Note class labels lie within 1 and  $N$ . Adaptation from [v7labs.com](https://v7labs.com).

### 2.2.1 Deep learning for Semantic segmentation

Deep learning is rapidly growing, and there has been so much progress in semantic segmentation in recent years that it is impossible for this master's thesis to capture the many facets and subdomains of semantic segmentation that are tackling the most pertinent problems in image segmentation today. Instead, this Master's thesis provides a brief description of the main breakthroughs on this topic over the past 10 years in deep learning-based models that have led to new research directions. These advances are widely used today. The application of these proven practices would be the foundation of the framework.

Before the deep learning revolution in computer vision emerged, semantic image segmentation models were based on hand-crafted low-level<sup>5</sup> features to locate object boundaries by thresholding or clustering. However, object occlusion usually results in poor performance [26]. In addition, this approach is probably based on the false assumption that a human designer knows all the relevant information for the task.

Another approach in the past observed the dependencies between adjacent pixels to

<sup>5</sup>Low-level features include edges and blobs based on image processing techniques. High-level features include objects or events based on machine learning techniques [25].

find an inference. This approach used graphical models such as forest-based methods or Markov Random Fields (MRF), state-of-the-art techniques for multi-class image segmentation before the deep learning era [27]. These types of models were used in some deep learning models (DeepLab [28]) as refinement methods (post-processing) to improve performance. However, the use of graphical models for refinement has lost popularity because they are slow and very difficult to optimize on large-scale data [29].

#### **2.2.1.1 Automatic feature extraction with convolutional neural networks**

The application of Convolutional Neural Networks (CNN, also called ConvNets) to extract features from visual input in CV has been a key research topic and the main solution in recent years. The performance of semantic segmentation systems is heavily influenced by the feature extraction approach. In the literature, the feature extractor usually receives the name: "backbone". Convolutional layers have much fewer connections and parameters than dense layers, so they are easier to train on high-dimensional input, such as image data [30]. In addition, they are designed to be applied to images [31]:

- They provide more flexibility in the receptive field (local connectivity), a crucial issue in visual tasks [32].
- Elements / features in different positions does not alter its semantic content (Spatial invariance).
- They can learn features hierarchically, from simple features to complex features. Intermediate representations of features are called features map .

For a great visualization and further exploration of convolution arithmetic for deep learning, I recommend the guide published by Vincent Dumolin and Francesco Visin [33].

CNNs were first introduced in 1998 by LeCun [34] who developed a model to recognize digits and zip codes, showing promising performance. It was the first step toward automatic feature extraction; this article showed CNN's ability to eliminate the need for hand-crafted feature extractors. However, memory and hardware limitations, as well as the inaccessibility of large training data sets, delayed progress on this approach for some years.

In 2010, with progress in hardware and memory, LeCun et al. [35] proposed a deeper net to learn multi-level hierarchies of features and set the path of deep learning research on computer vision tasks to find methods that can automatically learn good feature hierarchies. Since then, many models have been published. The number of variations is overwhelming; this section provides a description of the most influential and the challenges they face.

#### **2.2.1.1.1 Deep Convolutional Neural Networks for feature extraction - Alexnet**

In 2012, The "ImageNet Classification with Deep Convolutional Neural Networks" paper [30] introduced a deeper architecture with some key components for feature extraction called Deep Convolutional Neural Networks (DCNN). This architecture meant a great advance in computer vision and semantic segmentation, proving significant improvements from previous methods due to its learning capacity and low error.

This architecture was proposed by Alex Krizhevsky et al. of the University of Toronto to compete in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC)<sup>6</sup> and is considered one of the most influential papers in computer vision based on deep learning. It turned the mainstream solution towards CNNs.

The model, called "AlexNet", contains eight layers, five convolutional layers, and three fully connected or dense layers (see figure 2.5). It uses Rectified Linear Units (ReLU) as the activation function to model the output of neurons and shows that it reduces the training time compared to the equivalents (tanh) in CNNs. Non-saturating nonlinearities (ReLU) are much faster than saturating nonlinearities (Sigmoid) in terms of training time [36].

This neural network architecture contains 60 million trainable parameters that increase the risk of overfitting, the main concern when a model with a large number of parameters is trained. To address this problem, several mechanisms were introduced:

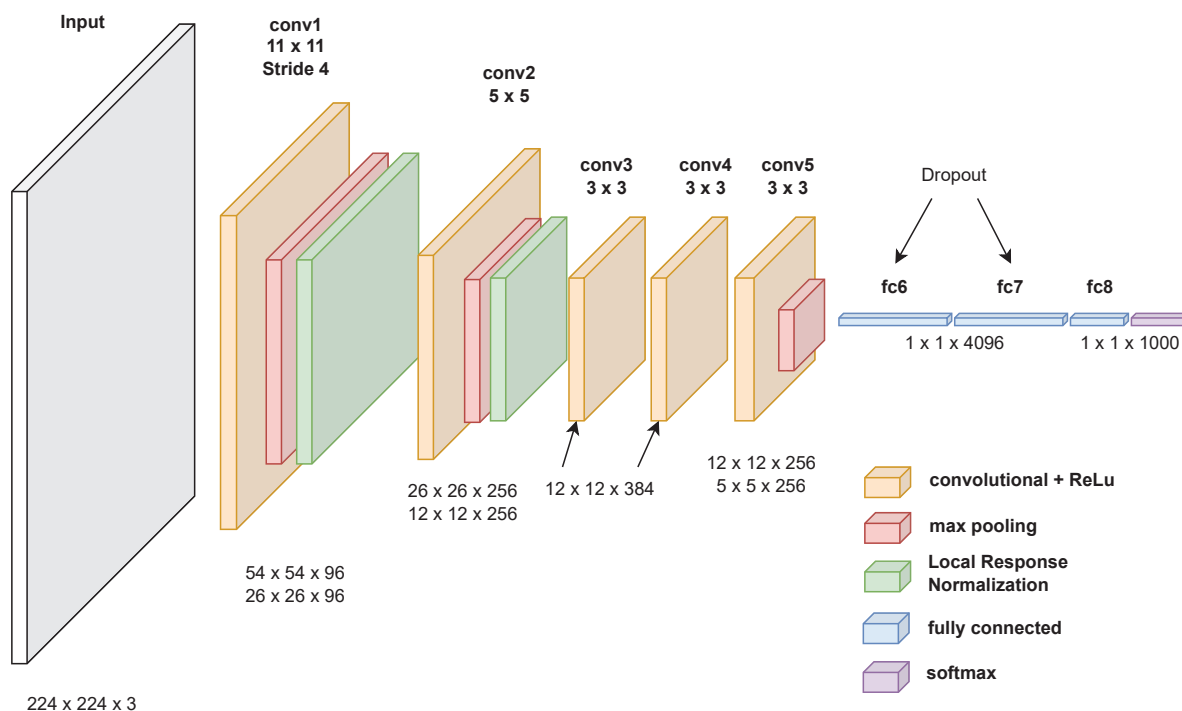
- Despite a non-saturating activation function is used, the first and second convolutional layers are followed by the response-normalization layers, improving generalization.
- The first, second, and fifth convolutional layers are followed by overlapping pooling layers, making the model slightly more difficult to overfit and reducing computational requirements.
- Data augmentations with label-preserving transformations, which are widely used in SOTA models these days. These transformations are image translation and horizontal reflections. This trick has been shown to increase performance [37].
- Dropout to prevent complex co-adaptations [38]. It is a method in which half of the feature detector are randomly omitted in each training case.

#### **2.2.1.1.2 Understanding why and how DCNNs are good for visual feature extraction - ZFNet**

A year later (2013), a modified version of Alexnet called ZFNet was proposed, extending the CNN investigation and improving accuracy on ILSVRC [39]. The paper focused on understanding how convolutional layers work in DCNNs and the research question of

---

<sup>6</sup>Well-known image classification competition with 10,000,000+ labeled images depicting 10,000+ object categories as training dataset.



**Figure 2.5:** An illustration of the architecture of AlexNet. GPU optimizations are omitted to simplify the model architecture. <sup>7</sup>

how they can be improved by proposing a novel way to visualize the activity within the model.

The experiment showed that CNNs are capable of capturing desirable properties, such as compositionality<sup>8</sup>, and increasing spatial invariance as we go deeper into the net. They also showed that trained models can generalize well to other datasets.

ZFNet tried to solve some of Alexnet's apparent problems aided by its visualization technique using their visualization technique. They reduced the first-layer filter size to  $7 \times 7$  and changed the stride to 2 to cope with the aliasing artifacts shown in the visualization of the second layer.

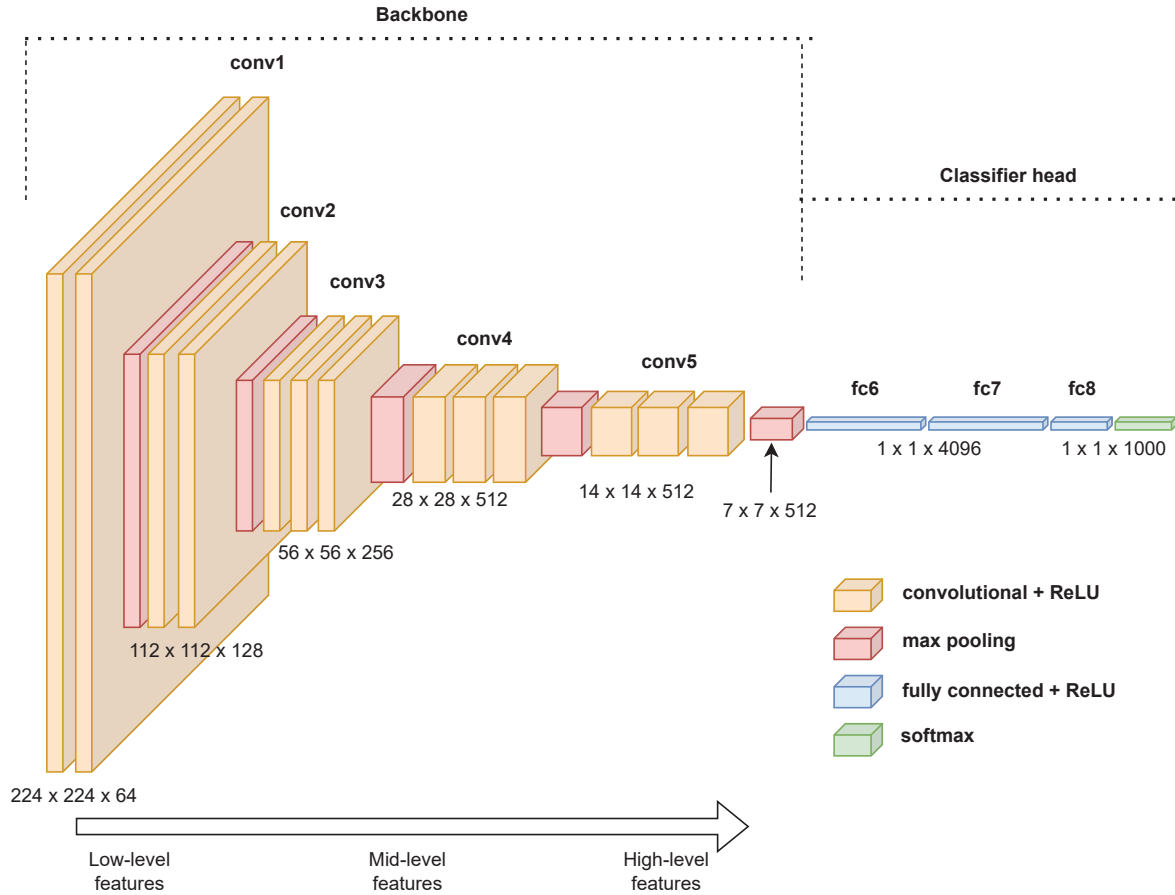
### 2.2.1.1.3 Effects of adding more layers to CNNs, deeper networks - VGG

As computing power evolved and the high performance of DCNNs became widely known, researchers developed interest in knowing the effect of increasing depth of the convolutional network on its accuracy. [40] showed significant improvement using a deeper architecture (16-19 layers) with small receptive fields ( $3 \times 3$  convolutional filters) throughout the net, even in the first layers, in contrast to AlexNet and ZFNet.

They claim that stacking two  $3 \times 3$  convolution layers has the same effective receptive field as a  $5 \times 5$  convolution layer. The benefits of this approach are: a more discrimina-

<sup>8</sup>CNNs are capable of learning complex features composed by simple features (e.g. A face is composed by two eyes, a nose, etc...

tive decision function and fewer parameters. Increasing depth led to more advanced feature hierarchies. However, the number of parameters increases as additional layers are stacked. VGG-16 (134-138M) is more than twice that of AlexNet (62M). Problems with training time, performance limitations, and overfitting due to increased depth became the main concern.



**Figure 2.6:** An illustration of the architecture of VGG-16.

In the next years, there will be an explosion of research to find efficient but high-performance modules to produce internal representations (modular concept of DCNNs).

#### 2.2.1.1.4 Optimal sparse connections - Inception modules

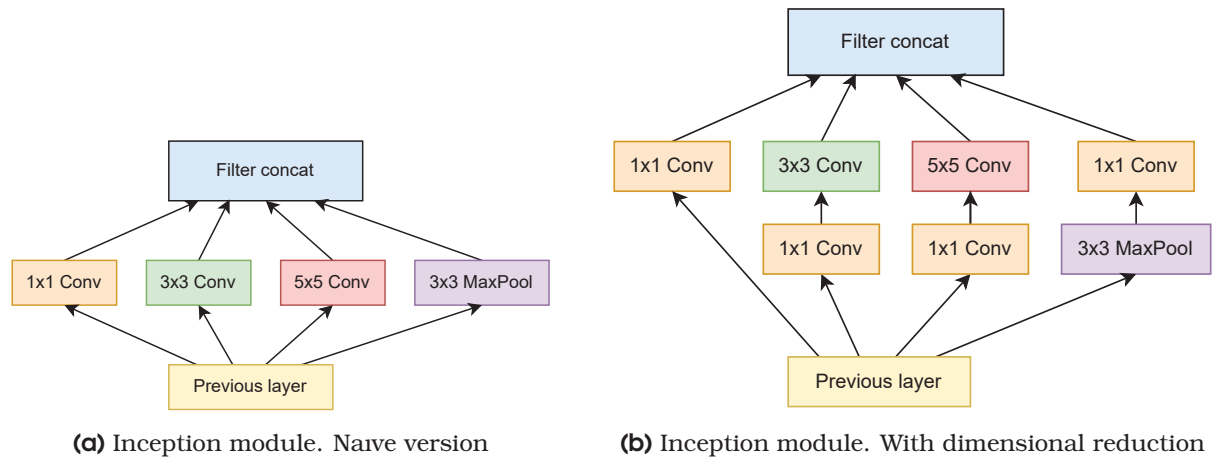
Further research aimed to increase the representational power of CNN going deeper, but keeping the computational cost constant. The simple approach to improve performance was to increase the depth of the net (more layers) or the width (number of features map per layer). However, this approach typically means more parameters, leading to more computational needs and a higher risk of overfitting (especially if the dataset is small) [41].

AlexNet and VGGNet used conventional convolutions, also called spatial convolutions,

that covers the entire depth of the layer, building learnable filters of size  $H \times W \times D^9$ , and studied the effect of changing the kernel size. The 22-layer model (GoogleNet) proposed in [42] extends and addresses the problem by finding the optimal sparsely connected architecture in CNNs and reducing the dimensionality with  $1 \times 1$  convolutional layers. Inception has a significantly lower computational cost than VGGNet or its similar ancestors.

This paper proposed a module called inception (see figure 2.7). This module aims to find the optimal local sparse structure, giving the network the ability to learn from different receptive fields in an efficient way, using several filter sizes and operations in each layer (parallel convolutional layers).

They also proposed a more computationally efficient module, adding a  $1 \times 1$  convolution (called Pointwise convolutions) before each path of the module where the dimensions are reduced<sup>10</sup>. The feature maps of each path are concatenated and passed on to the next layer. These modules separate the attention from cross-channel correlations with pointwise convolutions and spatial correlations with spatial convolutions (see figure 2.8).



**Figure 2.7:** Inception V1 module.

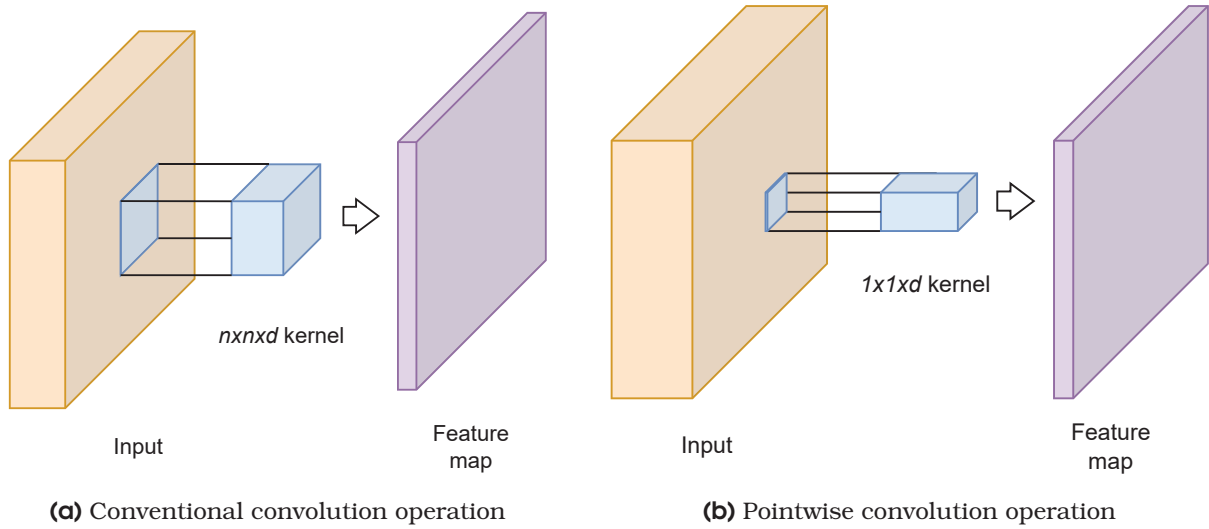
Although the number of parameters in the Inception architecture is 12x lower than that of AlexNet, concerns about the vanishing gradient are raised due to the relatively large depth of the network. The authors proposed a solution based on the hypothesis that the features produced in the middle layers should be discriminative enough to perform the classification. They attached two auxiliary classifiers to the intermediate layers for the training process. The loss of these auxiliary classifiers is added to the total loss with a discount weight<sup>11</sup>.

A subgroup of authors of Inception paper continued to explore ways to efficiently scale up networks. One of their following papers focused on reducing training time and preventing vanishing and exploding gradients of large networks, the paper called "Batch Normalization: Accelerating Deep Network Training by Reducing Internal

<sup>9</sup>  $H = Height = W = Width$ ,  $D = Depth$ . The depth is equal to the number of features maps.

<sup>10</sup> By reducing the number of feature maps, the number of parameters in each filter is reduced, thus, less operations are performed

<sup>11</sup>  $Weight = 0.3$



**Figure 2.8:** Spatial variations of convolution

Covariate Shift" [43] set batch normalization (BN) as a de facto standard<sup>12</sup> in the backbone.

It is based on the assumption that the covariate shift applies to the internal layers of the network; they call it internal covariate shift. Incorporating normalization to the internal layers of the network yields a substantial speed-up in training. These additional "layers" add only two additional learnable parameters per activation, adding little extra effort, and preserving the representation ability.

In [44] they extended the idea of Inception and Optimization of large networks by providing several design principles:

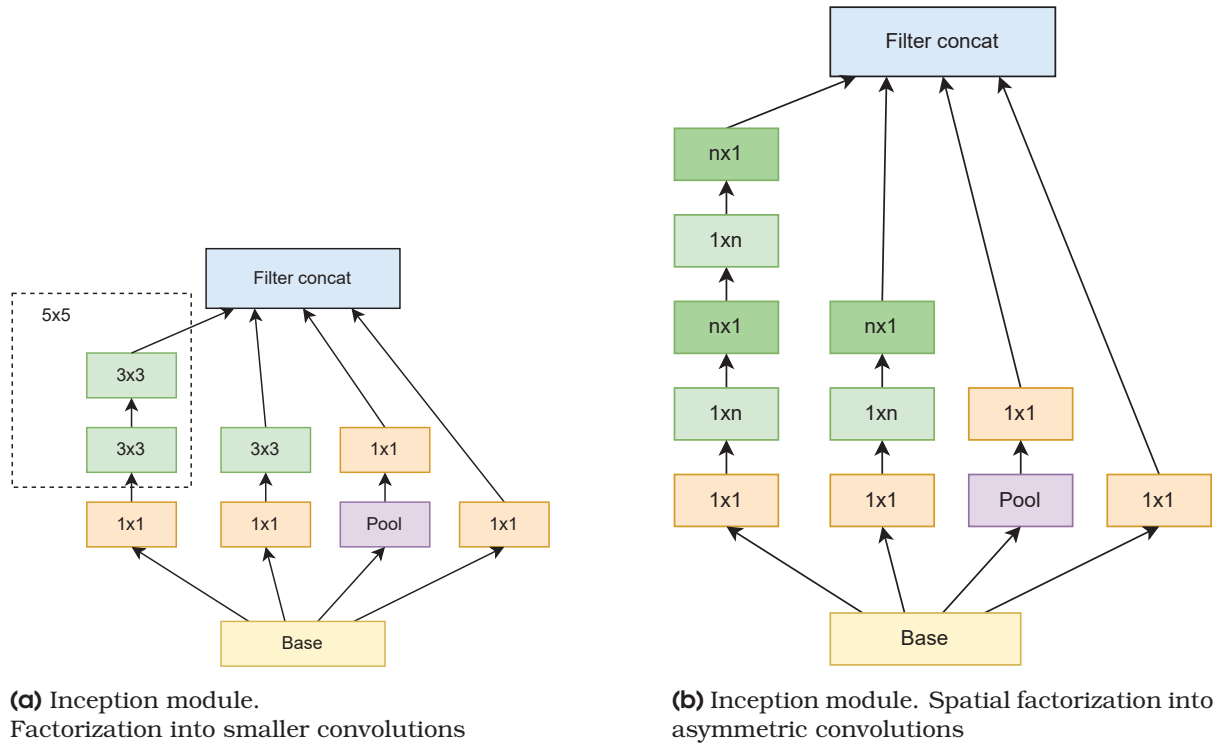
1. Avoid representational bottlenecks, that is, the size of the feature maps should gradually decrease until the final representation.
2. Higher dimensional representations are easier to process locally within a network.
3. Spatial aggregation can be done over lower-dimensional embedding without much or any loss in representational power.
4. Balance the width and depth of the network.

With these previous principles in mind and looking for new optimal sparse connectivity between layers, they provided new enhanced Inception modules [44]. These modules are based on factorizing convolutions in two ways to increase the representational capacity by using parameters more efficiently.

Factorization into smaller convolutions, for example, replacing the  $5 \times 5$  convolutions by two  $3 \times 3$  convolutions. Factorization into asymmetric convolutions reduces the computation even more, that is, replacing a  $n \times n$  convolution by a  $n \times 1$  convolution and a  $1 \times n$  convolution. However, spatially separable convolutions (asymmetric

<sup>12</sup>Technique that is used so widely that it is considered a standard for a given application

convolutions. Figure 2.10) does not work well on early layer but gives good result on medium grid-size<sup>13</sup> and cost saving greatly increase as  $n$  grows.



**Figure 2.9:** Inception V2 modules

To reduce the size of the feature map grid and comply with principles number 1 and 2, they proposed a parallel pooling and convolution layer with stride 2 (See figure 2.11).

So far, Inception modules have focused on the construction of informative features by searching an optimal correlation of spatial and channel-wise information within local receptive fields jointly. Researchers from [45] focused on channel correlations and proposed a novel module called the Squeeze-and-Excitation (SE) block, which adaptively adjusts channel-wise feature responses.

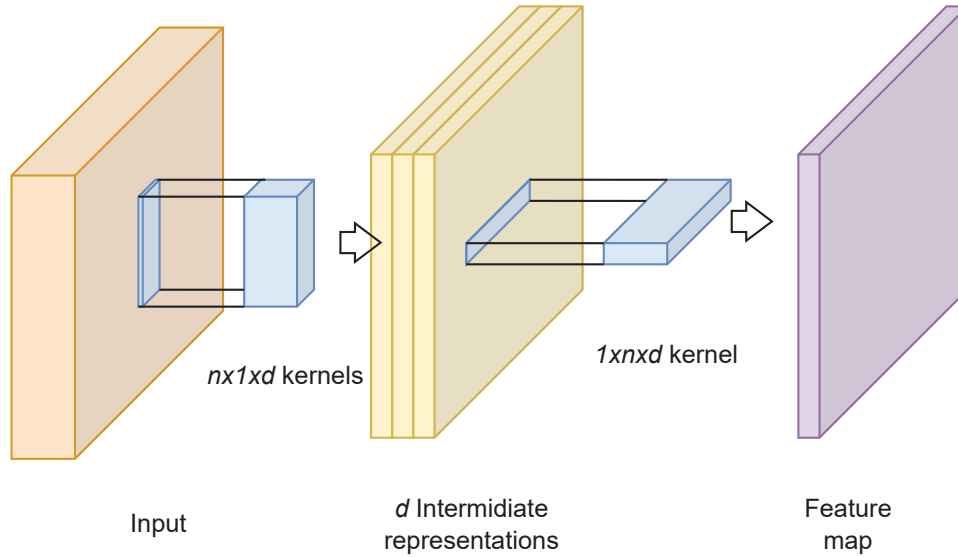
The SE block squeezes feature maps into a channel descriptor using global average pooling, creating embeddings of global spatial information. These embeddings are followed by a flexible<sup>14</sup> and non-mutually exclusive<sup>15</sup> function to capture channel-wise dependencies. The proposed excitation function consists of two dense layers, a ReLu function between dense layers, and a sigmoid function at the end.

Channel-wise feature adjustment was shown to improve the representational power. Their experiment shows the effectiveness of stacking SE blocks to form DCNNs, winning the ILSVRC 2017 classification challenge.

<sup>13</sup>On  $m \times m$  feature maps, where  $m$  ranges between 12 and 20

<sup>14</sup>Function capable of learning nonlinear interactions between channels.

<sup>15</sup>Multiple channels could be "excited"



**Figure 2.10:** Spatially separable convolution

#### 2.2.1.1.5 Optimization of deeper CNNs with multipath connections - ResNet

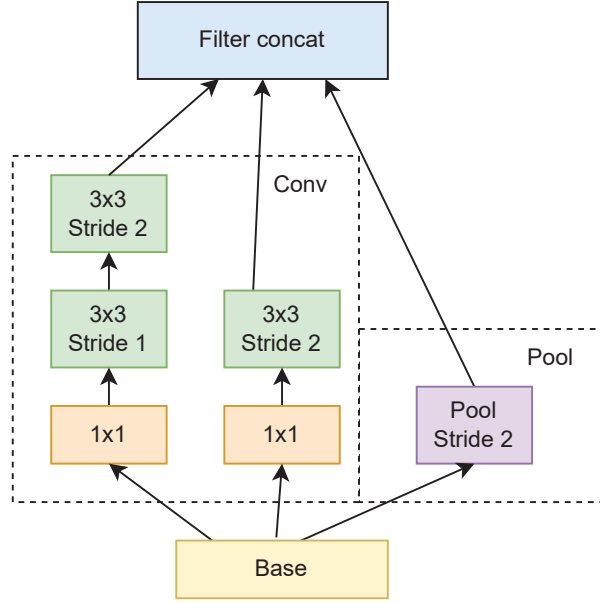
The depth of the network is essential to extract rich features and improve performance, as seen in 2.2.1.1.3. SOTA models went from 8-layer networks (AlexNet) to 22-layer networks (GoogleNet). However, deep networks are difficult to train due to well-known problems not related to overfitting, such as vanishing / exploding gradients [46].

Furthermore, deep network research exposed the problem of accuracy saturation / degradation; accuracy begins to decrease when a certain depth is exceeded, and the addition of more layers leads to a higher training error [47]. Numerous researchers were looking for a way to facilitate the training of deeper networks. Inception authors used auxiliary classifiers and BN layers 2.2.1.1.4 to address this problem.

The residual learning framework presented in the article [48] provided comprehensive empirical evidence showing that residual networks are easier to optimize and can gain accuracy from increased depth, solving the accuracy saturation problem. The proposed model, called ResNet, is arguably the most groundbreaking model in combination with AlexNet for CNN-based computer vision, being used in many semantic segmentation models as feature encoder.

They reformulate the layers as learning residual functions with reference to the input of the layers. The main idea is to introduce the so-called "shortcut connections" (See figure 2.12) to cope with the degradation problem and allowing the easy flow of gradients through the shortcut connections. These connections can be an identity mapping if the dimension is the same, or projection shortcuts if the dimension changes. It is based on the hypothesis that a deeper model should not have a greater training error than its shallower counterpart.

They proposed two basic building blocks for DCNN. Concerned by the training time required for deeper networks, they provided a modified version of the residual building block. Based on the same approach seen in Inception, they reduced the number of



**Figure 2.11:** Inception module that reduces the grid-size and avoids representational bottleneck.

feature maps using a  $1 \times 1$  convolution. They called it a "Bottleneck" building block (see figure 2.13b).

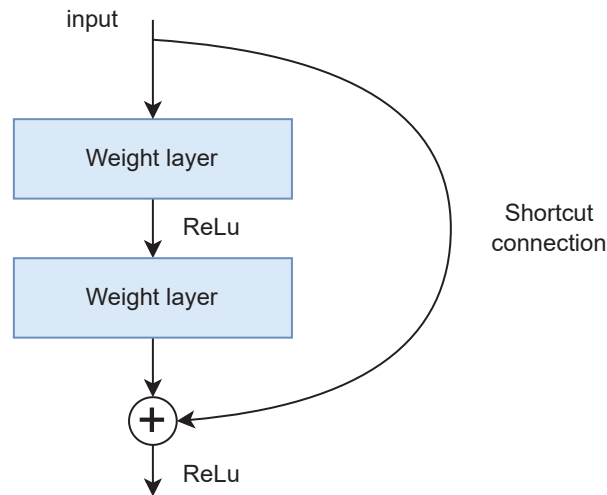
A study of different shortcut connections was performed, concluding that small differences are not essential to address the degradation problem. Identity mapping is the best approach to reduce complexity and model sizes. In addition, they explored models with 1000 layers, but the results were not satisfactory, setting a new limit in depth. The proposed models had 18, 34, 50, 101 and 152 layers.

At the same time as ResNet (2015), a paper exploring multipath information flow was also published to address deeper network problems [49]. "Highway networks" are based on the same idea of shortcut connections but using gating functions instead of the identity function. However, these functions depend on data and have parameters, which incurs the risk of having unused layers if the transform gates remain closed [50]. This problem is avoided by using parameter-free functions (identity mapping).

Several less influence researches exploring variations of the ResNet model were published in the next few years. DelugeNet proposed depthwise convolutional layers with learnable filters for intensive cross-layer connection [51]. FractalNet contains interacting subpaths of different lengths without involving residual learning [52]. The structural layout of the network is a truncated fractal.

Wide Residual Networks (WRNs) propose an increase in layer width in combination with a reduction in the depth of the network [53]. This approach demonstrated that the main strength of a residual network lies in the application of residual blocks, rather than in the depth of the network.

An extreme conceptualization of multipath connections is proposed in Dense Convolutional networks (DenseNet) [54]. Layers in a dense block have dense connections



**Figure 2.12:** Residual learning building block. The node "+" represents the element-wise addition.

(which are used as input) with all preceding layers. Direct connections are introduced between each two layers with the same feature map size.

A key difference is that these connections are channel-wise concatenations instead of addition (ResNet approach). This idea encourages the reuse of features to build hierarchical knowledge through the network, showing improvement in accuracy with deeper architectures and no sign of accuracy saturation or overfitting.

#### 2.2.1.1.6 Joining residual learning and optimal sparse connection - ResNeXt and Xception

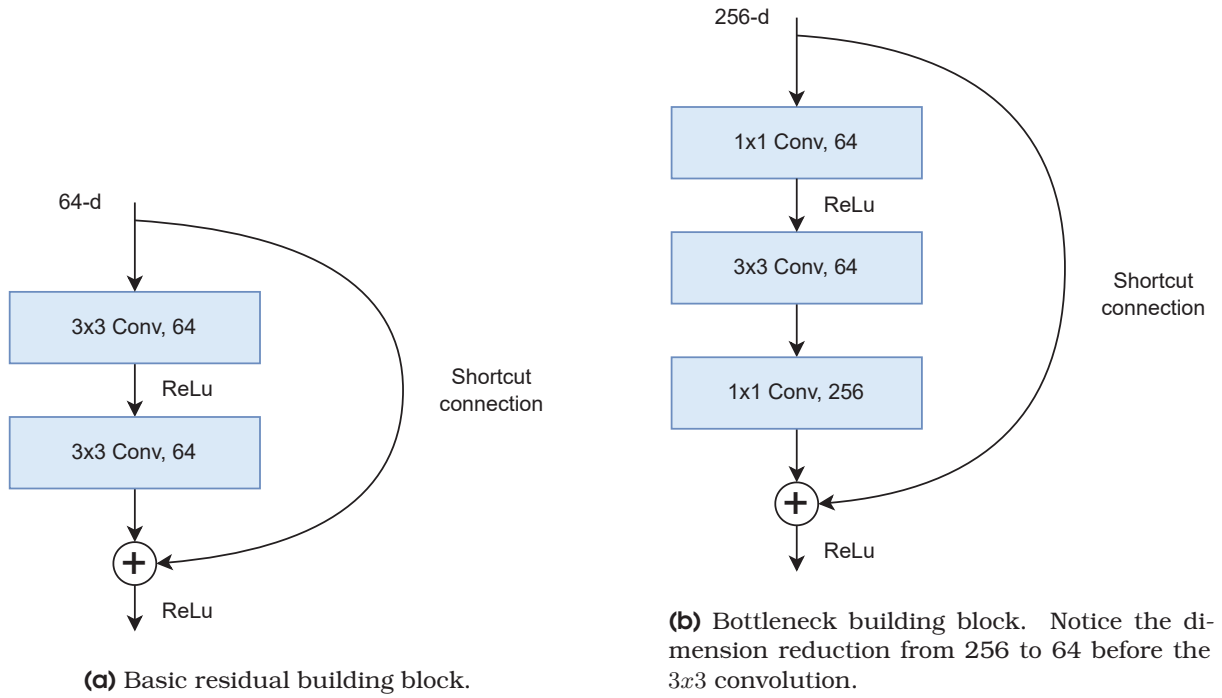
With the successful results of the Inception and ResNet architectures, several researchers raised the question whether combining these two concepts was beneficial. Inception's authors explored this question, showing empirical evidence that the combination could yield improvements in training time for Inception architectures and slightly better results.

They proposed two different combinations, "Inception-Resnet-v1" and "Inception-Resnet-v2", replacing the filter concatenation of Inception modules by the addition operation of residual blocks and adding a shortcut connection [55]. They also included a pointwise convolution without activation before the summation to increase the filter bank and compensate the dimensionality reduction induced by the Inception block.

Following the search for optimal sparse connections and the optimization of DCNNs, Xception modules were proposed in [56]. They are strongly based in depthwise separable convolutions (see Figure 2.15), in combination with the Inception modules<sup>16</sup> and residual learning.

The Xception modules replace the pointwise and spatial convolutions by depthwise

<sup>16</sup>Xception stands for "Extreme" Inception.



**Figure 2.13:** Residual building block for CNNs. Note that all convolutional layers are followed by batch normalization (not included in the diagram).

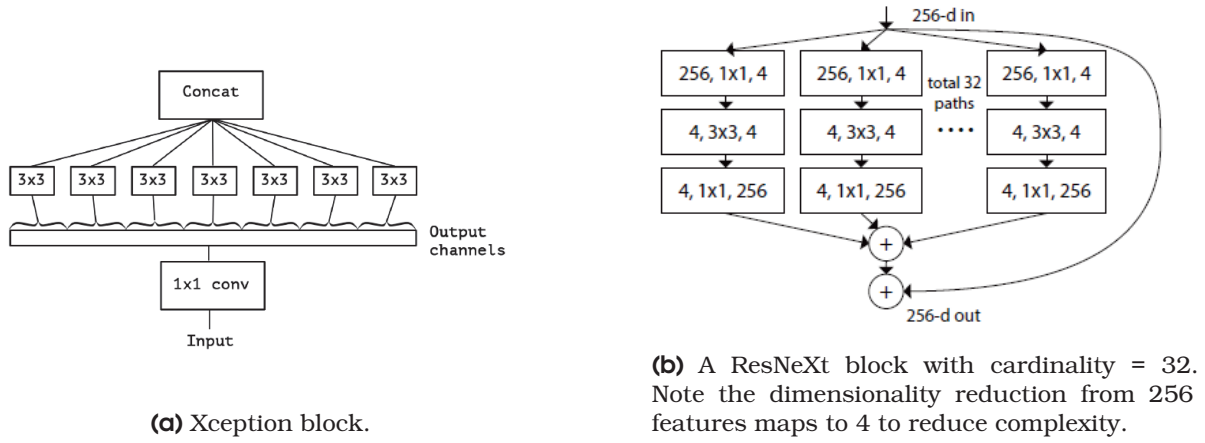
separable convolutions, assuming that cross-channel correlations and spatial correlations in DCNNs can be entirely decoupled (see figure 2.14a). The proposed 36-layer architecture, a linear stack of depthwise separable convolution layers with shortcut connections, achieved slightly better results than the Resnet and Inception v3 models in the ImageNet challenge with similar number of parameters as Inception v3.

Another architecture strongly based on Inception and ResNet that gained popularity was ResNeXt [57]. They proposed a multi-branch architecture with residual blocks. This architecture sets a new dimension called "cardinality", it refers to the size of the set of transformations (number of branches in the module). In figure 2.14 a visual comparison of the Xception and ResNeXt modules is presented. They are exploiting the learning capacity of networks to learn spatial and channel correlations with similar approaches.

#### 2.2.1.1.7 Efficient convolutional neural networks

Improvements in DCNNs were accompanied by more complex, time-consuming and memory hungry models to obtain better representations. The general trend was to develop deeper or wider networks [53], focusing mainly on higher performance while adding or adjusting elements to maintain reasonable efficiency levels. Real-world applications in commercial and industrial scenarios faced the requirement of a constraint time budget [47]. Additionally, mobile devices have gained popularity, adding another requirement for a limited computational budget.

Several researchers developed models specifically designed for mobile devices and



**Figure 2.14:** ResNext and Xception modules comparison.

embedded systems applied to computing and time-limited situations. Priorities change, looking for high-efficiency models with respect to time, memory footprint, and computation cost while maintaining performance. These types of models usually make extensive use of convolution factorization to reduce the model size (reducing the number of parameters).

Looking for an efficient trade-off between latency and accuracy, MobileNets introduced two global hyper-parameters to select the right model size [58]. The width multiplier ( $\alpha$ ) hyper-parameter is used to produce "Thinner models", that is, a coefficient to reduce the width of layers<sup>17</sup>. The result multiplier ( $\rho$ ) is a coefficient to reduce the input image and the size of the internal representation in the spatial dimension. These two hyper-parameters reduce the computational cost quadratically ( $\alpha^2$  and  $\rho^2$  respectively).

These models are strongly based on depthwise separable convolutions followed by pointwise convolutions<sup>18</sup>. Similar to Xception modules but with the order of convolutions inverted.

An extension of MobileNets, called Mobilenetsv2, introduced two main changes. They replace the MobileNet module by an inverted residual block in which the shortcut connections take place between bottleneck layers<sup>19</sup> and remove non-linearities in the last narrow layer [59] to maintain representational power (see figure /ref). They also use ReLu6<sup>20</sup> as non-linearity because of its increased robustness when used with low-precision computation [59].

ShuffleNet proposed a new extremely computationally efficient architecture with point-wise group convolutions and channel shuffling [60]. Noticing the lost efficiency of SOTA models (Xception, ResNeXt) in small networks due to dense pointwise convolutions, they replaced them with group convolutions<sup>21</sup> (see figure 2.17) to reduce complexity.

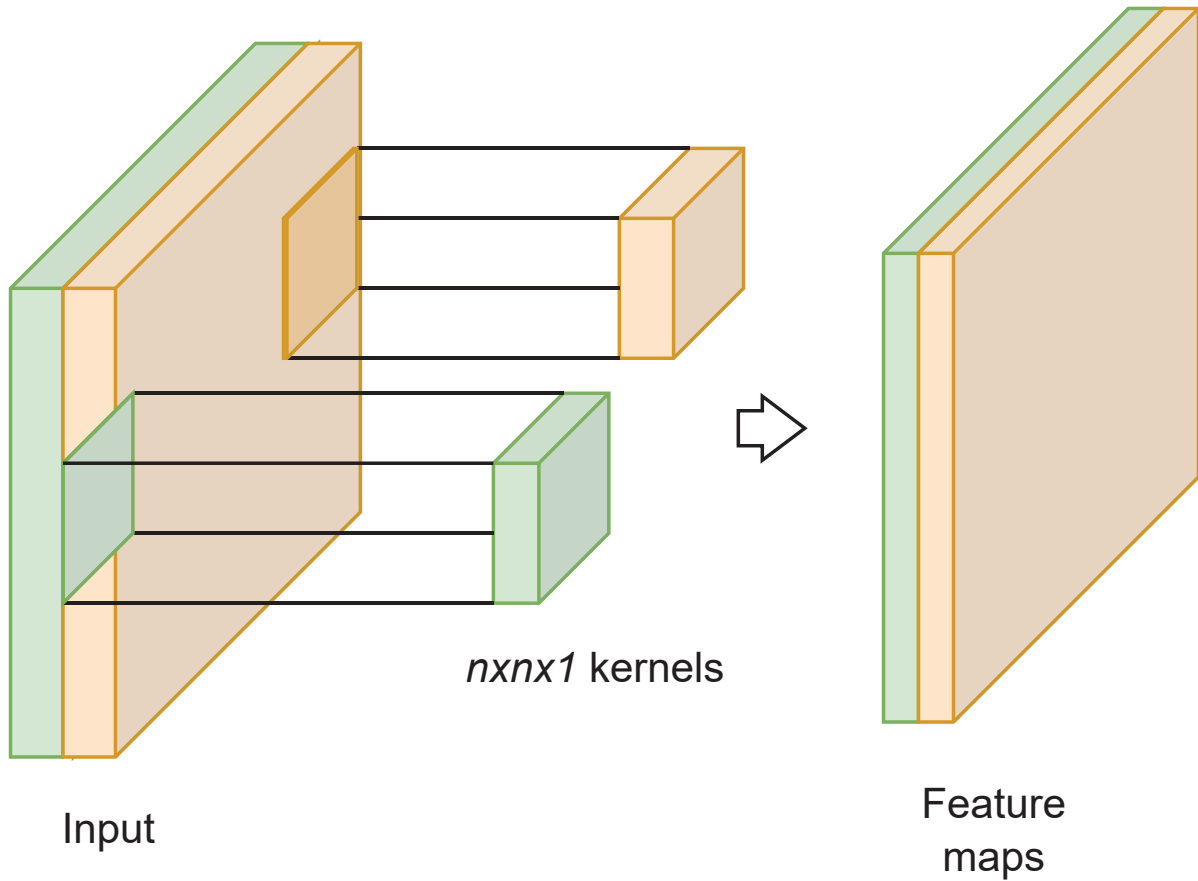
<sup>17</sup>For a given layer and a width multiplier, the number of input channels  $D$  becomes  $\alpha D$  and the number of output channels  $N$  becomes  $\alpha N$ . Typically,  $\alpha \in (0, 1]$ .

<sup>18</sup>This approach roughly reduces the computation in a quadratic factor. [59].

<sup>19</sup>High-dimensional representation appears in the middle and shortcut connections are performed between low-dimensional representations.

<sup>20</sup>ReLu modification the activation is limited to 6

<sup>21</sup>Usage of different set of filters for feature maps groups. Seen in AlexNet to distribute the model over



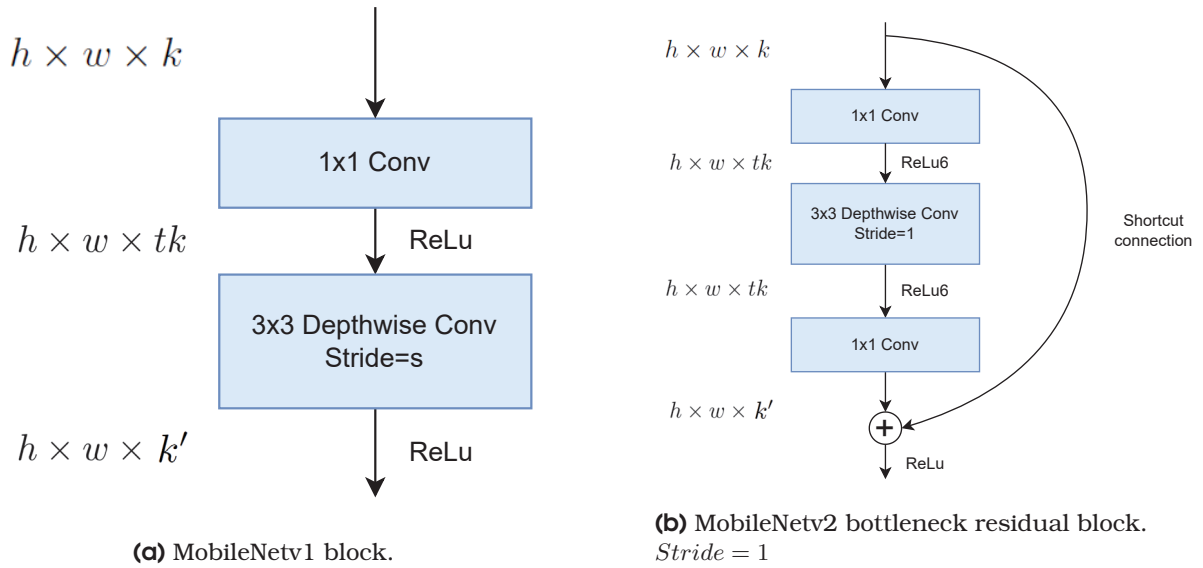
**Figure 2.15:** Depthwise separable convolutions

To overcome the problems brought about by group convolutions, a novel technique called channel shuffle was proposed.

#### **2.2.1.1.8 Neural Architecture Search (NAS)**

As described in section 2.2.1.1, complex models that automate the feature engineering process, learning hierarchical feature extractors from data rather than manually hand-crafting them, yielded successful results. Balancing network depth [40, 48], or width [53, 58] among other variables can lead to optimal trade-off between performance and computation, time, and memory cost, but requires significant effort of architecture engineering. Motivated by a reduction in this engineering process, a study of a method to automate the search for the optimal architecture was proposed in 2016 [61].

The neural architecture search, named for the method, sets a new paradigm in the design of deep neural networks. Many researchers have explored this technique to find the optimal architecture. The authors of EfficientNets studied a systematic approach for the efficient scaling of models using a compound coefficient [63]. The three dimensions considered in the search space of the NAS were depth, width, and resolution (see figure 2.19). They showed great results scaling well-known models, multiple GPUs.



**Figure 2.16:** MobileNetv1 and MobileNetv2 blocks.

MobileNet and ResNet.

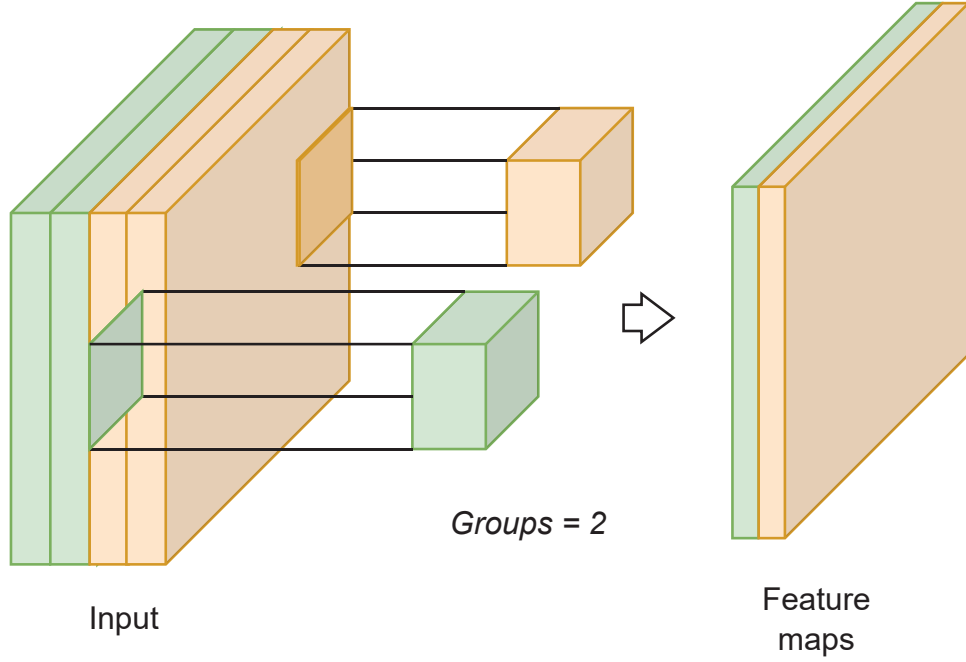
Their experiment showed that balancing width, depth, and resolution is critical to improve accuracy and efficiency (they tested their approach with ResNet and MobileNet as a baseline to scale). They also performed NAS to produce their own baseline, achieving SOTA results. They called this model EfficientNet.

#### 2.2.1.1.9 Summary

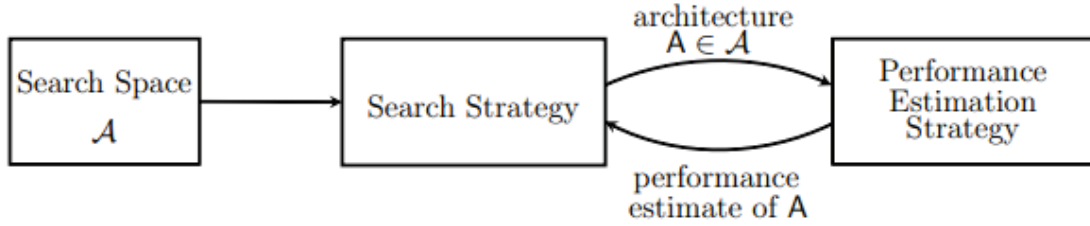
In 10 years, researchers went from 8-layer networks (AlexNet) to 200-layer networks (ResNet), explored the spatial and channel correlation of visual information to search for the optimal sparse connection of neurons, searched for efficient high-performance models using convolution factorization and even automatic design of feature extractor with NAS. These advances have achieved great results, reducing the error rate of ILSVRC classification challenge to less than 4% and showing the learning capacity of DCNNs.

Several concerns have been addressed by researchers during these years due to the problems raised with the increased depth and representational capabilities of DCNNs:

- Increased risk of overfitting due to an increased number of parameters.
- Accuracy saturation and vanishing / exploding gradient of DCNNs.
- Optimal sparse connections of visual information to obtain expressive representations.
- Trade-off between computational cost and memory footprint and performance for real-time applications / mobile devices.



**Figure 2.17:** Grouped convolutions.  $Groups = 2$ .

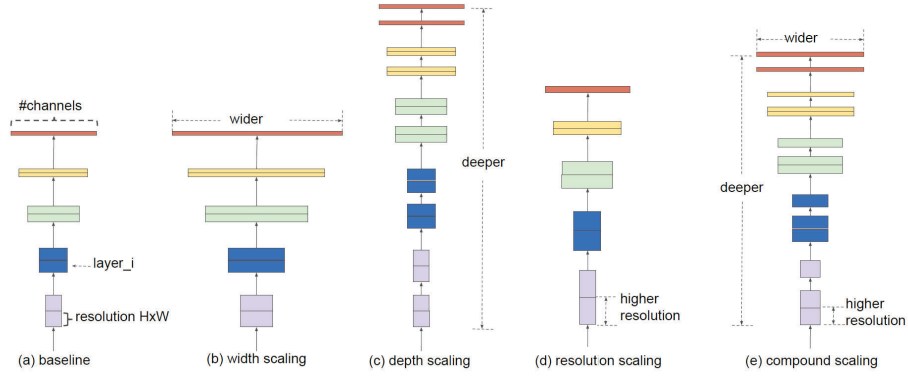


**Figure 2.18:** Abstract illustration of NAS methods. Figure from [62]

We have seen that convolutional backbones are strong feature extractors used in computer vision tasks; By changing the head structure (last dense layers of the net) after the convolutional backbone, the network can be changed to perform other computer vision tasks such as semantic segmentation. Furthermore, research showed that DCNNs trained on image classification could be repurposed to new generic tasks, improving the performance of a wide variety of tasks [64]. In the next section, this Master's thesis discusses problems related to the retrieval of fine-grained semantic information from DCNNs and the evolution of the architecture of these semantic segmentation models.

#### 2.2.1.2 From image classification networks to Semantic segmentation networks - Architecture paradigms

Motivated by the success of DCNNs in image classification, researchers started applying them to semantic segmentation models. Semantic segmentation is a dense prediction problem and is structurally different from image classification. It requires



**Figure 2.19:** Three dimensions used for model scaling in EfficientNet method. The compound scaling method uniformly scales the model in the three dimensions.

fine-grained local information to accurately locate objects, but at the same time, it requires global context to correctly label the pixels. Architectural innovations in the history of semantic segmentation models that aim to capture both types of information were typically included in the following architectures. A major breakthrough in semantic segmentation was the proposition of the Fully convolutional network (FCN) [65]. This section provides some of the most influential innovations in semantic segmentation architecture that created new paradigms for model design.

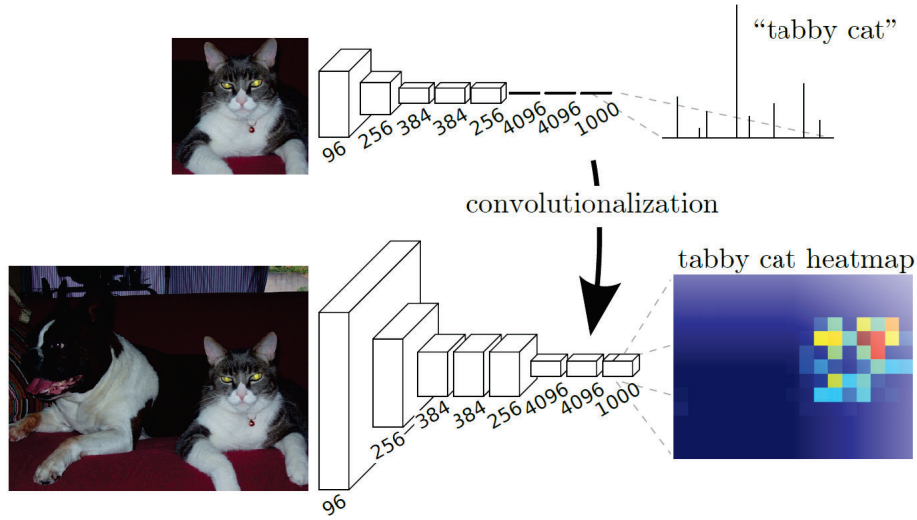
#### 2.2.1.2.1 End-to-end learning for semantic segmentation - FCN

FCN extended the DCNNs used for classification to the segmentation task (see figure 2.20), dramatically improving the previous performance of the SOTA architecture by 20% on the PASCALVOC2011 test (mean IoU) [65]. They adapted popular contemporary classification networks (AlexNet [30] (2.2.1.1.1), VGG net 2.2.1.1.3), and GoogleNet [42] (2.2.1.1.4) into FCNs. It is based on purely convolutional layers, replacing the dense layer at the end of the model by convolutional layers and outputting classification maps. Furthermore, this architecture allowed for quick and simple learning and inference using only convolutional layers.

The main contributions of the paper "Fully Convolutional Networks for Semantic Segmentation" [65] are the implementation of an architecture that takes an input image of arbitrary size and produces a correspondingly-size output map and the possibility to perform dense end-to-end training from labeled data. Its simplicity eliminated the need for pre-processing or post-processing elements such as object proposals [66].

One noticeable change of replacing dense layers by convolutional layers is that the network does not perform a general nonlinear function in the last layers because of the sparse connectivity of the convolutions; it computes a nonlinear filter. This is what they called a fully convolutional network, and can be trained using stochastic gradient descent, which allows the end-to-end learning.

When the head of the classification networks is removed (Last dense layers), the output is a set of downsampled feature maps, to retrieve dense predictions from these coarse

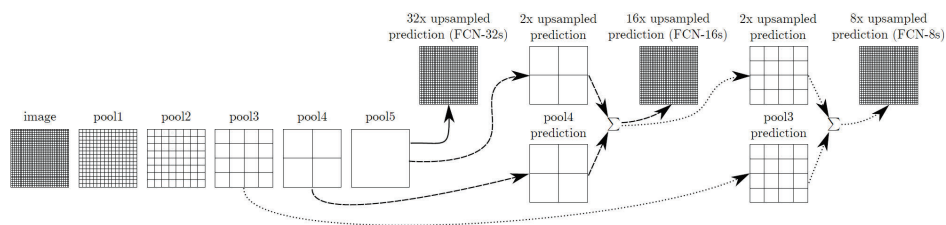


**Figure 2.20:** Transformation of classification networks to FCNs. Transforming dense layers into convolutional layers.

mainfolds, the authors of the paper proposed two approaches:

- Shif-and-stich, a technique to resample feature maps without interpolation, proposed in [67].
- Transposed convolutions, called in the paper *backwards convolutions*, consist of stacking learnable upsampling filters and activation functions. This approach yielded the best results in their experiment.

The output of a segmentation model is a segmentation mask. To achieve this, FCN annexed a pointwise convolution to produce  $N$  score maps, where  $N$  is the number of labels in the label set (in the case of PASCAL,  $N = 21$ ). It is followed by the transposed convolutions to upsample the coarse output to high-resolution score maps. In order to improve this transformation they proposed a direct acyclic graph (DAG) with skip connections to learn to combine course feature maps and obtain context information at different scales (see figure 2.21).



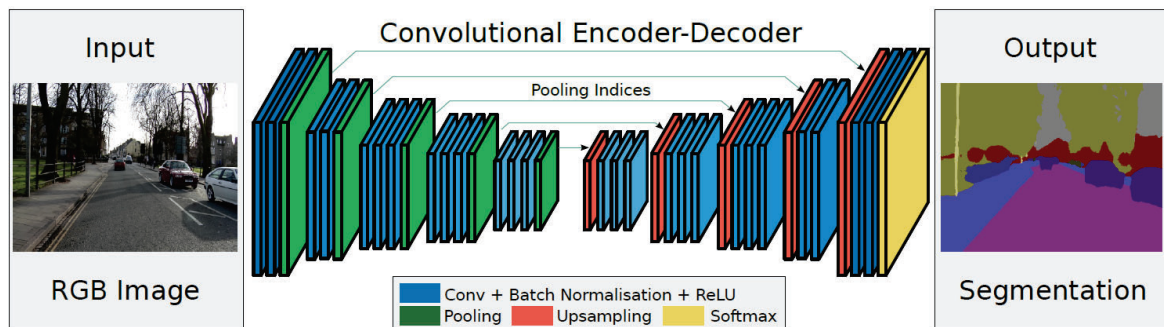
**Figure 2.21:** FCN architecture overview with skip connections.

### 2.2.1.2.2 Fine-grained localization of multi-scale objects

#### The Encoder-Decoder architecture

Researchers wanted to obtain smooth label predictions and generate coarse-to-fine information structures, refining a mechanism to map deep low-resolution feature maps to high-resolution segmentation mask. The general strategy of downsampling<sup>22</sup> leads to loss of spatial information, making it difficult to generate smooth labels. Several approaches based on an encoder-decoder architecture were proposed to improve the localization of class labels. The encoder, also known as the backbone, generates coarse feature maps, usually with a lower resolution than the input image, and the decoder maps low-resolution feature maps to fine-detailed feature maps for pixel-wise classification [68, 69].

In 2015, a convolution-based transposed decoder without skip connections was proposed [68]. The decoder was a multilayer deconvolution network capable of learning how to map low-resolution feature maps to dense predictions. The encoder adopted was the VGG-16 network. However, this architecture has many trainable parameters due to the upsampling filters, which require additional memory and time [70]. This inefficient approach motivated Segnet researchers to use pooling indices through skip connections [69] to efficiently increase the resolution of feature maps (see Figure 2.22 without the need to learn how to upsample).



**Figure 2.22:** An illustration of the Encoder-Decoder architecture (SegNet Network). The encoder transfers pool indices to the decoder to perform upsampling. Figure from [69]

Different approaches were also explored to use features extracted at different scales to include global information in the decoder component using skip connections. U-Net used skip connections to concatenate feature maps of the same size from the encoder into the decoder [71], that is, connect the same levels of feature maps of the encoder and decoder. A posterior revision of U-Net, called U-Net++, architecture revisited skip connections, allowing concatenation of features of varying semantic scales [72]. This is a fusion of the skip connections and the multi-scale aggregation strategy to capture global context explained in the next section.

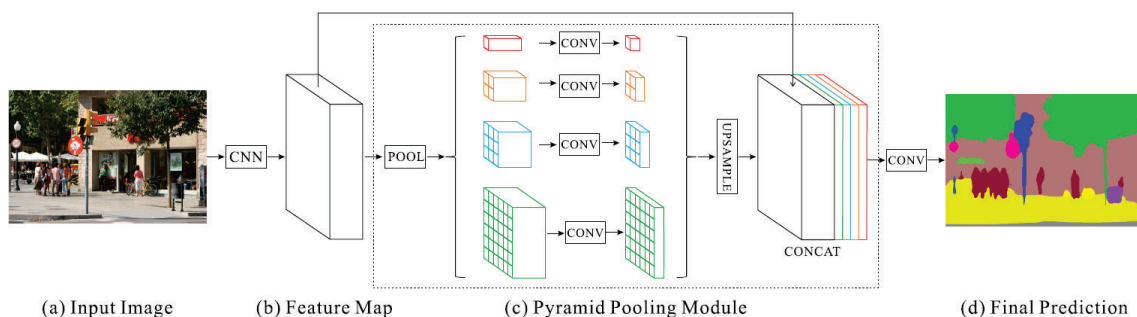
<sup>22</sup>Output stride is the term used to define the ratio of the input image size to the output feature map size.

### 2.2.1.2.3 Exploiting global context and spatial information - Pyramid architecture and dilated convolutions

Using skip connections, the model can utilize information at different levels to up-sample coarse feature maps to higher-resolution representations in the decoder stage. This is a proven strategy to capture global context on different scales. However, researchers could use another suitable strategy to use global context cues, increasing their accuracy. The lack of global context was said to be an important issue for the FCN [68, 73].

DilatedNet proposed a variation in the convolution operation, called dilated convolution [74]. Motivated by the adaptation of the SOTA segmentation model from classification networks to perform semantic segmentation, they proposed a module specifically designed for dense prediction tasks, which used dilated convolution. The module helps the model to aggregate multi-scale contextual information, avoiding losing spatial information and maintaining the resolution.

The researchers in [75] proposed a spatial pyramid pooling to handle information at different sizes, scales, and aspect ratios. This strategy was improved in PSPNet, a segmentation model that proposed an effective pyramid network to understand complex scenes [73]. PSPNet included a pyramid pooling module and the dilated convolution strategy, used a pre-trained ResNet model with dilated convolutions as encoder, and attached the pyramid pooling module at the end along with an upsample layer (see figure 2.23).



**Figure 2.23:** An illustration of PSPNet. The pyramid pooling module collects information on multiple scales and retrieve the input resolution with an upsample layer. Figure from [73]

### 2.2.1.2.4 Using graphical models to refine localization - Deeplab

A popular family of networks also used the dilated network strategy to expand the receptive field [70]. They called them atrous convolutions, but they follow the same concept as dilated convolutions. A spatial pyramid pooling (ASPP) was proposed to segment objects at different scales and collect contextual information of different aspect ratios. A novel strategy included in DeepLab, the name of the network, is the inclusion of a post-processing step based on graphical models. They used the output of the DCNN as input of a fully connected conditional random field model to produce smooth boundary labels.

DeepLab author revisited the atrous convolution strategy proposing modules that use dilated convolutions in cascade or in parallel with different dilation rates to capture multi-scale information. The ASPP is enlarged with image-level features and the dense custom random field is removed for efficiency reasons.

#### **2.2.1.2.5 Summary**

The adaption of DCCNs from image classification to semantic segmentation had faced two main problems that had been tried to solve with architectural innovations. The most successful approaches are: using an encoder-decoder architecture or using dilated convolution with a spatial pyramid pooling module.

- Loss of spatial resolution due to pooling operations (used by DCNNs to learn hierarchical abstract feature representations) that make it difficult to perform localization in dense prediction tasks.
- Difficulty to capture and segment objects at multiple scales for high pixel-level accuracy.

In the next sections, this Master thesis explores other approaches proposed to avoid losing spatial details and achieve high pixel-level accuracy such as using information from different modalities (e.g. depth data) or introducing attention mechanisms.

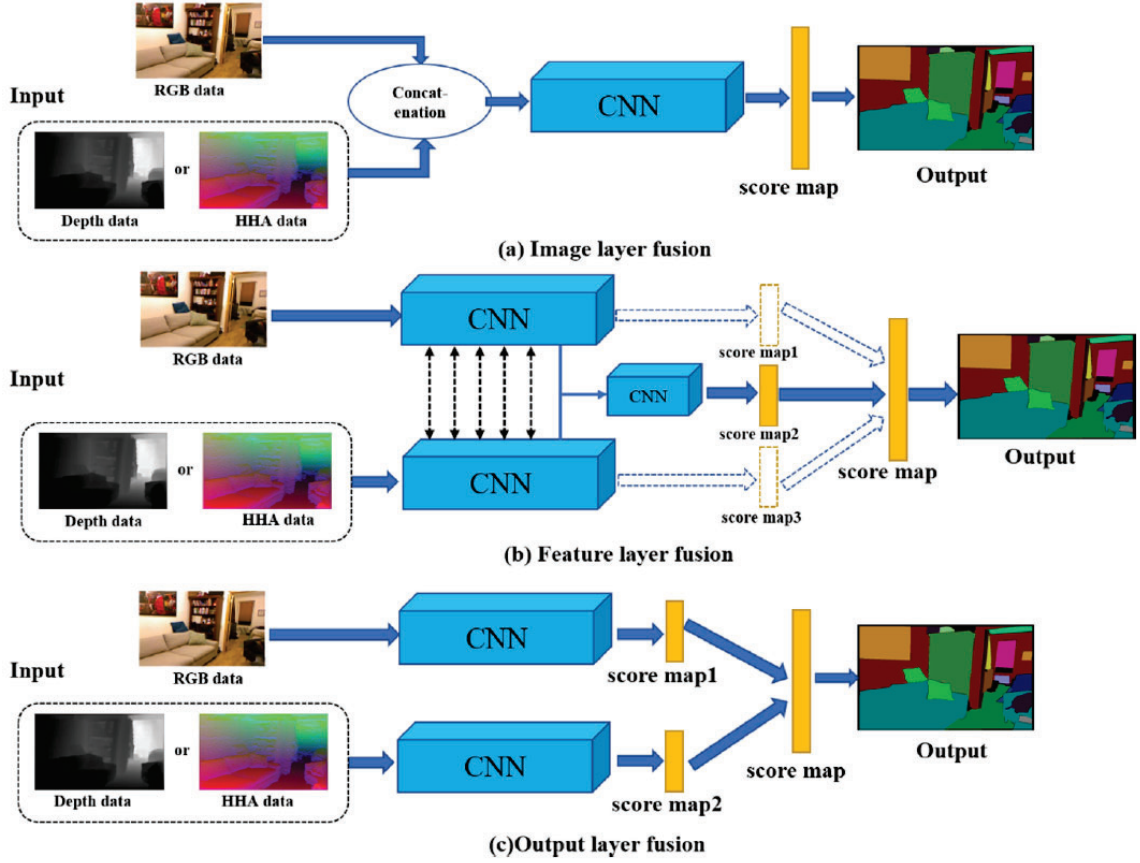
#### **2.2.1.3 Multi-modal networks - Incorporating geometric information**

Researchers have obtained successful results in semantic segmentation using RGB images, but sometimes they lack enough information to perform the task effectively in complex scenes, such as indoor scenes where object occlusion is frequent. An exploration of incorporating complementary data from other modalities into a semantic segmentation framework has been shown to improve performance [76].

However, the RGB and depth information shows different characteristics. The data discrepancy due to the measurement of different visual cues (light intensity and geometric information) makes it difficult to integrate them into an efficient representation [77]. Furthermore, the depth information provided by popular datasets cannot avoid the inclusion of noise because of the measurement tools used.

Approaches proposed in recent years addressed these problems by merging depth maps or features into the RGB encoder branch (see figure , but it has been shown to be not very effective, since geometric information in the deeper stages of the network is not accurate enough to assist RGB features [77].

Motivated by the success of the encoder-decoder networks, [78] proposes a encoder-decoder network, called FuseNet, where the encoder part is composed of two branches. One branch is a feature extractor of RGB images and the other branch is a feature extractor of depth maps. These two branches simultaneously extract features from RGB images and depth maps, whilst merging deep features into RGB feature maps as the network goes deeper (see figure 2.25).



**Figure 2.24:** Strategies to fuse depth and RGB information. Figure from [77]

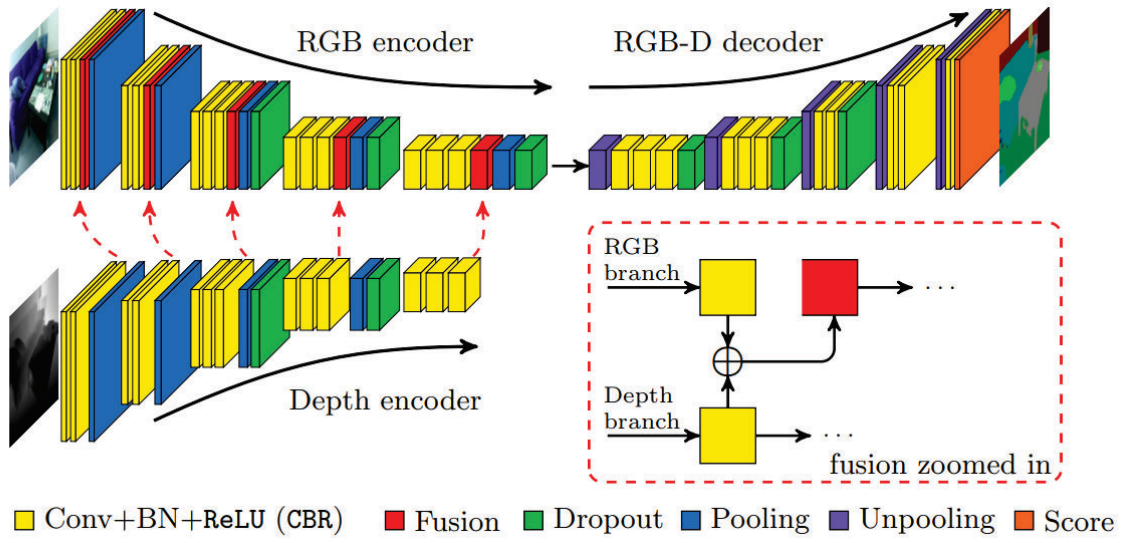
Multimodal networks have been shown to slightly improve performance of some models, although they require a lot of computation and memory because of the double deep convolutional network generated by additional branches.

#### 2.2.1.4 Attention mechanisms integration

Attention mechanisms are based on the human visual system. When we look through the windows, some objects attract our attention more than others, conditioning our perception of the scene. We have already seen the attention mechanism in previous sections of this Master's thesis. For example, the SE modules are channel attention mechanisms, and the residual modules are branch attention mechanisms [79].

An attention mechanism adjusts the model weights to give more or less relevance to different aspects of the information, thus specifying "what or where to pay attention to". In the case of channel attention, the model dynamically recalculates the weights associated with each channel. Branch attention recalculates the weights associated with different information flows or branches.

A taxonomy of different attention mechanisms according to the data domain is composed of 4 categories. Spatial attention, channel attention, temporal attention, and branch attention [77]. Some mechanisms have been shown to be more effective for



**Figure 2.25:** An illustration of FuseNet. The fusion layer is implemented as an element-wise summation. Figure from [78]

specific tasks; for example, spatial attention performs better in dense prediction tasks, while channel attention performs better in classification tasks [79].

#### 2.2.1.4.1 Visual transformers (ViT)

Transformers were first proposed in the paper "Attention is all you need" [80], referencing the contemporary success of the attention mechanism in Natural Language Processing (NLP). Transformers architecture are sequence-to-sequence models which use self-attention mechanism instead of recurrent or convolutional layers, breaking the mainstream solution in deep learning so far. These architectures showed impressive results in terms of performance and efficiency in NLP; thus, researchers study an approach to apply it to computer vision.

[81] showcase the effectiveness of transformers in computer vision by proposing a classification network entirely built with visual transformers. The main advantage of these kinds of models is that they allow one to learn the global information at the very first layer of the network and overcome the intrinsic limitation of convolutions to capture global context. Several models also make use of the visual transformer strategy. Segmenter, a purely ViT network, obtains dense predictions from patched embeddings with a pointwise linear decoder, also called mask transformer decoder [82].

Other approaches proposed a mixed architecture. ViTBis, a segmentation model for biomedical image segmentation, uses skip connections and multiscale convolutional architecture to take advantage of the fine localization benefit of convolutions. A linear projection as the head of the network is proposed to produce the segmentation mask [83].

#### 2.2.1.4.2 Summary

Recent progress in deep learning has enable great results in semantic segmentation. This section has provided a brief explanation of some of the milestones in the evolution of semantic segmentation models based on feature encoders and the problem they aimed to solved. However, architecture related to unsupervised and weakly supervised learning, recurrent neural networks (RNN), region proposals based methods, or graphical models are not included due to scope and time limits.

we have reviewed some of the most influential strategies to achieve high-performing models, including architectural innovations, the attention mechanism, or integrating depth information into the feature extraction process. To validate these models, researchers use different types of metrics that measure desirable quality attributes of models. The next section focuses on these metrics and the most popular metrics for evaluating semantic segmentation models.

### 2.2.2 Semantic segmentation metrics

To know if your model performs as expected, researchers make use of evaluation metrics. These metrics are calculations that measure different quality attributes of the model. Concerning the benchmarking of architectures designed for semantic segmentation, there are two main groups based on the related quality attribute:

- Metrics related to effectiveness. These metrics took as input the original image and the segmentation mask and measured the capacity of the model to perform the task correctly.
- Metrics related to efficiency. These metrics measure indirect qualities, such as memory consumption or the time required to generate the segmentation mask of a single image.

#### 2.2.2.1 Performance metrics

##### 2.2.2.1.1 Accuracy

Pixel accuracy (PA) , also called global accuracy, is the percentage of pixels in the image that are correctly classified, that is, a ratio between the number of correctly labeled pixels and the total number of pixels [26]. This metric defines four types of prediction: True positives, the pixel is correctly classified as  $C$  where  $C$  is a class label. False positive, pixel incorrectly classified as  $c$ . True Negative, the pixel is correctly classified as not  $c$ . False Negative, the pixel is incorrectly classified as not  $c$ .

$$\text{Pixel accuracy} = \frac{TP + TN}{\text{Total of pixels}} \quad (2.1)$$

Mean pixel accuracy (mPA), also called class average accuracy, is a version of the metric that calculates the ratio per class and then calculates the mean.

Although this metric is easy to calculate and straightforward, it is not very representative of the performance of the model. This happens because input images usually have a class imbalance, that is, a class dominates the image [84, 85]. For example, imagine an image of an ocean with a small boat in the middle. The accuracy could be high if all the pixels are classified as ocean and sky; however, the important semantics are the small boat. For this reason, this metric is no longer used in many popular semantic segmentation challenges.

### 2.2.2.1.2 Intersection over Union (IoU)

Intersection over Union, also known as the Jaccard index, is the overlap area between the predicted segmentation mask of a class and the ground-truth image mask of that class [85]. That is, the ratio of the intersection to the union of the predicted label mask and the ground-truth label. This is more representative of the performance of the model than the accuracy of the pixels because the predictions for each class are independent. Using the same variables as 2.2.2.1.1 the IoU of a class  $c$ :

$$IoU_c = \frac{Intersection}{Union} = \frac{TP_c}{TP_c + FP_c + FN_c} \quad (2.2)$$

The Mean IoU (mIoU), one of the most widely used, due to its simplicity and ease, takes the IoU over all classes and calculates the mean of them. [85]. The frequency-weighted IoU (FWIoU) is an extension of the mIoU which takes into account the frequency of each class to deal with class imbalance.

$$mIoU = \frac{1}{N_{cls}} \sum_{c=1}^{N_{cls}} \frac{TP_c}{TP_c + FP_c + FN_c} \quad (2.3) \quad FWIoU = \sum_{c=1}^{N_{cls}} \frac{1}{W_c} \frac{TP_c}{TP_c + FP_c + FN_c} \quad (2.4)$$

### 2.2.2.1.3 Precision, recall and F1 score

These measures are usually used in classification problems, thus, they can be used in pixel-wise classification tasks. Precision measures the ratio of correct predictions to positive predictions. It is calculated as the rate of true positives over the sum of true positives plus false positives [86].

Recall measures the ratio of correct predictions to all the predictions of the class. It is calculated as the rate of true positives over the sum of true positives plus false negatives.

The f1 score metric is a weighted average of Precision and Recall. It is a useful metric when an uneven class distribution is subject.

$$precision = \frac{TP_c}{TP_c + FP_c} \quad (2.5)$$

$$recall = \frac{TP_c}{TP_c + FN_c} \quad (2.6)$$

$$F1\ score = \frac{2\ precision\ recall}{precision + recall} = \frac{TP_c}{TP_c + \frac{1}{2}FP_c + \frac{1}{2}FN_c} \quad (2.7)$$

#### 2.2.2.1.4 Dice coefficient

The dice coefficient, commonly used in medical image segmentation, is defined as twice the overlap between the predicted label mask and the ground truth label divided by the total number of pixels. Dice scores not only measure how many positives you find, but also penalize false positives found by the method and positives not found by the method.

$$\frac{2TP_c}{2TP_c + FP_c + FN_c} \quad (2.8)$$

#### 2.2.2.2 Efficiency metrics

These metrics are common to many deep learning models and measure attributes such as computational and memory efficiency. However, memory and processing time are dependent on the hardware used. The most common are:

- Inference time. Time to process an image.
- Frames per second (FPS). The inverse of the inference time, which is the number of images that the model can process in one second.
- Memory footprint. Memory occupied by the model.

### 2.2.3 Semantic segmentation datasets and benchmarks

Datasets are an essential component to train supervised learning models. The construction of a new one is both laborious and time-consuming. In recent years many datasets have been published to facilitate the research. Some datasets such as PASCAL VOC [87], Cityscapes [88], NUYDv2 [89] or ADE20K [90] have become a standard in the research field to train and compare models.

An essential step in the validation of a semantic segmentation process is to benchmark the model with a dataset that represents real-world data. Good results in these benchmarks prove that the model will generalize well under the assumptions dictated by the dataset (e.g., indoor scenes).

These datasets are usually part of proposed challenges to encourage researchers to use them and position their approaches in the state-of-the-art, measured by performance in these challenges. This section provides a description of the most influential challenges and how they structure the data.

Datasets are usually divided into the training subset and the test subset. The training subset is usually divided into training and validation dataset as well. Supervised training challenges (scope of the framework) include complete annotation with segmentation masks for every image in the dataset. Additional information such as orientation, challenge performance metrics, and annotation procedure (also called labeling policy) is specified, as well as the additional metadata, to facilitate the creation of the data pipeline.

#### **2.2.4 Train and test a semantic segmentation model**

Supervised deep learning-based models are trained using an optimization algorithm that adjust the weights of the model based on the minimization an objective function. The popular FCN allowed semantic models to be trained end-to-end with this algorithm, and it has become a de facto standard in semantic segmentation. Training policy is usually included in research papers, including techniques used to reduce training time, improve model performance, and reduce the risk of overfitting. Some of the techniques are as follows:

##### **Data augmentations**

Data augmentation is a technique used to artificially increase the size of the training dataset. It has a relevant effect for deep learning trained with small-scale datasets [91]. It consists of label-preserving transformations, such as rotations, reflections, and elastic deformations [92].

##### **Weight initialization**

Weight initialization has an effect on the optimization of the neural network [41]. Researchers have studied robust weight initialization methods to improve the performance and training of deep models. There are two main popular methods:

- Pre-training the backbone for other tasks (image classification). However, this approach is time-consuming and experiments of networks pre-trained on ImageNet shown that it does not provide regularization or improve final performance although it speeds up early training [93].
- Initialization strategy of the filter weights. Depending on the activation function used in neurons, some researchers have studied alternatives to random initialization. [94] proposed the He weight initialization to be used with rectified linear activations and also proposed a parametric ReLu function.

#### 2.2.4.1 Loss functions

In a deep learning pipeline, the loss function evaluates how well the predicted segmentation mask matches the ground truth label. It plays a key role in the optimization of deep neural networks. [95] made a compilation of loss functions, aggregating more than 15 loss functions into 4 categories.

- Distribution-based losses, which aim to minimize the dissimilarity between two distributions (e.g. cross-entropy, focal loss).
- Region-based loss which aims to minimize the mismatch (or maximize the overlap) of the predicted segmentation mask and the ground truth label (e.g. Dice loss, IoU loss)
- Boundary-based loss, which aims to minimize the distance between the predicted segmentation mask and the ground truth label (e.g. Boundary loss).
- Compound loss, which joins two loss functions (e.g. Combo loss)

The optimal loss function depends on the properties of the data set used for training, such as distribution or boundaries [95].

#### Improving the loss function

The most common objective function is the cross-entropy loss, but it is common that the variation in the number of pixels of each class in the training set to be unbalanced. A solution is to assign a weight to each class and include it in the loss functions. This is known as class balancing.

An approach to class balance is the median frequency balancing [96], in which the weight assigned to a class in the objective function is the ratio of the median of class frequencies computed on the entire training set divided by the class frequency. There are several variants of loss functions to deal with class imbalance, a common problem in semantic segmentation (2.2.4.1).

## Chapter 3

# A deep learning ecosystem for semantic segmentation

### Contents

---

|       |                                                                                       |           |
|-------|---------------------------------------------------------------------------------------|-----------|
| 3.1   | Pytorch - An open source machine learning framework . . . . .                         | <b>37</b> |
| 3.1.1 | Overview of Pytorch . . . . .                                                         | 38        |
| 3.1.2 | Pytorch packages . . . . .                                                            | 38        |
| 3.1.3 | Vanilla Pytorch limitations . . . . .                                                 | 40        |
| 3.2   | A framework for segmentation using Pytorch - <code>segmentation_models.pytorch</code> | <b>40</b> |
| 3.3   | Speeding-up the training and evaluation - Ignite . . . . .                            | <b>41</b> |
| 3.4   | A configuration system for experiments - YACS . . . . .                               | <b>42</b> |
| 3.5   | Conceptual framework for semantic segmentation . . . . .                              | <b>42</b> |
| 3.5.1 | Basic layers . . . . .                                                                | 43        |
| 3.5.2 | Segmentation models . . . . .                                                         | 44        |
| 3.5.3 | Data pipeline . . . . .                                                               | 46        |
| 3.5.4 | Training process . . . . .                                                            | 46        |
| 3.5.5 | Evaluation process . . . . .                                                          | 47        |
| 3.5.6 | Configuration system . . . . .                                                        | 47        |

---

Deep learning has been the focus of many researchers and software developers; as a result, there is a rich ecosystem of tools to develop deep learning systems. This section provides a description of the tools proposed to build, train, and evaluate semantic segmentation models and a description of the main features that make them suitable for deep learning research.

### 3.1 Pytorch - An open source machine learning framework

This section provides an brief overview of Pytorch and its existing toolbox for creating, training, and deploying deep learning-based models for semantic segmentation. The framework proposed in this Master's thesis is built upon Pytorch. Pytorch [97] is an

optimized open-source machine learning framework that speeds up the research and deployment process. This library performs fast execution of dynamic tensor operations with automatic differentiation and GPU acceleration [97].

Pytorch is based on four major trends in the deep learning community: array-based programming, automatic differentiation to automate the calculation of derivatives, open-source ecosystem, and hardware accelerators. These characteristics make it an excellent breeding ground for new deep learning-based tools [97].

### 3.1.1 Overview of Pytorch

Pytorch enables rapid experimentation and efficient deployment through a user-friendly API and an ecosystem of tools and libraries. It provides tools to seamlessly transform the graph-based model built on Pytorch into production-ready structures (TorchScript) and accelerate the path to production with flexible tools (TorchServe)<sup>1</sup>. It is optimized for distributed training (DistributedDataParallel) and supports asynchronous execution.

### 3.1.2 Pytorch packages

#### 3.1.2.1 Torch

Torch is the base package of Pytorch and provides several subpackages and modules with basic functionalities for deep learning. It contains the tensor data structure and defines the mathematical operations on these tensors. `torch.Tensor` is the basic data structure of Pytorch, it is a multidimensional matrix containing elements of a single data type<sup>2</sup>. It also supports serialization of the tensor and casting of well-known data structures (Numpy `ndarrays`<sup>3</sup>) among other core functionalities.

##### 3.1.2.1.1 Support for model building - `Torch.nn`

`torch.nn` package is a high-level API that provides basic modules and classes to build neural networks. `torch.nn` defines `nn.Parameter`<sup>4</sup> as a subclass of `Tensor`, which is used as attributes of `nn.Module` subclasses instances to add the parameter to the list of parameters. Container classes are used to build more complex modules such as a linear neuron (`nn.Linear`) or a 2D convolutional layer (`nn.Conv2d`).

The primitive for all modules of the neural network is `nn.Module`. The subclasses of `nn.Module` can have nested subclasses of `nn.Module` and must implement the

---

<sup>1</sup>Visit <https://pytorch.org/features/> to see all features.

<sup>2</sup><https://pytorch.org/docs/stable/tensors.html>

<sup>3</sup>Documentation of `numpy.ndarray`:

<https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html#numpy.ndarray>

<sup>4</sup>`torch.nn` is abbreviated to just `n` to improve readability and simplicity

`forward` method, which returns the output of the module. Pytorch provides some implementations of `nn.Module` with a specific arrangement of parameters and operations in the forward pass:

- Specific containers
- Convolutional layers
- Pooling layers
- Padding layers
- Non-linear activations
- Normalization layers
- Recurrent layers
- Transformer layers
- Linear layers
- Dropout layers
- Sparse, distance, and loss functions

#### **3.1.2.1.2 Support for model training - `torch.optim`**

`torch.optim` is a package that implements several optimization algorithms. It is highly flexible, allowing the developer to build a custom optimizer inheriting from the class `Optimizer`. `torch.optim` also provides implementation of a learning rate policy based on a schedule to adjust the learning rate as the training progresses in the module `torch.optim.lr_scheduler`. The `Optimizer` must implement the method called `step`, which updates the parameters. Pytorch provides implementations for the most common optimization algorithms (SGD, Adam, etc.).

#### **3.1.2.1.3 Support for data pipeline - Dataset and Dataloader**

Pytorch provides classes to decouple data sample processing from model training and building code for better readability and modularity. Provides two primitives: `torch.utils.data.Dataset` to store samples and their corresponding labels, and `torch.utils.data.DataLoader` to define the sampling policy.

A custom `Dataset` must implement `__init__`, `__len__`, `__getitem__`.

#### **3.1.2.2 Torchvision**

Torchvision is a library that belongs to the Pytorch project and aims to support computer vision systems. It provides several functionalities that are useful for computer vision tasks such as transforming and augmenting images on PIL format or `Tensor` format. The subpackages `torchvision` provided are listed below:

- `torchvision.transforms`. Common image transformations are supported and can be chained using `Compose`, although `Compose` is not supported by `torchscript`.
- `torchvision.models`. It is a subpackage that contains the definition of popular computer vision models seen in previous sections (e.g. AlexNet, MobileNetv3, EfficientNet, VisionTransformer) and pre-trained models<sup>5</sup>.
- Built-in datasets classes for popular datasets (e.g. Cityscapes, PASCALVOC).
- `torchvision.utils` provides several utilities, such as drawing segmentation masks on a given RGB, or saving a `tensor` into an image.
- `torchvision.io` provides support to read and write images and videos.

A very useful functionality provided is the feature extraction utility with `torchvision.models.feature_extraction` that lets the researcher access intermediate representations from the network. `create_feature_extractor` takes as input the model and the stages (nodes) of the network and returns a modified network.

### 3.1.3 Vanilla Pytorch limitations

Pytorch and Torchvision have a rich ecosystem of functionalities to create and train deep learning models. It supports many high-level operations and makes it easy to build complex modules by nesting `nn.Modules` or using built-in classes such as `nn.Sequential` for greater abstraction. It also supports several algorithms to optimize neural networks and provides the implementation of many objective functions. Datasets and data flow provide huge flexibility by the abstractions `Datavision` and `DataLoader`, and Torchvision provides basic functionality and the implementation of some of the most popular computer vision models for a few tasks.

However, Pytorch train scripts are usually big and messy, with tens of imports and hard-coded hyperparameters, making it difficult to keep track of experiments. It also lacks support for several metrics used in evaluation and tracking, which hinders model evaluation and ablation studies. In addition, specific tasks or processes of supervised learning semantic segmentation research require a lot of boilerplate. Libraries described in the following sections aim to cope with these shortcomings.

## 3.2 A framework for segmentation using Pytorch - `segmentation_models.pytorch`

Segmentation models [98] is a Python library with artificial neural networks for image segmentation based on Pytorch. It provides the implementation of 9 model architectures, such as PSPNet (explained in section 2.2.1.2.3) or Unet (explained in section 2.2.1.2.2) among others.

---

<sup>5</sup>See <https://pytorch.org/vision/stable/models.html> for complete documentation

It also includes the implementation and pre-trained weights of 113 feature extractors such as ResNet or VGG19. The possibility of quickly accessing popular architectures and feature extractors makes this repository very useful for building segmentation systems for custom datasets.

### 3.3 Speeding-up the training and evaluation - Ignite

Pytorch Ignite [99] is a high-level library that helps train and evaluate neural networks based on Pytorch in a flexible and transparent way. It is based on four concepts:

- The `Engine` class. It is an abstraction of a processing function that loops over data points a specified number of times to return a result. A trainer is an engine that loops over a training dataset and updates the weights of the model.

The engine output is not restricted and is stored in `engine_object.state.output`. Method `engine_object.run()` executes the processing function the specified number of times passed by argument (`max_epochs`).

- Events and handlers. To improve the flexibility of the engine, an event system is introduced. This allows the user to execute custom code (handlers) at specific steps in the engine's execution (events). Ignite provides the flexibility to attach handlers to an engine through a decorator or a function.

It includes the implementation of custom events and a set of handlers already implemented to check the best model, the training pipeline state, or experiment tracking systems.

- State of `Engine`. This class stores helpful information of the engine such as the output of the process function, the current epoch, iteration, or the seed to use at each epoch for the data.
- Metrics. Pytorch Ignite provides a list of metrics for various computer vision tasks, including image segmentation. These metrics can be attached to `Engine` or used independently. The metrics are computed in an on-line fashion; that is, they have a state that is updated over the iterations, and the metric value is computed once the epoch is completed. It has support for confusion matrixes, IoU, MIoU, precision, recall, and dice coefficient, among others.

Ignite provides a clear and simple method to represent training processes with their `Engine` approach. This engine supports built-in handlers for quick and recurrent interactions with the model and the training state, which makes it useful for research experimentation. In addition, it provides a set of out-of-the-box metrics for semantic segmentation that covers a gap in Pytorch metrics for semantic segmentation.

### 3.4 A configuration system for experiments - YACS

YACS [100] was designed as a lightweight library to define and manage system configurations such as those common to deep learning experimentation. YACS is quite simple to use, you first need to create a project configuration file called `config.py` or `default.py`. This file provides a reference point for all configurable options. It should be well documented and provide reasonable defaults for all options.

In addition to the central configuration file, the experimenter must create a YAML file for each of the experiments. This YAML file contains specific configuration values for the experiment at hand.

### 3.5 Conceptual framework for semantic segmentation

This section presents a visual representation of concepts and processes previously visited on semantic segmentation (see Section 2.2) and their relationship with sound libraries/frameworks that support their creation from Section 3.

This Master's thesis proposed a conceptual framework that encompassed the main process of semantic segmentation research. Identifying the core functionalities covered by mature libraries and pointing out some of the software elements used.

The color coding of the nodes presented in the figures below is as follows:

- The orange nodes are elements of the topic of semantic segmentation with support from Pytorch [101], that is, there is a class to represent this concept or element.
- The yellow nodes are elements of the semantic segmentation topic with support from Segmentation models [98], that is, there is a class to represent such a concept or element.
- The purple nodes are elements of the semantic segmentation topic with support little support that are complemented by Segwork; that is, there is a class to represent such a concept or element.
- The red nodes are elements of the semantic segmentation topic with support from Pytorch Ignite [99], that is, there is a class to represent such concept or element.
- The Pink nodes are elements of the topic of semantic segmentation with the support of YACS [100], that is, there is a class to represent such concept or element.
- The green nodes are the parameters passed to the class constructor to represent concepts or elements.
- The blue nodes are the reason to use such an element or component.

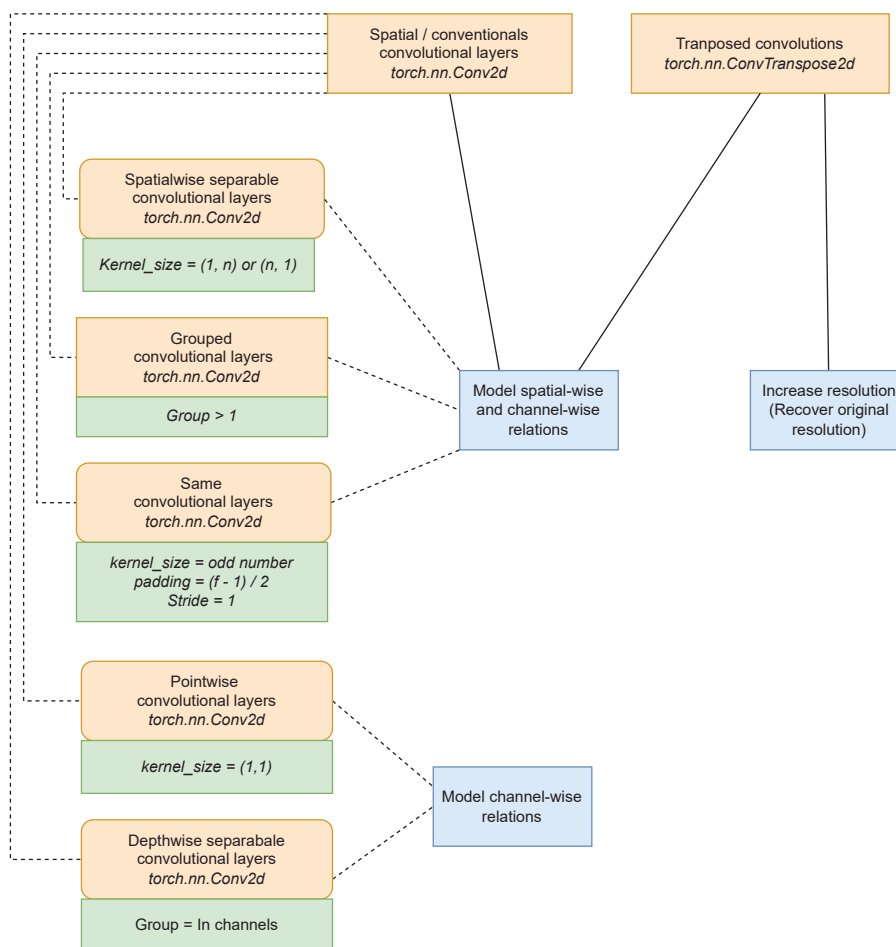
### 3.5.1 Basic layers

All neural networks are made up of smaller modular components called layers. Pytorch provides a wide and high-quality support for implementing these layers in its package `torch.nn` (see Figures 3.1 and 3.2).

The relationship between nodes represented by the edges are as follows:

- Solid line: Relationship between the concept and the role it plays in the semantic segmentation task.
- Dashed line: Relationship between concepts represented by the same class but with different attribute values.

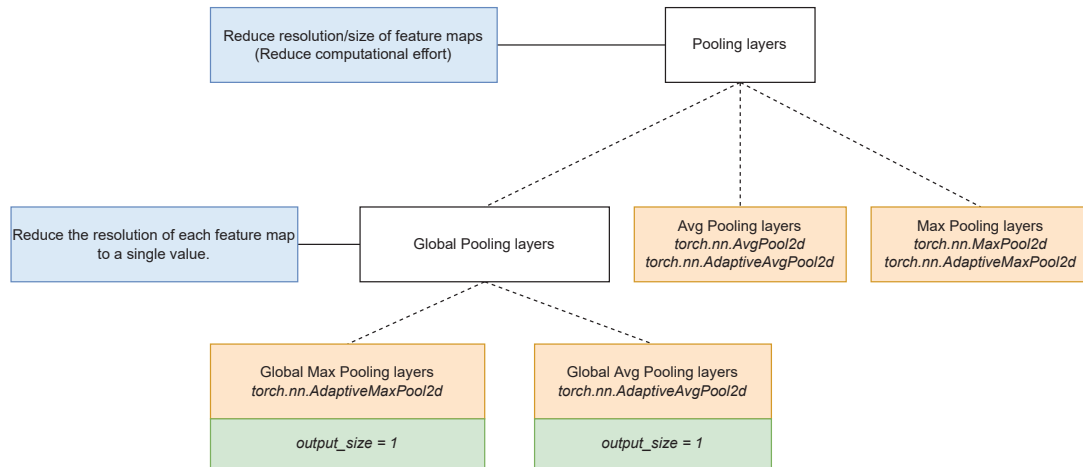
Convolutional layers are easily implemented through the class `torch.nn.Conv2d` (see Figure 3.1). Varying different parameters in the constructor, the layer is instantiated as convention convolutional layers or any other specific convolutional layer such as pointwise convolutional layers or convolutional layers that preserve the resolution of the input (the same convolutional layers) depending on the parameters.



**Figure 3.1:** Convolutional layers types and Pytorch class mapping

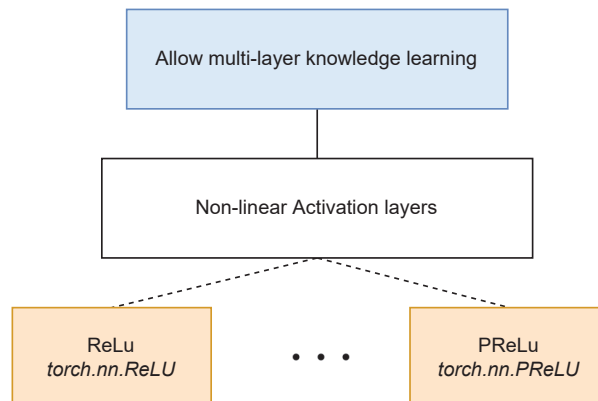
For pooling layers (see figure 3.2), the best support is using `torch.nn.AdaptativeMaxPool2d`

or `torch.nn.AdaptiveAvgPool2d`, which applies a 2D adaptive avg/max pooling over an input signal composed of several input planes. These classes reduce the resolution of the input according to the parameters passed to the constructors.



**Figure 3.2:** Pooling layers types and Pytorch class mapping

Activation functions (see Figure 3.3) are also supported by the package `torch.nn` of Pytorch. It is composed of several non-linear activations, such as the well-known ReLu.

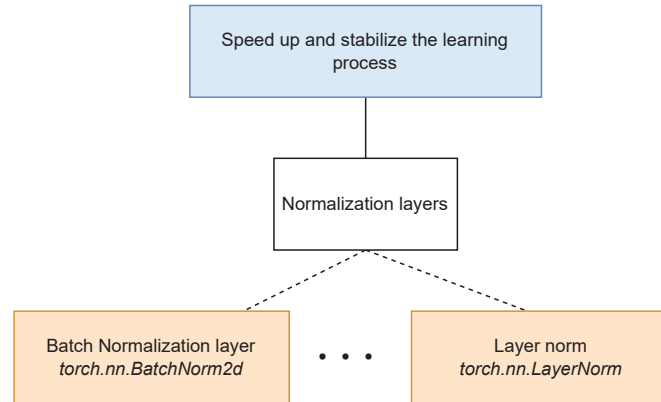


**Figure 3.3:** Non-linear activation layers types and Pytorch class mapping

Related to normalization layers, the popular batch normalization [43] is represented by the class `torch.nn.BatchNorm2d` of Pytorch. It also supports other normalization layers such as the layer normalization method [102].

### 3.5.2 Segmentation models

Layers are stacked and connected to form complex neural networks capable of performing semantic segmentation. A simplification of these neural networks is to see segmentation models as a three-component element formed by a feature extractor, a decoder, and a segmentation head as implemented in Segmentation Pytorch (see Figure 3.5) [98].

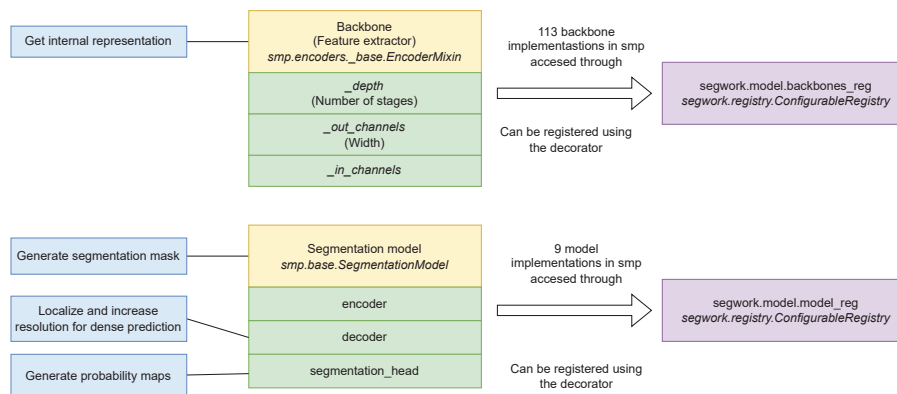


**Figure 3.4:** Normalization layers types and Pytorch class mapping

The feature extractor obtains the relevant features, that is, forms an internal representation of the images. The decoder projects the internal representation to a higher resolution to obtain a dense classification. The segmentation head reduces the number of planes to obtain the probability maps for each class.

Segmentation Models library provides basic support for building these kind of models using their abstract class `SegmentationModel`. It also supports the creation of modular backbones that can be used by subclasses of `SegmentationModel` in a flexible way.

However, a good approach is missing to instantiate different models or backbones in a flexible way. Although a factory method for the backbone is provided, the models must be instantiated using the class directly. The problem of experimenting with models without changing the base code in excess is raised. A registry for models or any other modular component along with a decorator to register the classes for the components is a reasonable solution to this problematic (see figure purple nodes in 3.5).



**Figure 3.5:** Backbone / segmentation model and class mapping. Purple nodes represents a registry of modular components to build configurable components

### 3.5.3 Data pipeline

The basic data pipeline (see Figure 3.6) proposed by Pytorch <sup>6</sup> consists of three components: a set of transformations, a data set object that retrieves images and applies transformations, and a data loader object that batches data and controls data delivery.

These concepts are captured by Pytorch classes, such as `torch.utils.data.Dataset`. They are defined to be as flexible as possible, giving the researcher the responsibility to map between files and images/labels.

It is also a common step to preprocess the file containing the labels in digestible format for the model. Some of the labels are stored in a standard format (XML) that needs to be transformed into a more suitable data structure (torch tensor).

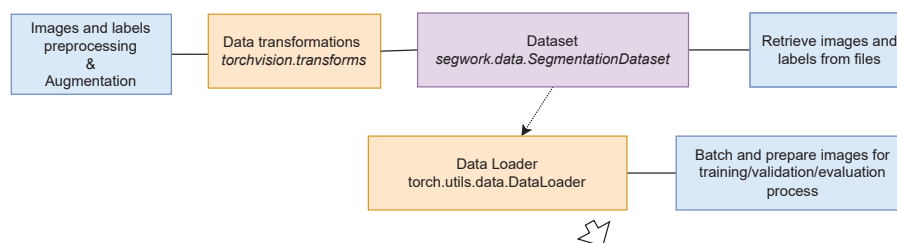


Figure 3.6: Data pipeline for semantic segmentation tasks

### 3.5.4 Training process

The training process (see Figure 3.7) is abstracted by Ignite as a processing function that iterates over a set of data points. This approach allows the researcher to test different configurations for training and evaluation if needed. The processing function is executed by an engine that stores states and throw events that can be handled to interact during training.

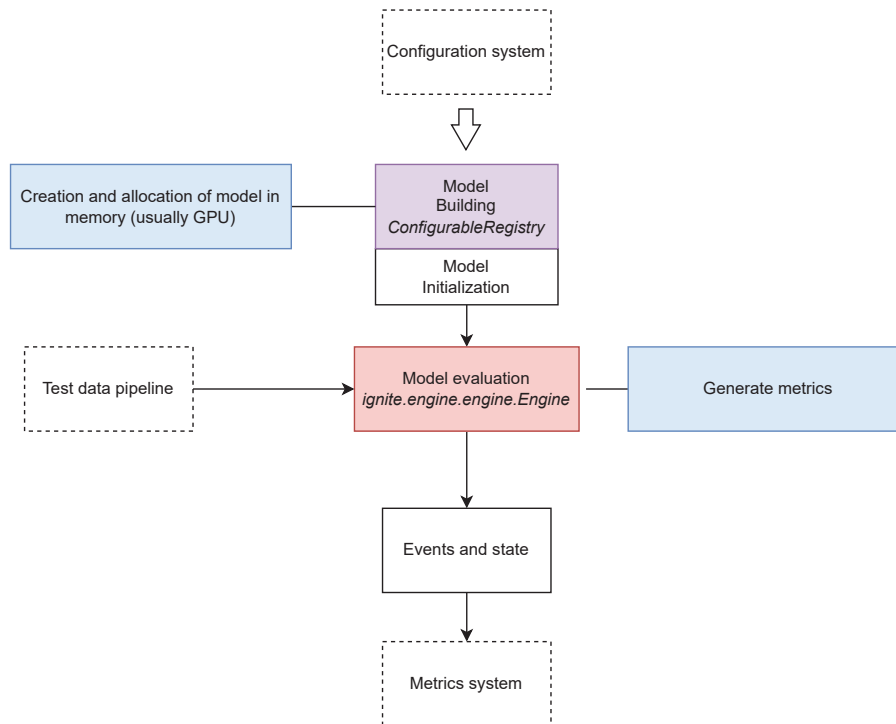
This engine takes data points from the data pipeline (see Figure 3.6) and generates events and data for the metrics pipeline (see Figure 3.9). The experiment variables are supplied by a configuration system (see Figure 3.10).

At the end of the training process, the validation process is not passed or the max epoch is reached, the model checkpoint along with other relevant information is stored to reproduce the experiment and retrieve the trained model.

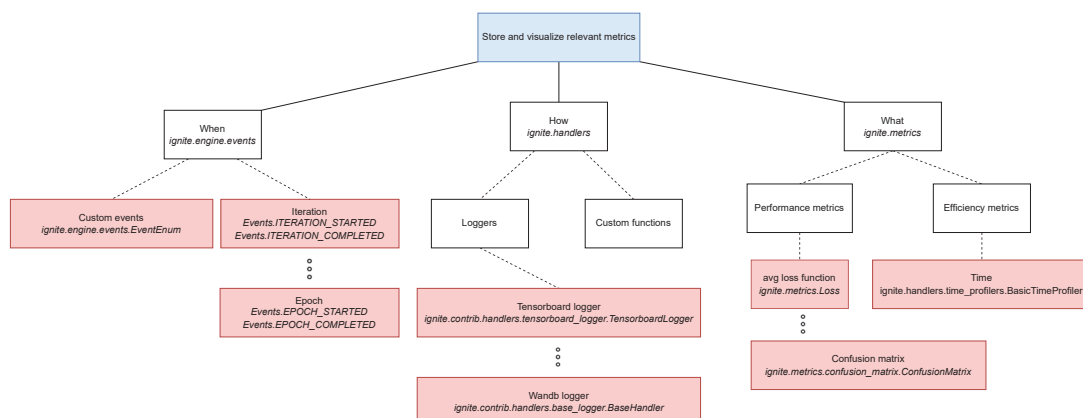
Segmentation models provide different loss functions specialized in semantic segmentation that can be used along Ignite classes.

<sup>6</sup>[https://pytorch.org/tutorials/beginner/basics/data\\_tutorial.html](https://pytorch.org/tutorials/beginner/basics/data_tutorial.html)

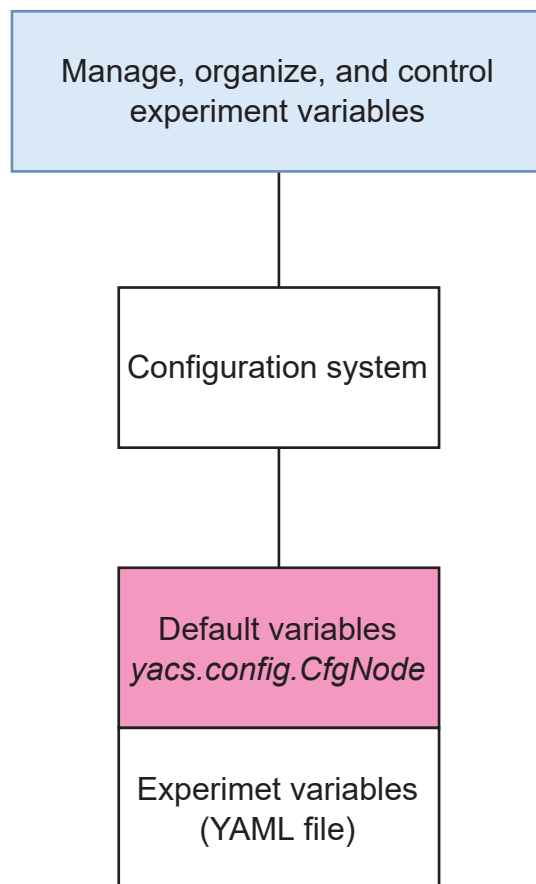




**Figure 3.8:** Evaluation process proposed in the framework



**Figure 3.9:** Metrics tracking and visualization pipeline proposed in the framework



**Figure 3.10:** Configuration system proposed in the framework

## Chapter 4

# Improving the semantic segmentation ecosystem

### Contents

---

|       |                                                           |    |
|-------|-----------------------------------------------------------|----|
| 4.1   | A repository of modular components - ConfigurableRegistry | 50 |
| 4.2   | A dataset for semantic segmentation - SegmentationDataset | 52 |
| 4.2.1 | Class balancing with weights . . . . .                    | 53 |
| 4.3   | Parsing color mask to index masks . . . . .               | 53 |

---

This section describes the software components developed to complement the technology stack selected for the framework and their design. Visit <https://arubiose.github.io/segwork/> for a more in-depth description of the software components.

This section identifies the limitations of existing tools and proposes a design to overcome these limitations. Describing the shortcomings of unimplemented functionality and showing the benefits of the proposed design.

### 4.1 A repository of modular components - ConfigurableRegistry

When evaluating a module, a backbone, or even a whole deep learning model, researchers generally modify small components in the so-called ablation study to understand the contribution of these components to the overall system [59, 60]. These components are represented by classes such as `torch.nn.Module` or `torch.utils.data.Dataset` when using the Pytorch framework.

This fact impules the creation of factory methods for each one of the "family" of components, replacing the direct object construction calls with calls to the factory method. The repository `pytorch-image-models`[103] with more than 19k stars on github, makes extensive use of factory methods for modular components such as datasets, layers, models, and even data transformations.

For a researcher, it might be tedious and slow to have to develop a factory method

for each of the types of component they want to test in their experiment. Here, the `ConfigurableRegistry` class comes in handy. This class follows the Python registry pattern. The registry pattern creates a label map for class types and allows you to access a single registry that is common to all of these classes. `ConfigurableRegistry` allows the creation of new objects and the addition of new class types to the registry using the decorator pattern, providing a fast and elegant way to instantiate and add new components to the registry.

This class is implemented by adding functionality from a base class called `Registry` to `ConfigurableRegistry`. The classes implemented and their responsibility are:

- `Registry`: Dict-like class to store and retrieve classes
- `MutableRegistry`: Add functionality to modify the registry. Add and remove entries.
- `ClassRegistry`: Provides a generic factory method to instantiate the classes contained in the registry.
- `ConfigurableRegistry`: Provides default settings for the factory method and supports the storage of default arguments for the classes in the registry.

The constructor of `ConfigurableRegistry` takes as parameters:

- `(str) attr_name`: Name of the attribute of the class to be registered that contains the label of the registry.
- `(bool) unique`: Whether the addition of classes with the same label should override the existing entry (False) or throw an exception (True)
- `(Callable) register_hook`: The constructor accepts an optional function that will receive the key and value as arguments when registering a new item. This hook allows retrocompatibility with third-party component repository, such as the backbone repository in `segmentation_models.pytorch`[\[98\]](#).
- `(Callable) get_hook`: The constructor accepts an optional function that will receive the value as an argument when retrieving an item. This hook allows for the modification of arguments or any other functionality required by the solution.
- `(Dict) initial_registry`: Dict-like object to initialize the registry. It is validated by the function `_validate_initial_registry` that can be overridden in subclasses that extend the functionality of `ConfigurableRegistry`.
- `(str) class_key`: The values in `ConfigurableRegistry` are dict objects as well, `class_key` specify the key to look for when retrieving the class.
- `(str) attr_args`: Similarly, `attr_name`, `attr_args` is the name of the attribute of the class that stores the default arguments.
- `(str) attr_kwargs`: The same as `attr_args`, but stores keyword arguments.

- (str) `additional_args`: The same as `attr_args`, but stores any additional information that may be useful. For example, additional information to be used in any of the hooks.

As described above, to provide flexibility in the process of building new objects using `ConfigurableRegistry`, during the implementation of the classes to be included in the registry, the developer can define static attributes for:

- The label the class would have in the registry (`attr_name`). The label of the class can also be specified as an argument in the decorator.
- Default arguments passed to the constructor when creating an object (`attr_args` and `attr_kwargs`).
- Additional information of the component (`additional_args`).

Also, the framework provides two objects of this class already populated to speed-up research:

- `backbones_reg`: Object populated with tested backbones from the `pytorch-image-models`[103] and `segmentation_models.pytorch`[98]. It includes 113 feature extractors, including ResNet, VGG19, EfficientNet-B0 to EfficientNet-B6, and many more.
- `models_reg`: Object populated with segmentation models tested from `segmentation_models.pytorch`[98]. It includes nine modifiable model architectures, including Unet, Unet++, DeepLab3, DeepLab3+ and PSPNet.

In figure 6.1 from Appendix C, a diagram <sup>1</sup> is shown. Diagrams for the following classes are also included in the Appendix.

## 4.2 A dataset for semantic segmentation - `SegmentationDataset`

Torchvision and Pytorch provide some primitive classes for machine learning datasets (`torch.utils.data.Dataset` and `torchvision.datasets.VisionDataset`). However, the classes provided are very simple to give complete flexibility to the user. A segmentation dataset can be represented by an abstract class, reducing the boilerplate required to define a class with the `Dataset` class.

The Master's thesis framework includes a class called `SegmentationDataset` that provides a basic skeleton for managing a segmentation dataset. `SegmentationDataset` inherits from `torchvision.datasets.VisionDataset` and adds a new parameter to the constructor to specify the role of the dataset, that is, a parameter (str) `split` is included to specify whether the data set is used for training, validating, or evaluating a model.

---

<sup>1</sup>Created by Pycharm using the diagrams plug-in, a Python IDE for professionals

It includes two static attributes to specify the height and width of the images contained in the dataset and six methods decorated with the `property` decorator<sup>2</sup>:

- `images`: Returns a list containing the paths of the image files.
- `annotations`: Returns a list containing the paths of the label files.
- `mask_color`: Returns a dict-like object containing maps between colors and classes.
- `num_classes`: Returns the number of classes in the dataset
- `classes`: Returns a list containing the classes.
- `num_data_points`: Returns the number of data points (images/labels) contained in the dataset.

`SegmentationDataset` implements the `__len__()` and `__getitem__(idx)`. It also includes abstract methods: `load_image(idx)`, `load_label(idx)`, `load_weight_label(idx)`. The latter is used in the method `compute_class_weights(weight_algorithm, path)` to calculate the class weights following the approach described in [96].

See Figure 6.2 for the diagram.

#### 4.2.1 Class balancing with weights

To cope with imbalanced datasets, a common problem in segmentation datasets, Seg-work counts with two abstract classes: one for the counting of pixels (`PixelCounter`) and another for using this counter to calculate the weights (`WeightAlgorithm`). Seg-work provides an implementation of the counter using Numpy [104], and two algorithms to calculate the weights:

- `LinearWeight`. This algorithm is an association of classes to the total number of pixels belonging to that class in the dataset.
- `NumpyMedianFrequencyWeight`. An implementation of the Median frequency balancing algorithm [96] with numpy.

See Figure 6.3 for the diagram.

### 4.3 Parsing color mask to index masks

Many custom datasets have their segmentation mask in the image format<sup>3</sup>, that is, a PNG or JPG file. This format is not digestible for segmentation models, which requires

---

<sup>2</sup>This design decision has been made because it is better to define this properties while implementing the dataset than during execution time

<sup>3</sup>For example: <https://www.kaggle.com/datasets/bulentsiyah/semantic-drone-dataset>

the labels to be class index masks. Segwork provides two transformation classes to deal with this problem. These classes follow the format specified by Pytorch<sup>4</sup>:

- `ColorMasktoIndexMask`: Transform a color mask into a class index mask using a color map specified in the constructor.
- `IndexMasktoColorMask`: Transform a class index mask into a color segmentation mask using the color map specified in the constructor.

Taking advantage of the `ColorMasktoIndexMask`, segwork provides a function called `generate_numpy_files` that takes as argument a `path`, a `dataset`, a `color_map` and a flag (`index_name`) to store the files with index names or class names and transforms the color mask of the dataset into index class mask to store them in the `path`.

See Figure 6.4 for the diagram.

---

<sup>4</sup>[https://pytorch.org/tutorials/beginner/data\\_loading\\_tutorial.html#transforms](https://pytorch.org/tutorials/beginner/data_loading_tutorial.html#transforms)

# Chapter 5

## Usage of the framework

### Contents

---

|       |                                                   |    |
|-------|---------------------------------------------------|----|
| 5.1   | Development environment . . . . .                 | 55 |
| 5.2   | How to use it . . . . .                           | 55 |
| 5.2.1 | Configurable registry . . . . .                   | 56 |
| 5.2.2 | SegmentationDataset and WeightAlgorithm . . . . . | 59 |
| 5.2.3 | Numpy mask generator and mask transform . . . . . | 60 |

---

This section provides a description of the tools used to develop the software artifacts proposed to improve the semantic segmentation framework and how an user can use them in their projects. A description on how to use the third-party libraries of the framework is not included. To learn all the features of these tools, please, visit their documentation.

Jupyter notebooks with the examples below can be found in the appendix A, or in the github repository under the notebooks folder.

### 5.1 Development environment

The following tools have been used during the development of this project.

- VSCode as main Integrated Development Environment (IDE).
- Python and Python libraries reviewed in section 3.
- Git and github as version control manager and remote repository.

### 5.2 How to use it

To install the package is as easy as using the following command (Using pip):

```
pip install git+https://github.com/ARubiose/segwork.git
```

## 5.2.1 Configurable registry

### 5.2.1.1 Create your own registry of modular components

Creating your own repository of modular components is very easy. First you need to import the class `ConfigurableRegistry` from the Segwork package.

```
from segwork import ConfigurableRegistry
```

And then instantiate the registry for the classes. For instance, a registry of datasets to train our models using different datasets.

```
dataset_reg = ConfigurableRegistry(  
    class_key='dataset',  
    unique = True,  
    additional_args=['transform', 'target_transform'],  
)
```

In this examples we have set the `unique` attribute to `True`, if we try to store a class with the same key, the system will throw an error (see Figure 5.1).

```
KeyError: 'Entry with key drone already exists. Set the unique attribute to false to overried items.'
```

**Figure 5.1:** Unique key error in `ConfigurableRegistry`. If user tries to add an item with an already existing key, the system will throw a `KeyError`

### 5.2.1.2 Adding items to the registry

There are two ways to add items to the registry. Using the `register` decorator at the time of class definition or following the same approach as adding an item to a dictionary.

#### 5.2.1.2.1 Using the register decorator

To use the registry decorator, first you need to import the registry into the file where the component (class) is implemented. For instance, let us use the `models_reg` used by Segwork.

```
from segwork.model import models_reg
```

```
@models_reg.register
```

```

class NeuralNetworkDecorated(nn.Module):

    _register_name='Net'

    _default_kwargs = {
        'size' : 28
    }

    def __init__(self, size: int = 28):
        super(NeuralNetworkDecorated, self).__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(size*size, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10),
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

```

In this example, the key for the model is *Net*, and the default kwargs passed to the factory method is `size=28`. If a new version of the model is wanted to be included in the experiments, it would be as easy as creating another class with different arguments and key. However, these arguments can also be defined in the configuration file, giving the user the flexibility to choose the appropriate place to define them.

```

@models_reg.register
class NeuralNetworkDecoratedB(nn.Module):

    _register_name='NetBig'

    _default_kwargs = {
        'size' : 112
    }

```

The output of printing the registry will show the status of the registry (see Figure 5.2).

#### 5.2.1.2.2 Adding an element like a dictionary item

It is also possible to include external classes or already implemented classes that the user does not want to modify. The only condition is that the dictionary item must include `class_key` of the registry containing the class, otherwise the system will throw an error.

```
ConfigurableRegistry
  attr_name: _register_name
  unique: False
  Number of registered classes: 11
  Registered classes: ['unet', 'unet++', 'manet', 'linknet', 'fpn', 'psp', 'pan', 'deeplabv3',
'deeplabv3plus', 'Net', 'NetBig']
  Class key: model
  Attribute args: _default_args
  Attribute kwargs: _default_kwargs
  Additional info from attributes: []
```

**Figure 5.2:** models\_reg registry status print after adding two models to the registry

For example, if the user defines *NetBig* and afterwards he/she wants to include it in the registry, but uses the wrong key, it would throw an error (see Figure 5.3)

```
models_reg['NetBig'] = {
    'wrong_key': NeuralNetworkDecoratedB
}
```

```
AssertionError: Value must have a key model containing a reference to the class.
```

**Figure 5.3:** Wrong class\_key error

The correct way to add the item to the registry would be using the `class_key` of the registry (see figure 5.2 to check the correct key).

```
models_reg['NetBig'] = {
    'model': NeuralNetworkDecoratedB
}
```

### 5.2.1.3 Calling the factory method

To use the classes in a registry in your scripts, the method:

`registry_object.get_instance(key, *args, **kwargs)`

must be used. For example, to use the model created previously, the user only needs one line of code.

```
model = models_reg.get_instance('Net')
```

This method also accepts arguments that will be passed to the constructor of the class, for example:

```
model = models_reg.get_instance('Net', size=224)
```

#### 5.2.1.4 Integration with Segmentation Pytorch

In the figure 5.2 the model architectures of Segmentation Pytorch are available to use by the user. In addition, Segwork provides easy access to all the backbones of this library in the registry `backbones_reg` and offer retrocompatibility if the user wants to use the API of Segmentation Pytorch<sup>1</sup>. To do so, the backbones must inherit from `smp.encoders._base.EncoderMixin`

```
from segwork import backbones_reg
```

It is possible to add new attributes to be tracked by the registry using the method `registry_object.add_additional_args(attr_name)`. For instance, if we want to add a description of the backbones created by the user:

```
backbones_reg.add_additional_args('_description')
```

#### 5.2.2 SegmentationDataset and WeightAlgorithm

New segmentation datasets must inherit from `SegmentationDataset` and implement the methods listed in section 4.2. This abstraction allows the developer to be guided throughout the implementation of a new dataset by mapping paths for images and labels. An example of the implementation of the aerial drone dataset from Kaggle<sup>2</sup> can be seen in Appendix A.

##### 5.2.2.1 Weight calculator

The weight calculator implemented in Segwork begins with the hypothesis that weights are calculated from pixel counts [96]. Provides an abstract class to implement a pixel counter and provides an implementation using Numpy (note that a pixel counter using Pytorch would be quite similar).

The constructor of `NumpyPixelCounter` takes as arguments the number of classes of the datasets and the dtype of the ndarray that contains the counters.

```
pixel_counter = \
NumpyPixelCounter( num_classes=dataset.num_classes, dtype=np.longlong)
```

To update the state of the counter, method `pixel_counter.update(label)` is used. Calculating the pixel count of a dataset is as easy as iterating over the labels and passing them to the update function. This design pattern is the same as Pytorch Ignite metrics. A simple function using a `SegmentationDataset` to calculate the pixel count would be:

---

<sup>1</sup>Backbones registered in `backbones_reg` can be used with `smp` methods like `get_encoder` or as argument to model classes

<sup>2</sup><https://www.kaggle.com/datasets/bulentsiyah/semantic-drone-dataset>

```
for idx in tqdm(range(dataset.num_data_points)):
    label = dataset.load_weight_label(idx)
    pixel_counter.update(label)
```

This is an expensive process; to reduce the time spent, `NumpyPixelCounter` provides two methods to save and load the counter from a file by passing the path as string or `pathlib.Path` type.

```
pixel_counter.load_counters(os.path.join(ASSETS_DIR, 'pixel_count_all.npz'))
```

Pixel counters can be passed to a weight calculator to calculate the weights as the user wishes. The algorithm must be encapsulated in a class that inherits `WeightAlgorithm`. The class constructor requires a `PixelCounter` object to calculate the weights. Subclasses must implement the method `compute`.

Segwork provides an implementation for a widely used algorithm called Median Frequency Balancing. It is implemented as follows:

```
class NumpyMedianFrequencyWeight(WeightAlgorithm):
    """Median frequency algorithm for the calculation of weights

    https://arxiv.org/pdf/1411.4734.pdf"""

    def compute(self, *args, **kwargs):
        """Compute method to calculate weights based on Median frequency"""
        frequency = \
            np.divide(self.pixel_counter.pixel_count, \
                      self.pixel_counter.class_count)
        weights = np.divide(np.median(frequency), frequency)
        self._weights = weights
        return weights
```

The user can implement their own algorithm following the same approach.

### 5.2.3 Numpy mask generator and mask transform

The way of using `ColorMasktoIndexMask` and `IndexMasktoColorMask` is the same as using any torchvision transformation. The parameter passed to the constructor must be a dict-like object with the tuple colors as keys and the index as value (see figure 5.4)

The input for `ColorMasktoIndexMask` is a tensor of shape `CxHxW` and the output a tensor of shape `HxW`. An example of usage with the `DroneDataset` would be:

```
_, label = dataset[0]
transform = torchvision.transforms.Compose([
```

```

        torchvision.transforms.transforms.PILToTensor(),
        ColorMasktoIndexMask(colors=dataset.mask_colors_index)
    ])
    index_mask = transform(label)

```

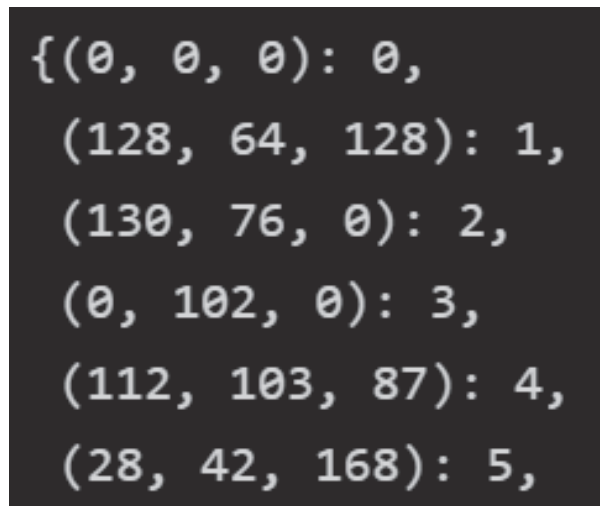
Looking at the other transformation, it does the opposite transformation. An example of usage would be as follows:

```

reverse_transform = torchvision.transforms.Compose([
    IndexMasktoColorMask(colors=dataset.mask_colors_index),
    torchvision.transforms.ToPILImage()
])

color_mask = reverse_transform(index_mask)

```



**Figure 5.4:** Format of colormap for transformations. Partial output of `dataset.mask_colors_index` for DroneDataset

### 5.2.3.1 Generate numpy mask from color masks

The `generate_numpy_files` takes three parameters and one optional parameters.

- `path` Path to the directory where the numpy files are stored.
- `dataset` Dataset that implements the method `load_label` (If inherit from `SegmentationDataset`, this method is enforce by the abstract class). `load_label` should return a PIL image with the color mask.
- `colormap` A dict-like object with the format shown in figure 5.4.
- `index_name`.

An example usage using the DroneDataset is shown below:

```
generate_numpy_files(  
    path = dataset.TRAINING_SEMANTICS,  
    dataset = dataset,  
    color_map = dataset.mask_colors_index  
)
```

# Chapter 6

## Conclusions

### Contents

---

|                                   |    |
|-----------------------------------|----|
| 6.1 Contributions . . . . .       | 63 |
| 6.2 Future perspectives . . . . . | 64 |

---

This Master's thesis has proposed a framework to facilitate research on Semantic segmentation based on a set of already existing and sound deep learning tools along with key software components for fast experimentation.

The literature of semantic segmentation is very extensive, and it has been limited to the following. CNN-based models, RGB input, and supervised training. Taking into consideration that the author of this Master's thesis is no expert in deep learning, the realization of this project has been a continuous discovery of new knowledge.

Although a first view of the Master thesis was to build a framework using Pytorch vanilla, the tool review carried out showed that mature libraries were already established in the field<sup>1</sup>, providing extensive benefits if they are used. For this reason, instead of "reinventing the wheel" and building a software solution from scratch, the proposed package leverages the contribution of open-source projects and provides complementary tools for experimenting.

### 6.1 Contributions

Following the goals presented in the introduction, this Master's thesis serves as an introduction to CNN-based semantic segmentation for the reader and provides an overview of SOTA models that caused an impact in the field at the moment. It also describes the core processes of semantic segmentation research.

An overview of the currently maintained and starred open-source project is performed and represented in a conceptual framework to easily associate concepts with software components (mainly classes that belong to the libraries in the proposed framework).

---

<sup>1</sup>Segmentation Pytorch repository has been used to win many competitions

It gives a verified tool belt for new researchers to experiment with segmentation, reducing the amount of effort spent building models by providing fast implementation of out-of-the-box models and simplifying the management of modular components.

It also introduces the reader to the core components of the libraries that comprise such a tool belt and adds additional functionality for quick and simple experimentation with semantic segmentation models. These components are built according to tested design patterns.

## **6.2 Future perspectives**

Segwork is build flexible enough to adapt to new versions of Segmentation Pytorch and adhere to the patterns established by torchvision in their transformations. A first future perspective must be the maintenance of the proposed package. In a future version, it must give support to

- Visual transformers for semantic segmentation. A new research direction that is gaining momentum due to its simplicity and scalability.
- Provide other architectures not supported for Segmentation Pytorch.
- Extension of segwork to other computer vision tasks

# Bibliography

- [1] Neil C. Thompson et al. “The Computational Limits of Deep Learning”. In: *CoRR* abs/2007.05558 (2020). arXiv: 2007.05558. URL: <https://arxiv.org/abs/2007.05558>.
- [2] A. M. Turing. “I.—COMPUTING MACHINERY AND INTELLIGENCE”. In: *Mind* LIX.236 (Oct. 1950), pp. 433–460. ISSN: 0026-4423. DOI: 10.1093/mind/LIX.236.433. URL: <https://doi.org/10.1093/mind/LIX.236.433>.
- [3] Joost N Kok et al. “Artificial intelligence: definition, trends, techniques, and cases”. In: *Artificial intelligence* 1 (2009), pp. 270–299.
- [4] James Moor. “The Dartmouth College artificial intelligence conference: The next fifty years”. In: *Ai Magazine* 27.4 (2006), pp. 87–87.
- [5] John McCarthy. “What is artificial intelligence?” In: (2004). URL: <http://jmc.stanford.edu/articles/whatisai/whatisai.pdf>.
- [6] David H Hubel and Torsten N Wiesel. “Receptive fields of single neurones in the cat’s striate cortex”. In: *The Journal of physiology* 148.3 (1959), p. 574.
- [7] IBM. *What is Computer Vision?* URL: <https://www.ibm.com/topics/computer-vision>.
- [8] Richard Szeliski. *Computer vision: algorithms and applications*. Springer Science & Business Media, 2010.
- [9] Nicu Sebe et al. *Machine learning in computer vision*. Vol. 29. Springer Science & Business Media, 2005.
- [10] Justin Johnson. *Lecture 1: Introduction to Deep Learning for Computer Vision*. Aug. 22020.
- [11] Branislav Mičušík and Jana Košecká. “Semantic segmentation of street scenes by superpixel co-occurrence and 3d geometry”. In: *2009 IEEE 12th International Conference on Computer Vision Workshops, ICCV Workshops*. IEEE. 2009, pp. 625–632.
- [12] Niall O’ Mahony et al. “Deep Learning vs. Traditional Computer Vision”. In: *CoRR* abs/1910.13796 (2019). arXiv: 1910.13796. URL: <http://arxiv.org/abs/1910.13796>.
- [13] Yanming Guo et al. “Deep learning for visual understanding: A review”. In: *Neurocomputing* 187 (2016). Recent Developments on Deep Big Vision, pp. 27–48. ISSN: 0925-2312. DOI: <https://doi.org/10.1016/j.neucom.2015.09.116>. URL: <https://www.sciencedirect.com/science/article/pii/S0925231215017634>.

- [14] Jinjiang Wang et al. “Deep learning for smart manufacturing: Methods and applications”. In: *Journal of manufacturing systems* 48 (2018), pp. 144–156.
- [15] Shufei Li, Pai Zheng, and Lianyu Zheng. “An ar-assisted deep learning-based approach for automatic inspection of aviation connectors”. In: *IEEE Transactions on Industrial Informatics* 17.3 (2020), pp. 1721–1731.
- [16] Mohammad Moeen Valipoor and Angélica de Antonio. “Recent trends in computer vision-driven scene understanding for VI/blind users: a systematic mapping”. In: *Universal Access in the Information Society* (2022), pp. 1–23.
- [17] Andre Esteva et al. “Deep learning-enabled medical computer vision”. In: *NPJ digital medicine* 4.1 (2021), pp. 1–9.
- [18] J. Brownlee. *Deep Learning for Computer Vision: Image Classification, Object Detection, and Face Recognition in Python*. Machine Learning Mastery, 2019.
- [19] Shervin Minaee et al. “Image segmentation using deep learning: A survey”. In: *IEEE transactions on pattern analysis and machine intelligence* (2021).
- [20] Alexander Kirillov et al. “Panoptic segmentation”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2019, pp. 9404–9413.
- [21] Yifan Liu. “Efficient Fully Convolutional Networks for Dense Prediction Tasks”. PhD thesis. 2021.
- [22] Qusay Sellat et al. “Intelligent Semantic Segmentation for Self-Driving Vehicles Using Deep Learning”. In: *Computational Intelligence and Neuroscience* 2022 (2022).
- [23] Tanmay Anand et al. “AgriSegNet: Deep aerial semantic segmentation framework for IoT-assisted precision agriculture”. In: *IEEE Sensors Journal* 21.16 (2021), pp. 17581–17590.
- [24] Nils Plath, Marc Toussaint, and Shinichi Nakajima. “Multi-class image segmentation using conditional random fields and global classification”. In: *Proceedings of the 26th Annual International Conference on Machine Learning*. 2009, pp. 817–824.
- [25] Amit Sethi. “Interaction between Modules in Learning Systems for Vision Applications”. AAI3223715. PhD thesis. USA, 2006. ISBN: 9780542775611.
- [26] Irem Ulku and Erdem Akagündüz. “A survey on deep learning-based architectures for semantic segmentation on 2d images”. In: *Applied Artificial Intelligence* (2022), pp. 1–45.
- [27] Philipp Krähenbühl and Vladlen Koltun. “Efficient inference in fully connected crfs with gaussian edge potentials”. In: *Advances in neural information processing systems* 24 (2011).
- [28] Liang-Chieh Chen et al. “DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs”. In: *CoRR* abs/1606.00915 (2016). arXiv: 1606.00915. URL: <http://arxiv.org/abs/1606.00915>.
- [29] Marvin TT Teichmann and Roberto Cipolla. “Convolutional CRFs for semantic segmentation”. In: *arXiv preprint arXiv:1805.04777* (2018).
- [30] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. “Imagenet classification with deep convolutional neural networks”. In: *Advances in neural information processing systems* 25 (2012).

- [31] Walter Hugo Lopez Pinaya et al. “Convolutional neural networks”. In: *Machine learning*. Elsevier, 2020, pp. 173–191.
- [32] Wenjie Luo et al. “Understanding the effective receptive field in deep convolutional neural networks”. In: *Advances in neural information processing systems* 29 (2016).
- [33] Vincent Dumoulin and Francesco Visin. “A guide to convolution arithmetic for deep learning”. In: *arXiv preprint arXiv:1603.07285* (2016).
- [34] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [35] Yann LeCun, Koray Kavukcuoglu, and Clément Farabet. “Convolutional networks and applications in vision”. In: *Proceedings of 2010 IEEE international symposium on circuits and systems*. IEEE. 2010, pp. 253–256.
- [36] Vinod Nair and Geoffrey E Hinton. “Rectified linear units improve restricted boltzmann machines”. In: *ICML*. 2010.
- [37] Ken Chatfield et al. “Return of the devil in the details: Delving deep into convolutional nets”. In: *arXiv preprint arXiv:1405.3531* (2014).
- [38] Geoffrey E Hinton et al. “Improving neural networks by preventing co-adaptation of feature detectors”. In: *arXiv preprint arXiv:1207.0580* (2012).
- [39] Matthew D Zeiler and Rob Fergus. “Visualizing and understanding convolutional networks”. In: *European conference on computer vision*. Springer. 2014, pp. 818–833.
- [40] Karen Simonyan and Andrew Zisserman. “Very deep convolutional networks for large-scale image recognition”. In: *arXiv preprint arXiv:1409.1556* (2014).
- [41] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [42] Christian Szegedy et al. “Going deeper with convolutions”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2015, pp. 1–9.
- [43] Sergey Ioffe and Christian Szegedy. “Batch normalization: Accelerating deep network training by reducing internal covariate shift”. In: *International conference on machine learning*. PMLR. 2015, pp. 448–456.
- [44] Christian Szegedy et al. “Rethinking the inception architecture for computer vision”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 2818–2826.
- [45] Jie Hu, Li Shen, and Gang Sun. “Squeeze-and-excitation networks”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 7132–7141.
- [46] Xavier Glorot and Yoshua Bengio. “Understanding the difficulty of training deep feedforward neural networks”. In: *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings. 2010, pp. 249–256.
- [47] Kaiming He and Jian Sun. “Convolutional neural networks at constrained time cost”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2015, pp. 5353–5360.

- [48] Kaiming He et al. “Deep residual learning for image recognition”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.
- [49] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. “Highway networks”. In: *arXiv preprint arXiv:1505.00387* (2015).
- [50] Rupesh K Srivastava, Klaus Greff, and Jürgen Schmidhuber. “Training very deep networks”. In: *Advances in neural information processing systems* 28 (2015).
- [51] Jason Kuen et al. “DelugeNets: deep networks with efficient and flexible cross-layer information inflows”. In: *Proceedings of the IEEE International Conference on Computer Vision Workshops*. 2017, pp. 958–966.
- [52] Gustav Larsson, Michael Maire, and Gregory Shakhnarovich. “Fractalnet: Ultra-deep neural networks without residuals”. In: *arXiv preprint arXiv:1605.07648* (2016).
- [53] Sergey Zagoruyko and Nikos Komodakis. “Wide residual networks”. In: *arXiv preprint arXiv:1605.07146* (2016).
- [54] Gao Huang et al. “Densely connected convolutional networks”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 4700–4708.
- [55] Christian Szegedy et al. “Inception-v4, inception-resnet and the impact of residual connections on learning”. In: *Thirty-first AAAI conference on artificial intelligence*. 2017.
- [56] François Chollet. “Xception: Deep learning with depthwise separable convolutions”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 1251–1258.
- [57] Saining Xie et al. “Aggregated residual transformations for deep neural networks”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 1492–1500.
- [58] Andrew G Howard et al. “Mobilenets: Efficient convolutional neural networks for mobile vision applications”. In: *arXiv preprint arXiv:1704.04861* (2017).
- [59] Mark Sandler et al. “Mobilenetv2: Inverted residuals and linear bottlenecks”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 4510–4520.
- [60] Xiangyu Zhang et al. “Shufflenet: An extremely efficient convolutional neural network for mobile devices”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 6848–6856.
- [61] Barret Zoph and Quoc V Le. “Neural architecture search with reinforcement learning”. In: *arXiv preprint arXiv:1611.01578* (2016).
- [62] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. “Neural architecture search: A survey”. In: *The Journal of Machine Learning Research* 20.1 (2019), pp. 1997–2017.
- [63] Mingxing Tan and Quoc Le. “Efficientnet: Rethinking model scaling for convolutional neural networks”. In: *International conference on machine learning*. PMLR. 2019, pp. 6105–6114.

- [64] Jeff Donahue et al. “Decaf: A deep convolutional activation feature for generic visual recognition”. In: *International conference on machine learning*. PMLR. 2014, pp. 647–655.
- [65] Jonathan Long, Evan Shelhamer, and Trevor Darrell. “Fully Convolutional Networks for Semantic Segmentation”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2015. URL: [https://openaccess.thecvf.com/content\\_cvpr\\_2015/papers/Long\\_Fully\\_Convolutional\\_Networks\\_2015\\_CVPR\\_paper.pdf](https://openaccess.thecvf.com/content_cvpr_2015/papers/Long_Fully_Convolutional_Networks_2015_CVPR_paper.pdf).
- [66] Ross B. Girshick et al. “Rich feature hierarchies for accurate object detection and semantic segmentation”. In: *CoRR abs/1311.2524* (2013). arXiv: 1311.2524. URL: <http://arxiv.org/abs/1311.2524>.
- [67] Pierre Sermanet et al. “Overfeat: Integrated recognition, localization and detection using convolutional networks”. In: *arXiv preprint arXiv:1312.6229* (2013).
- [68] Hyeonwoo Noh, Seunghoon Hong, and Bohyung Han. “Learning deconvolution network for semantic segmentation”. In: *Proceedings of the IEEE international conference on computer vision*. 2015, pp. 1520–1528.
- [69] Vijay Badrinarayanan, Alex Kendall, and Roberto Cipolla. “Segnet: A deep convolutional encoder-decoder architecture for image segmentation”. In: *IEEE transactions on pattern analysis and machine intelligence* 39.12 (2017), pp. 2481–2495.
- [70] Liang-Chieh Chen et al. “Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs”. In: *IEEE transactions on pattern analysis and machine intelligence* 40.4 (2017), pp. 834–848.
- [71] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. 2015. DOI: 10.48550/ARXIV.1505.04597. URL: <https://arxiv.org/abs/1505.04597>.
- [72] Zongwei Zhou et al. *UNet++: Redesigning Skip Connections to Exploit Multiscale Features in Image Segmentation*. 2019. DOI: 10.48550/ARXIV.1912.05074. URL: <https://arxiv.org/abs/1912.05074>.
- [73] Hengshuang Zhao et al. “Pyramid scene parsing network”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 2881–2890.
- [74] Fisher Yu and Vladlen Koltun. “Multi-scale context aggregation by dilated convolutions”. In: *arXiv preprint arXiv:1511.07122* (2015).
- [75] Kaiming He et al. “Spatial pyramid pooling in deep convolutional networks for visual recognition”. In: *IEEE transactions on pattern analysis and machine intelligence* 37.9 (2015), pp. 1904–1916.
- [76] Hao Zhou et al. “RGB-D Co-attention network for semantic segmentation”. In: *Proceedings of the Asian Conference on Computer Vision*. 2020.
- [77] Changshuo Wang et al. “A brief survey on RGB-D semantic segmentation using deep learning”. In: *Displays* 70 (2021), p. 102080.
- [78] Caner Hazirbas et al. “Fusenet: Incorporating depth into semantic segmentation via fusion-based cnn architecture”. In: *Asian conference on computer vision*. Springer. 2016, pp. 213–228.

- [79] Meng-Hao Guo et al. "Attention mechanisms in computer vision: A survey". In: *Computational Visual Media* (2022), pp. 1–38.
- [80] Ashish Vaswani et al. *Attention Is All You Need*. 2017. DOI: [10.48550/ARXIV.1706.03762](https://arxiv.org/abs/1706.03762). URL: <https://arxiv.org/abs/1706.03762>.
- [81] Alexey Dosovitskiy et al. "An image is worth 16x16 words: Transformers for image recognition at scale". In: *arXiv preprint arXiv:2010.11929* (2020).
- [82] Robin Strudel et al. *Segmenter: Transformer for Semantic Segmentation*. 2021. DOI: [10.48550/ARXIV.2105.05633](https://arxiv.org/abs/2105.05633). URL: <https://arxiv.org/abs/2105.05633>.
- [83] Abhinav Sagar. "ViTBIS: Vision Transformer for Biomedical Image Segmentation". In: (2022). DOI: [10.48550/ARXIV.2201.05920](https://arxiv.org/abs/2201.05920). URL: <https://arxiv.org/abs/2201.05920>.
- [84] Irem Ulku and Erdem Akagündüz. "A survey on deep learning-based architectures for semantic segmentation on 2d images". In: *Applied Artificial Intelligence* (2022), pp. 1–45.
- [85] Fahad Lateef and Yassine Ruichek. "Survey on semantic segmentation using deep learning techniques". In: *Neurocomputing* 338 (2019), pp. 321–348.
- [86] Junjiao Tian et al. "Striking the Right Balance: Recall Loss for Semantic Segmentation". In: (2021). DOI: [10.48550/ARXIV.2106.14917](https://arxiv.org/abs/2106.14917). URL: <https://arxiv.org/abs/2106.14917>.
- [87] *Visual object classes challenge&nbsp;2012 (VOC2012)*. URL: <http://host.robots.ox.ac.uk/pascal/VOC/voc2012/index.html>.
- [88] Marius Cordts et al. "The Cityscapes Dataset for Semantic Urban Scene Understanding". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2016.
- [89] Nathan Silberman et al. "Indoor segmentation and support inference from rgb-d images". In: *European conference on computer vision*. Springer. 2012, pp. 746–760.
- [90] Bolei Zhou et al. *Semantic Understanding of Scenes through the ADE20K Dataset*. 2016. DOI: [10.48550/ARXIV.1608.05442](https://arxiv.org/abs/1608.05442). URL: <https://arxiv.org/abs/1608.05442>.
- [91] Rui Ma, Pin Tao, and Huiyun Tang. "Optimizing data augmentation for semantic segmentation on small-scale dataset". In: *Proceedings of the 2nd international conference on control and computer vision*. 2019, pp. 77–81.
- [92] Zach Eaton-Rosen et al. "Improving data augmentation for medical image segmentation". In: (2018).
- [93] Kaiming He, Ross Girshick, and Piotr Dollar. "Rethinking ImageNet Pre-Training". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2019.
- [94] Kaiming He et al. *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. 2015. DOI: [10.48550/ARXIV.1502.01852](https://arxiv.org/abs/1502.01852). URL: <https://arxiv.org/abs/1502.01852>.
- [95] Shruti Jadon. "A survey of loss functions for semantic segmentation". In: *2020 IEEE Conference on Computational Intelligence in Bioinformatics and Computational Biology (CIBCB)*. IEEE. 2020, pp. 1–7.

- [96] David Eigen and Rob Fergus. *Predicting Depth, Surface Normals and Semantic Labels with a Common Multi-Scale Convolutional Architecture*. 2014. DOI: [10.48550/ARXIV.1411.4734](https://doi.org/10.48550/ARXIV.1411.4734). URL: <https://arxiv.org/abs/1411.4734>.
- [97] Adam Paszke et al. “Pytorch: An imperative style, high-performance deep learning library”. In: *Advances in neural information processing systems* 32 (2019).
- [98] Pavel Yakubovskiy. *Segmentation Models Pytorch*. [https://github.com/qubvel/segmentation\\_models.pytorch](https://github.com/qubvel/segmentation_models.pytorch). 2020.
- [99] V. Fomin et al. *High-level library to help with training neural networks in PyTorch*. <https://github.com/pytorch/ignite>. 2020.
- [100] rbgirshick. *Yet Another Configuration System*. Version 0.1.8. 2020. URL: <https://github.com/rbgirshick/yacs>.
- [101] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035. URL: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- [102] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. *Layer Normalization*. 2016. DOI: [10.48550/ARXIV.1607.06450](https://doi.org/10.48550/ARXIV.1607.06450). URL: <https://arxiv.org/abs/1607.06450>.
- [103] Ross Wightman. *PyTorch Image Models*. <https://github.com/rwightman/pytorch-image-models>. 2019. DOI: [10.5281/zenodo.4414861](https://doi.org/10.5281/zenodo.4414861).
- [104] Charles R. Harris et al. “Array programming with NumPy”. In: *Nature* 585.7825 (Sept. 2020), pp. 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2). URL: <https://doi.org/10.1038/s41586-020-2649-2>.

## **Appendix**

### **Appendix A - Notebooks**

# Registry

## Model registry - Create registry

```
In [ ]: from segwork import ConfigurableRegistry

dataset_reg = ConfigurableRegistry(
    class_key='dataset',
    unique = True,
    additional_args=['transform', 'target_transform'],
)
```

```
In [ ]: from segwork.data.drone_dataset import DroneDataset
```

```
In [ ]: dataset_reg['drone'] = {
    'dataset': DroneDataset}
```

```
In [ ]: dataset_reg
```

```
Out[ ]: ConfigurableRegistry
        attr_name: _register_name
        unique: True
        Number of registered classes: 1
        Registered classes: ['drone']
        Class key: dataset
        Attribute args: _default_args
        Attribute kwargs: _default_kwargs
        Additional info from attributes: ['transform', 'target_transform']
```

## Model registry - Add items to a registry

```
In [ ]: import torch.nn as nn
        from segwork.model import models_reg
```

```
In [ ]: @models_reg.register
        class NeuralNetworkDecorated(nn.Module):
```

```

    _register_name='Net'

    _default_kwargs = {
        'size' : 28
    }

    def __init__(self, size: int = 28):
        super(NeuralNetworkDecorated, self).__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(size*size, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10),
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

class NeuralNetworkDecoratedB(nn.Module):

    _register_name='NetBig'

    _default_kwargs = {
        'size' : 112
    }

```

```

In [ ]: models_reg['NetBig'] = {
        'wrong_key': NeuralNetworkDecoratedB
    }

```

```

-----
AssertionError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_4840\2406053431.py in <module>
      1 models_reg['NetBig'] = {
----> 2     'wrong_key': NeuralNetworkDecoratedB
      3 }

c:\Users\alvar\Projects\segwork\segwork\registry.py in __setitem__(self, key, value)
    148     key (typing.Hashable): Lookup key.
    149     """
--> 150     self._validate_register(key, value)
    151     self._register(key, value)
    152

c:\Users\alvar\Projects\segwork\segwork\registry.py in _validate_register(self, key, value)
    168     """Validate register"""
    169     self._validate_key(key)
--> 170     self._validate_value(value)
    171
    172     def _validate_key(self, key):

c:\Users\alvar\Projects\segwork\segwork\registry.py in _validate_value(self, value)
    376
    377     def _validate_value(self, value: typing.Dict):
--> 378     assert self._class_key in value, f'Value must have a key {self._class_key} containing a reference to the class.'
    379     # Warning if no args are store.
    380

AssertionError: Value must have a key model containing a reference to the class.

```

```

In [ ]: models_reg['NetBig'] = {
        'model': NeuralNetworkDecoratedB
        }

```

```

In [ ]: models_reg

```

```
Out[ ]: ConfigurableRegistry
        attr_name: _register_name
        unique: False
        Number of registered classes: 11
        Registered classes: ['unet', 'unet++', 'manet', 'linknet', 'fpn', 'psp', 'pan', 'deeplabv3', 'deeplabv3plus', 'Net', 'NetBig']
        Class key: model
        Attribute args: _default_args
        Attribute kwargs: _default_kwargs
        Additional info from attributes: []
```

```
In [ ]: model_args = {}
        model = models_reg.get_instance('Net', **model_args)
        model
```

```
Out[ ]: NeuralNetworkDecorated(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)
```

## Backbones registry - Integration with smp

```
In [ ]: import typing

import torch
import torch.nn as nn
import segmentation_models_pytorch as smp

from segwork.model import backbones_reg
```

```
In [ ]: backbones_reg
```

```

Out[ ]: ConfigurableRegistry
        attr_name: _register_name
        unique: False
        Number of registered classes: 113
        Registered classes: ['resnet18', 'resnet34', 'resnet50', 'resnet101', 'resnet152', 'resnext50_32x4d', 'resnext101_32x4d', 'resnext101_32x8d', 'resnext101_32x16d', 'resnext101_32x32d', 'resnext101_32x48d', 'dpn68', 'dpn68b', 'dpn92', 'dpn98', 'dpn107', 'dpn131', 'vgg11', 'vgg11_bn', 'vgg13', 'vgg13_bn', 'vgg16', 'vgg16_bn', 'vgg19', 'vgg19_bn', 'senet154', 'se_resnet50', 'se_resnet101', 'se_resnet152', 'se_resnext50_32x4d', 'se_resnext101_32x4d', 'densenet121', 'densenet169', 'densenet201', 'dense_net161', 'inceptionresnetv2', 'inceptionv4', 'efficientnet-b0', 'efficientnet-b1', 'efficientnet-b2', 'efficientnet-b3', 'efficientnet-b4', 'efficientnet-b5', 'efficientnet-b6', 'efficientnet-b7', 'mobilenet_v2', 'xception', 'timm-efficientnet-b0', 'timm-efficientnet-b1', 'timm-efficientnet-b2', 'timm-efficientnet-b3', 'timm-efficientnet-b4', 'timm-efficientnet-b5', 'timm-efficientnet-b6', 'timm-efficientnet-b7', 'timm-efficientnet-b8', 'timm-efficientnet-l2', 'timm-tf_efficientnet_lite0', 'timm-tf_efficientnet_lite1', 'timm-tf_efficientnet_lite2', 'timm-tf_efficientnet_lite3', 'timm-tf_efficientnet_lite4', 'timm-resnest14d', 'timm-resnest26d', 'timm-resnest50d', 'timm-resnest101e', 'timm-resnest200e', 'timm-resnest269e', 'timm-resnest50d_4s2x40d', 'timm-resnest50d_1s4x24d', 'timm-res2net50_26w_4s', 'timm-res2net101_26w_4s', 'timm-res2net50_26w_6s', 'timm-res2net50_26w_8s', 'timm-res2net50_48w_2s', 'timm-res2net50_14w_8s', 'timm-res2next50', 'timm-regnetx_002', 'timm-regnetx_004', 'timm-regnetx_006', 'timm-regnetx_008', 'timm-regnetx_016', 'timm-regnetx_032', 'timm-regnetx_040', 'timm-regnetx_064', 'timm-regnetx_080', 'timm-regnetx_120', 'timm-regnetx_160', 'timm-regnetx_320', 'timm-regnetx_320', 'timm-regnety_002', 'timm-regnety_004', 'timm-regnety_006', 'timm-regnety_008', 'timm-regnety_016', 'timm-regnety_032', 'timm-regnety_040', 'timm-regnety_064', 'timm-regnety_080', 'timm-regnety_120', 'timm-regnety_160', 'timm-regnety_320', 'timm-skresnet18', 'timm-skresnet34', 'timm-skresnext50_32x4d', 'timm-mobilenetv3_large_075', 'timm-mobilenetv3_large_100', 'timm-mobilenetv3_large_minimal_100', 'timm-mobilenetv3_small_075', 'timm-mobilenetv3_small_100', 'timm-mobilenetv3_small_minimal_100', 'timm-gernet_s', 'timm-gernet_m', 'timm-gernet_l']
        Class key: encoder
        Attribute args: _default_args
        Attribute kwargs: params
        Additional info from attributes: ['pretrained_settings']

```

```

In [ ]: backbones_reg.add_additional_args('_description')

@backbones_reg.register
class DummyBackboneDecorated(nn.Module, smp.encoders._base.EncoderMixin):
    """Dummy encoder to test compatibility with smp architectures

    Testing:
    - Custom attributes in registry
    - To be used in smp framework it is required to inherit from EncoderMixin
    """

    _register_name='Net'

    # Default params
    params = {
        'out_channels': (3, 64, 256, 512),
        'depth': 3
    }

```

```

}

# Additional settings
pretrained_settings = None

_description = 'Formal description of encoder'

def __init__(self, out_channels: typing.List, depth:int):
    super(DummyBackboneDecorated, self).__init__()

    # A number of channels for each encoder feature tensor, List of integers
    self._out_channels: typing.Iterable[int] = out_channels

    # A number of stages in decoder (in other words number of downsampling operations), integer
    # use in in forward pass to reduce number of returning features
    self._depth: int = depth

    # Default number of input channels in first Conv2d Layer for encoder (usually 3)
    self._in_channels: int = 3

    blocks = []

    for idx in range(len(out_channels) - 1):
        blocks.append(nn.Sequential(
            nn.Conv2d(out_channels[idx], out_channels[idx + 1], 3, padding=1),
            nn.Conv2d(out_channels[idx + 1], out_channels[idx + 1], 3, stride=2, padding=1),
        ))

    self.stages = nn.Sequential(*blocks)

def forward(self, x):
    out = [x]

    for stage in self.stages:
        x = stage(x)
        out.append(x)

    return out

```

```

In [ ]: encoder_name = 'Net'

# Framework entrypoint
backbone_fr = backbones_reg.get_instance(encoder_name)

```

```
# SMP entrypoint compatibility
backbone = smp.encoders.get_encoder(encoder_name)

# print(backbone)
print(list(backbones_reg['Net'].keys()))
print(list(backbones_reg['resnet34'].keys()))

['encoder', '_default_args', 'params', 'pretrained_settings', '_description']
['encoder', 'pretrained_settings', 'params']
```

## Output of registered backbone

```
In [ ]: x = torch.rand(1,3,224,224)

out = (backbone(x))

print('Features size...')
for idx, f in enumerate(out):
    print(f'Stage {idx:02d}: {f.size()}')
```

```
Features size...
Stage 00: torch.Size([1, 3, 224, 224])
Stage 01: torch.Size([1, 64, 112, 112])
Stage 02: torch.Size([1, 256, 56, 56])
Stage 03: torch.Size([1, 512, 28, 28])
```

## Using custom backbone

```
In [ ]: model_args = {
    'encoder_name' : 'Net',
    'encoder_depth' : 3,
    'encoder_weights' : None,
    'decoder_channels' : (512, 256, 64),
    'in_channels' : 3,
    'classes' : 20
}

model = smp.Unet(**model_args)
```

```
In [ ]: out = model(x)
out.size()
```

---

Out[ ]: torch.Size([1, 20, 224, 224])

In [ ]: model\_fr = models\_reg.get\_instance('unet', \*\*model\_args)

In [ ]: out\_fr = model\_fr(x)  
out\_fr.size()

Out[ ]: torch.Size([1, 20, 224, 224])

# Segmentation Dataset

```
In [ ]: import os
import csv

from PIL.Image import Image
import numpy as np

from segwork.data import SegmentationDataset
```

```
In [ ]: class DroneDataset(SegmentationDataset):
    """Dataset for Semantic Drone dataset

    The Semantic Drone Dataset focuses on semantic understanding of urban scenes for
    increasing the safety of autonomous drone flight and landing procedures.
    The imagery depicts more than 20 houses from nadir (bird's eye) view acquired at an
    altitude of 5 to 30 meters above ground. A high resolution camera was used to acquire
    images at a size of 6000x4000px (24Mpx). The training set contains 400 publicly available
    images and the test set is made up of 200 private images.

    https://www.tugraz.at/index.php?id=22387"""

    HEIGHT = 4000
    WIDTH = 6000

    def __init__(self, pil_target:bool = True, *args, **kwargs):
        super().__init__(*args, **kwargs)
        _TRAINING_DIR = os.path.join(self.root, 'training_set')
        self.TRAINING_IMAGES_DIR = os.path.join(_TRAINING_DIR, 'images')
        self.TRAINING_SEMANTICS = os.path.join(_TRAINING_DIR, 'gt', 'semantic')
        self.TRAINING_LABELS_DIR = os.path.join(self.TRAINING_SEMANTICS, 'label_images')
        self.TRAINING_LABELS_DIR_NUMPY = os.path.join(self.TRAINING_SEMANTICS, 'label_numpy')
        self.pil_target = pil_target

    @property
    def images(self):
        data_dir = self.TRAINING_LABELS_DIR if self.split == 'train' else self.TRAINING_LABELS_DIR
        return self._get_listdir(data_dir)
```

```

def load_image(self, idx:int):
    return Image.open(self.images[idx]).convert("RGB")

@property
def annotations(self):
    data_dir = self.TRAINING_LABELS_DIR
    return self._get_listdir(data_dir)

def _get_listdir(self, dir:str):
    """Return a list with the path to the files in it"""
    return [os.path.join(dir, file) for file in os.listdir(dir)]

def load_label(self, idx:int):
    if self.pil_target:
        return Image.open(self.annotations[idx]).convert("RGB")
    return self.load_numpy_label(idx)

@property
def mask_colors(self):
    with open(os.path.join(self.TRAINING_SEMANTICS, 'class_dict.csv' ), 'r') as csvfile:
        reader = csv.reader(csvfile)
        return { tuple([int(r.strip()),int(g.strip()),int(b.strip())]) : name for (name, r, g, b) in reader }

@property
def mask_colors_index(self):
    return { key : idx for idx, key in enumerate(self.mask_colors)}

@property
def classes(self):
    return list(self.mask_colors.values())

def load_numpy_label(self, idx:int, *args, **kwargs):
    """Return a :py:class:`numpy.ndarray` with the label for the specified idx"""
    file_name = f'{idx:03d}.npz'
    path_name = os.path.join(self.TRAINING_LABELS_DIR_NUMPY, file_name)
    return np.load(path_name, *args, **kwargs)

def load_weight_label(self, idx):
    """Load label to be used by the calculator"""
    return self.load_numpy_label(idx)

```

```

In [ ]: DATA_DIR = os.path.join('data')
dataset = DroneDataset(

```

```
root = os.path.join(DATA_DIR, 'semantic_drone_dataset'),  
pil_target=True,  
)
```

In [ ]: dataset

Out[ ]: Dataset DroneDataset  
Number of datapoints: 400  
Root location: data\semantic\_drone\_dataset

## PixelCounter and WeightAlgorithm classes

```
In [ ]: # Built-in
import os
import logging
logging.basicConfig(level=logging.DEBUG)

# Third-party packages
import numpy as np
from tqdm import tqdm

# Segwork framework
from segwork.data import DroneDataset, NumpyPixelCounter
```

```
In [ ]: DATA_DIR = os.path.join(os.pardir, 'data')
ASSETS_DIR = 'assets'
```

```
In [ ]: # Instantiate dataset
dataset = DroneDataset(
    root = os.path.join(DATA_DIR, 'semantic_drone_dataset'),
    pil_target=False,          # Target: numpy.ndarray
)
```

Instantiating the numpy pixel counter

```
In [ ]: # The `num_classes = dataset.num_classes - 1` because background is not weighted.
pixel_counter = \
    NumpyPixelCounter( num_classes=dataset.num_classes - 1, dtype=np.longlong)
pixel_counter
```

```
Out[ ]: Counter of pixels with 23 classes.
Pixel count:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0].
Class count:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

```
In [ ]: print(f'Number of pixel per image: {dataset.HEIGHT * dataset.WIDTH}')
label = dataset.load_weight_label(2)
pixel_counter.update(label)
```

```

Out [ ]: Number of pixel per image: 24000000
(array([ 33858, 15296778, 365294, 2688081, 2233916, 0,
        0, 0, 1046133, 0, 743212, 218419,
        0, 0, 0, 150142, 0, 0,
        0, 572787, 0, 0, 651380], dtype=int64),
array([24000000, 24000000, 24000000, 24000000, 24000000, 0,
        0, 0, 24000000, 0, 24000000, 24000000,
        0, 0, 0, 24000000, 0, 0,
        0, 24000000, 0, 24000000,
        0, 24000000, 0, 0, 24000000], dtype=int64))

```

```

In [ ]: pixel_counter.save_counters(os.path.join(ASSETS_DIR, 'pixel_count.npz'))

```

Weight file pixel\_count.npz already exists, replacing file. Pass exist\_ok=False attr to prevent it.

```

In [ ]: pixel_counter.reset_counters()

```

```

Out [ ]: Counter of pixels with 23 classes.
Pixel count:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
Class count:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]

```

```

In [ ]: pixel_counter.load_counters(os.path.join(ASSETS_DIR, 'pixel_count_all.npz'))

```

INFO:segwork.data.balance:Pixel counts loaded from assets\pixel\_count\_all.npz

```

Out [ ]: True

```

```

In [ ]: for idx in tqdm(range(dataset.num_data_points)):
        label = dataset.load_weight_label(idx)
        pixel_counter.update(label)

```

100%|██████████| 400/400 [02:06<00:00, 3.17it/s]

## Using a weight algorithm

```

In [ ]: from segwork.data import NumpyMedianFrequencyWeight
weight_algorithm = NumpyMedianFrequencyWeight( pixel_counter = pixel_counter)

```

```

In [ ]: weights = weight_algorithm.compute()
weights

```

```
Out[ ]: array([16.85484274,  0.09279877,  0.95600028,  0.12590574,  0.41609928,  
              0.462153   ,  2.69340118,  0.57643398,  0.46594422,  0.24907447,  
              1.       ,  2.487816   , 10.87277168,  2.03968625, 28.96594261,  
              3.21416904, 13.60558168,  0.74984716,  6.58144315,  0.83559121,  
              0.57470975,  6.68112193,  1.01167484])
```

## Transformations

```
In [ ]: import os

import torchvision
from segwork.data import ColorMasktoIndexMask, IndexMasktoColorMask, generate_numpy_files, DroneDataset
```

```
In [ ]: DATA_DIR = os.path.join('data')
# Instantiate dataset
dataset = DroneDataset(
    root = os.path.join(DATA_DIR, 'semantic_drone_dataset'),
    pil_target=True,          # Target: numpy.ndarray
)
```

```
In [ ]: # Hard process, it takes on avg 10s
_, label = dataset[0]
transform = torchvision.transforms.Compose([
    torchvision.transforms.PILToTensor(),
    ColorMasktoIndexMask(colors=dataset.mask_colors_index)
])
index_mask = transform(label)
index_mask.size()
```

```
Out[ ]: torch.Size([4000, 6000])
```

```
In [ ]: reverse_transform = torchvision.transforms.Compose([
    IndexMasktoColorMask(colors=dataset.mask_colors_index),
    torchvision.transforms.ToPILImage()
])

color_mask = reverse_transform(index_mask)
color_mask
```

Out[ ]:



### Generate numpy masks

```
In [ ]: generate_numpy_files(
```

```
path = dataset.TRAINING_SEMANTICS,  
dataset = dataset,  
color_map = dataset.mask_colors_index  
)
```

100%|██████████| 400/400 [1:01:12<00:00, 9.18s/it]

## **Appendix B - Segwork documentation**

# Welcome to Segwork's documentation!

Segwork is a Python library that includes a set of tools for semantic segmentation.

## ! Note

This project is part of a Master Thesis for the European Master in Software Engineering (EMSE).

## Content

- [Index](#)
- [Module Index](#)
- [Installation](#)
- [Registry](#)
- [Segmentation dataset](#)
- [Pixel counter and Weight algorithm](#)
- [Transformations](#)

## Installation

To use Segwork, first install it using pip:

```
(.venv) $ pip install git+https://github.com/ARubiose/segwork.git
```

## Registry

Classes for the implementation of a registry of components and a basic factory method.

---

```
class segwork.registry.ClassRegistry(register_hook: Optional[Callable] = None, get_hook: Optional[Callable] = None, initial_registry: Optional[MutableMapping] = None, *args, **kwargs)
```

### Description

Maintains a registry of classes and provides a generic factory for instantiating them. Useful for modular components.

### Args:

register\_hook (typing.Callable): function that will receive the key and value as argument when registering a new item.

get\_hook (typing.Callable): function that will receive the value as an argument when retrieving an item.

initial\_registry (typing.MutableMapping): Dict-like object to initialize the registry with.

### **get\_class(key)**

Get the class associated with the specified key.

Value is not formatted. Value is directly the class without params. Same as get\_item in this class

### **Returns:**

Class associated with the specified key.

### **Raise:**

`KeyError` if the key is not registered.

### **get\_instance(key, \*args, \*\*kwargs)**

Create instance of class associated with the specified key

### **get\_item(key: Hashable, \*args, \*\*kwargs)**

Implementation of get item.

### **is\_unique(key: Hashable)→ bool**

Returns whether a key exists or not in the registry

### **items()→ Iterable[Tuple[Hashable, str]]**

Iterates over all registered classes, in the order they were added.

---

```
class segwork.registry.ConfigurableRegistry(class_key: str, attr_args: str = '_default_args',  
attr_kwargs: str = '_default_kwargs', additional_args=[], **kwargs)
```

Subclass of `ClassRegistry` that includes default settings and additional information of the registered class.

### **Args:**

class\_key (str): Key to look for when retrieving the class.

attr\_args (str): Name of the attribute of the class that stores the default arguments.

attr\_kwargs (str): The same as attr\_args, but stores keyword arguments.

additional\_args (list): The same as attr\_args, but stores any additional information that may be useful.

### **add\_additional\_args(attr\_name: str)**

Add attribute name to be stored in the registry

**get\_class**(key: Hashable)

Get class for the specified key.

**Raise:**

KeyError

**get\_instance**(key: Hashable, \*args, \*\*kwargs)

Get instance of class with default params

**Raise:**

KeyError

**get\_subfield**(key: Hashable, subkey: Hashable, default: Optional[Any] = None)

Get subfield for the specified key

**Raise:**

KeyError

**set\_field**(key: Hashable, subkey: Hashable, value: Any, update: bool = True)

Set field of registry

---

**class** segwork.registry.MutableRegistry(attr\_name: Optional[str] = '\_register\_name', unique: bool = False)

Extends `Registry` with methods to add, modify and remove registered items.

Supports registering classes through decorators.

**Args:**

attr\_name (str): If provided, `register()` will automatically detect the key to use when registering new classes. unique (bool): Determines what happens when two classes are registered with the same key:

- `True`: The second class will replace the first one.
- `False`: A `ValueError` will be raised

**register**(key: Union[type, Hashable])

Decorator that registers a class with the registry.

**Args:**

key: The registry key to use for the registered class. Optional if the registry's `attr_name` is set.

**Raise:**

`ValueError` if ket is not provided.

**`unregister(key: Any)→ type`**

Unregisters the class with the specified key.

**Args:**

key: Lookup key.

**Returns:**

The class that was unregistered.

**Raise:**

`KeyError` if the key is not registered.

---

**`class segwork.registry.Registry`**

Base abstract class for a registry.

**`get(key: Hashable)→ ItemType`**

Get the item matching the key.

**Args:**

key (typing.Hashable): Lookup key.

**Returns:**

Instance of the item matching the key

**`abstract get_item(key: Hashable)→ ItemType`**

Returns registry item with specified key

**`abstract items()→ Iterable[Tuple[Hashable, ItemType]]`**

Get registered classes and their asociated keys.

**Returns**

A generator for iterating over registered pairs (key, class).

**`keys()→ Iterable[Hashable]`**

Get registered keys

**Returns**

A generator for iterating over registered keys.

**`values()→ Iterable[ItemType]`**

Get registered classes.

### Returns

A generator for iterating over registered classes.

## Segmentation dataset

---

```
class segwork.data.SegmentationDataset(root: str, transform: Optional[Callable] = None,  
target_transform: Optional[Callable] = None, transforms: Optional[Callable] = None, split: str = 'train')
```

Common interface to describe segmentation datasets for supervised training

### *abstract property* annotations

Returns a list of paths to the files containing the ground truth masks

### *abstract property* classes

Returns a list of the classes

```
compute_class_weights(weight_algorithm: WeightAlgorithm, path: Optional[Union[str, Path]] =  
None, *args, **kwargs)
```

Basic method to calculate dataset weights.

Make sure your calculator input and your dataset output are compatible through the `target_transform` attribute

### *abstract property* images

Returns a list of paths to the files containing the images

### *abstract* load\_image(*idx: int*)

Returns a `PIL.Image.Image` object for the specified image `idx`

### *abstract* load\_label(*idx*)

Returns a `PIL.Image.Image` object for the specified label `idx`

### *abstract* load\_weight\_label(*idx*)

Load label to be used by the calculator

*abstract property* mask\_colors

Returns a mapping object of the class index and class colors

*property* num\_classes

Returns the number of classes

*property* num\_data\_points

Returns the number of datapoints in the dataset

## Pixel counter and Weight algorithm

---

*class* segwork.data.balance.LinearWeight(pixel\_counter: PixelCounter)

Linear weight algorithm for calculation of weights

*compute*(\*args, \*\*kwargs)

Compute method to calculate weights based on a linear relation

---

*class* segwork.data.balance.LogarithmicWeight(c: float = 1.02, \*args, \*\*kwargs)

Logarithmic algorithm for calculation of weights

*calculate*(\*args, \*\*kwargs)

Compute method to calculate weights based on logarithms

---

*class* segwork.data.balance.NumpyMedianFrequencyWeight(pixel\_counter: PixelCounter)

Median frequency algorithm for calculation of weights

<https://arxiv.org/pdf/1411.4734.pdf>

*compute*(\*args, \*\*kwargs)

Compute method to calculate weights based on Median frequency

---

*class* segwork.data.balance.NumpyPixelCounter(dtype: Optional[dtype] = None, \*args, \*\*kwargs)

Implementation of PixelCounter with numpy.

*Args:*

dtype (numpy.dtype) : dtype of the ndarrays to be used for the operations.

*is\_empty*() → bool

Return whether the calculator is empty or not

*load\_counters*(path: Union[str, Path], \*args, \*\*kwargs)

Update object status en returns the loaded counters

**Raise:**

FileNotFoundError

**save\_counters**(path: Union[str, Path], exist\_ok: bool = True, \*args, \*\*kwargs)

Saves counters in a file.

**update**(label: ndarray)

Updates the total count of pixels per class

---

**class** segwork.data.balance.PixelCounter(num\_classes: int)

Base class to count pixels in an image

**Args:**

num\_classes (int): Number of classes of the dataset.

**property class\_count**

Returns the data structure with the class count.

The class count represents the total number of pixel in images where class c is present

**abstract is\_empty()**

Return whether the calculator is empty or not

**abstract load\_counters**(path: Union[Path, str], \*args, \*\*kwargs)

Update object status en returns the loaded counters

**Raise:**

FileNotFoundError

**property pixel\_count**

Returns the data structure with the pixel count

The pixel count represents the total number of pixel of class c

**abstract save\_counters**(path: Union[Path, str], \*args, \*\*kwargs)

Saves counters in a file.

**abstract update**(label)

Updates the state of the pixel counters

It will update the number of pixels per class

---

**class** `segwork.data.balance.WeightAlgorithm(pixel_counter: PixelCounter)`

Abstract base class to represent algorithm that calculate class weights

**Args:**

pixel\_counter (PixelCounter): PixelCounter object use to calculate the weights

**abstract** `compute(*args, **kwargs)`

Compute weights

## Transformations

---

**class** `segwork.data.transform.ColorMasktoIndexMask(colors: MutableMapping, dtype: Optional[dtype] = None)`

Class to transform RGB color mask to index numpy masks with the specified color mapping

**Args:**

colors (dict): Dict-like object with color mappings.

dtype: (torch.dtype): Type of tensor used in the transformation.

---

**class** `segwork.data.transform.IndexMasktoColorMask(colors: MutableMapping, dtype: dtype = torch.uint8)`

Class to transform index numpy masks to RGB/GreyScale with the specified color mapping.

**Args:**

colors (dict): Dict-like object with color mappings.

dtype: (torch.dtype): Type of tensor used in the transformation.

---

**segwork.data.transform.generate\_numpy\_files(path: Union[str, Path], dataset: SegmentationDataset, color\_map: MutableMapping[Tuple[int, int, int], int], index\_name: bool = True)**

Generate numpy files containing segmentation masks from PIL images

**Args:**

path (str, pathlib.Path): Output path for the numpy files.

dataset (SegmentationDataset): Dataset with labels as color images. It must implement the method `load_label(idx())` to retrieve labels.

color\_map (dict): Dict-like object with color mappings.

index\_name (bool) = True: If *True* generate files are named as *000.npy* onwards.



## Appendix C - Pycharm Diagrams

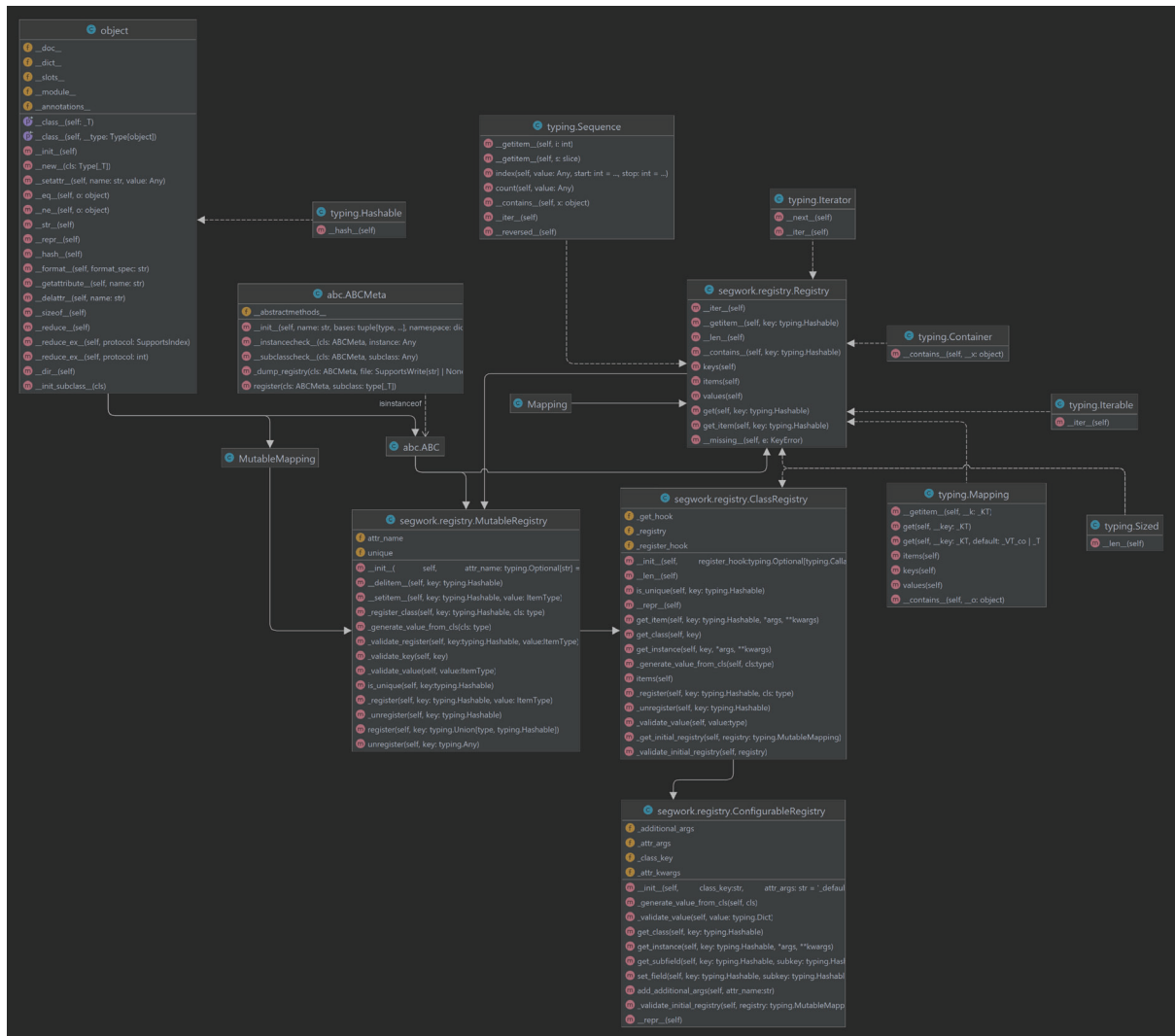
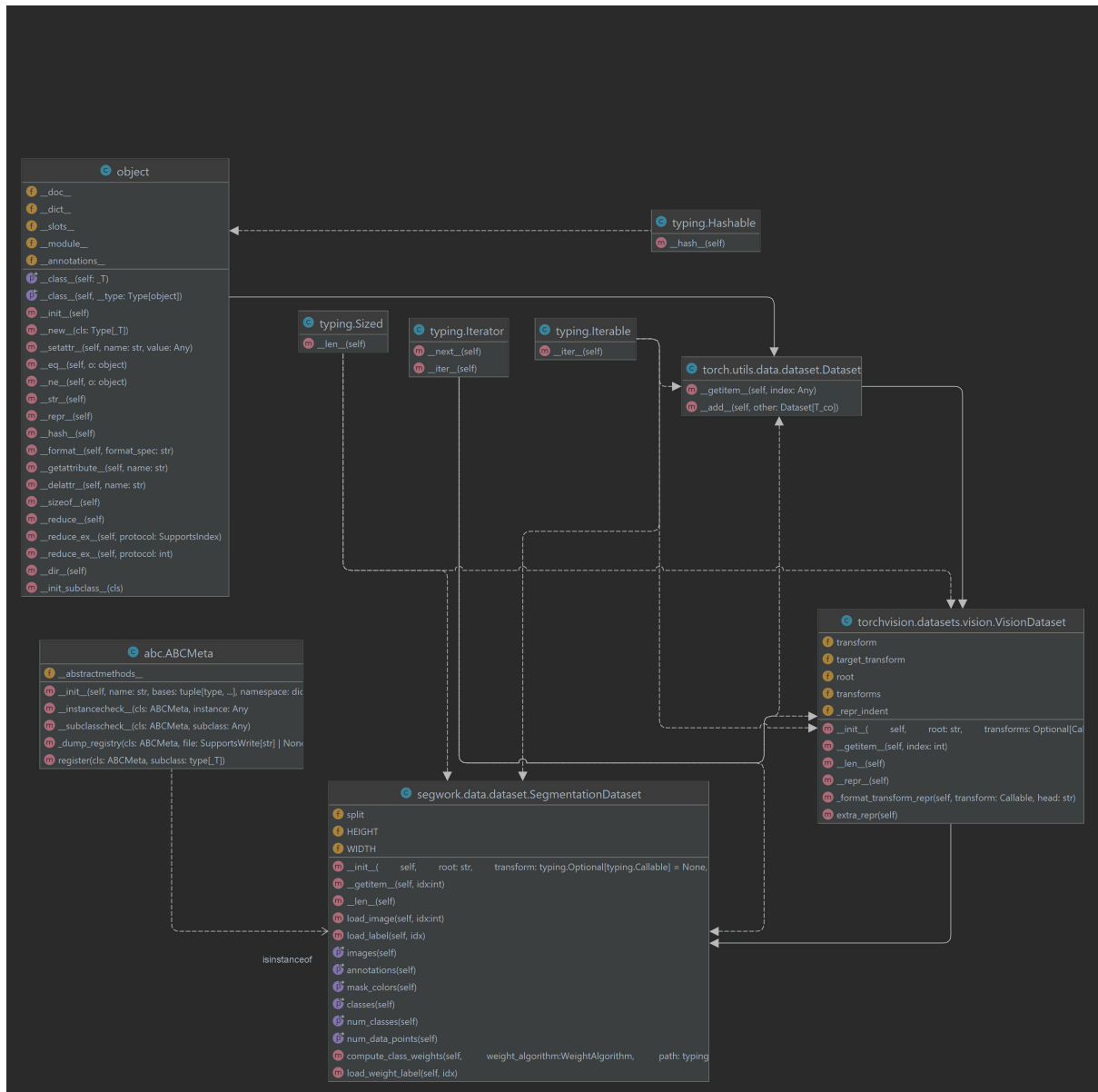
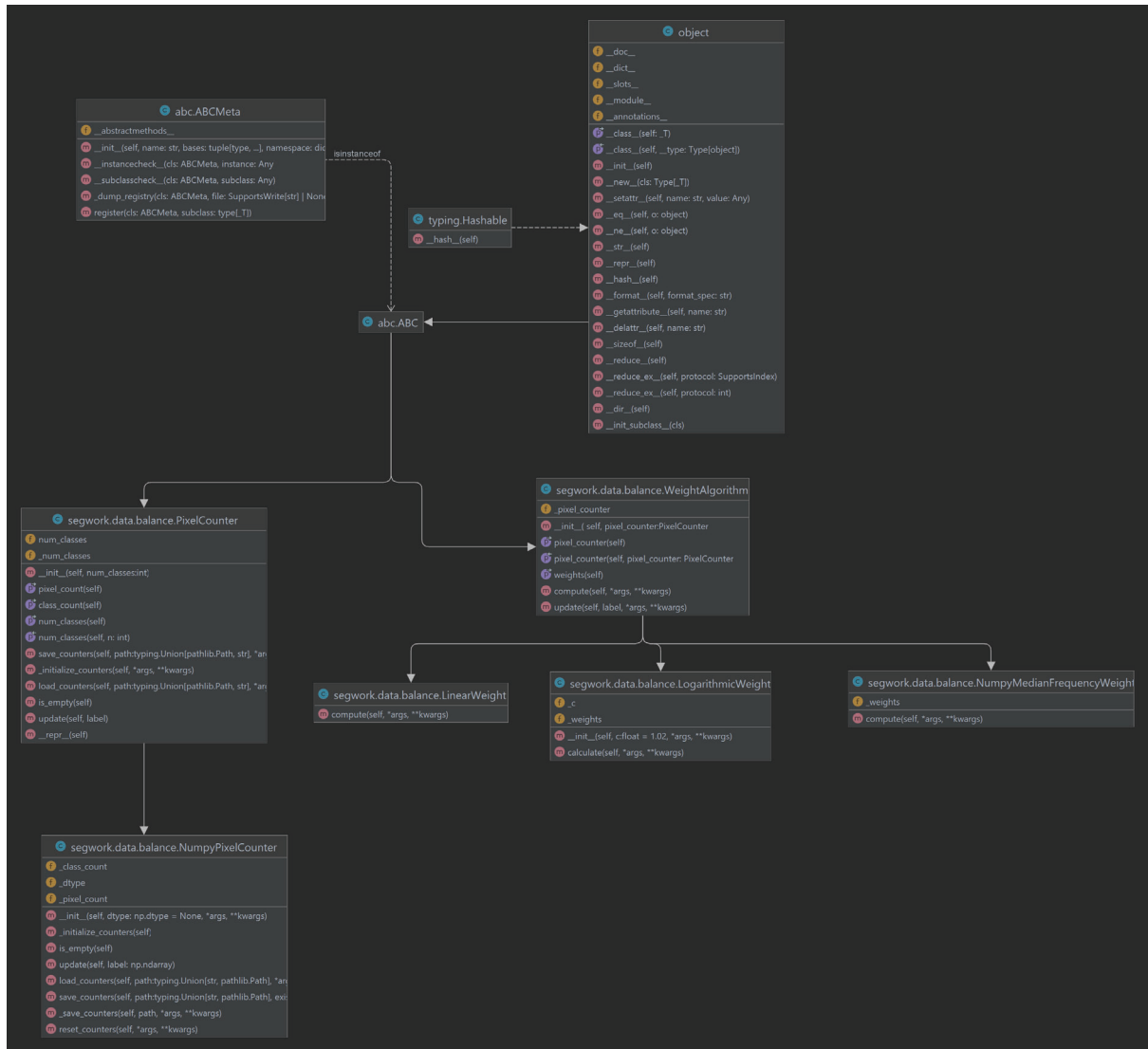


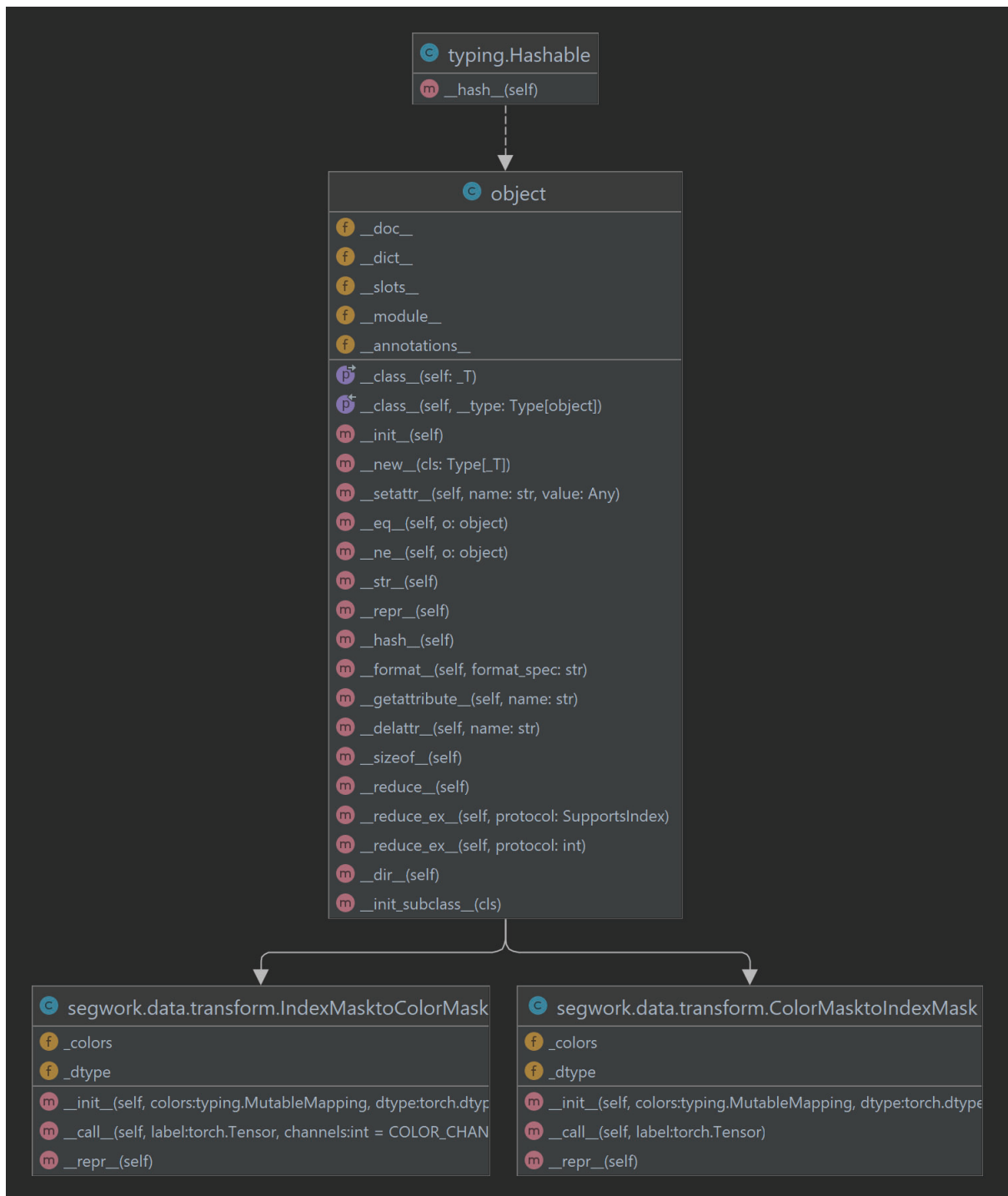
Figure 6.1: Diagram of registry.py from Pycharm using the Diagrams plugin



**Figure 6.2:** Diagram of dataset.py from Pycharm using the Diagrams plugin



**Figure 6.3:** Diagram of balance.py from Pycharm using the Diagrams plugin



**Figure 6.4:** Diagram of `transform.py` from PyCharm using the Diagrams plugin